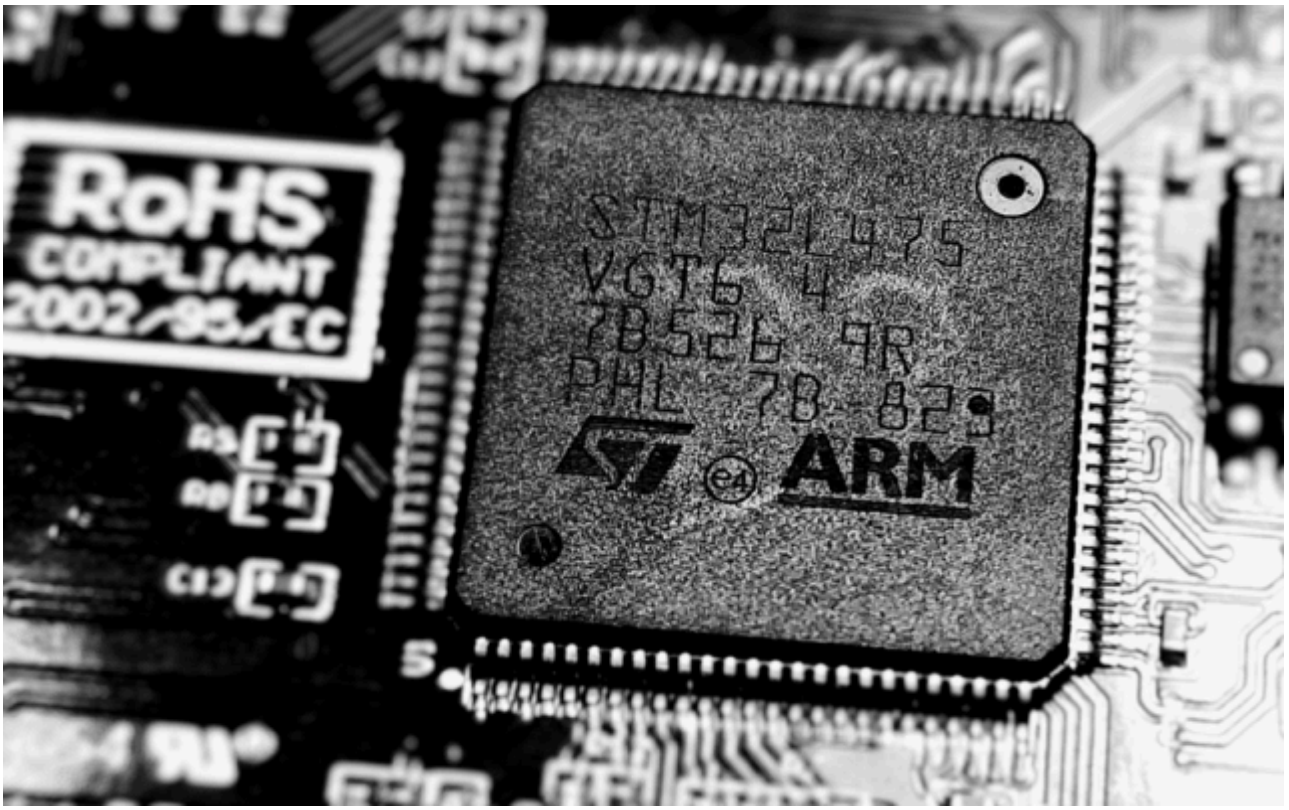
Perlatecnica
Research and Technological Education
ETS

The Heart of Embedded Systems

Getting Started

Theory, Code and Applications with STM32 Microcontrollers

Mauro D'Angelo



The Heart of Embedded Systems © 2026 by Mauro D'Angelo is licensed under Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/



Index

Introduction **7**
Chapter 1 Microcontrollers **9**
Introduction to Microcontrollers 9
The number of CPU bits 10
What does a microcontroller address? 11
How addressing works 11
The memory map 12
Applications 14
Alternate Functions of Microcontrollers 15
What Are Alternate Functions? 15
Why they are important 15
How They Work 15
Alternate functions of the STM32F401RE microcontroller 16
Physical Computing **20**
Sensors **21**
Example: Temperature Sensor on the IKS4A1 Expansion Board 22
The actuators **23**
Electric Actuators 23
Hydraulic Actuators 25
Pneumatic Actuators 25
Thermal Actuators 26
Digital signals **27**
Analog signals **28**
Analog to Digital Conversion **29**
Practical example: ADC on STM32F401RE 30
The main applications of the ADC 30
Communication protocols **31**
SPI - Serial Peripheral Interface 31
I2C 32
Serial 34
Let's recap **37**
Chapter 2 The Nucleo F401RE board **38**
STM32 Microcontroller Naming Convention 39
NucleoF401RE Hardware Features 39
Pin Labels 40
Installation 43
System Requirements 43
Download STM32CubeMX and STM32CubeIDE 44
Download STM32CubeMX 44
Download STM32CubeIDE 45
Registration and login 45
Installation on Windows 45

Installing STM32CubeMX 45
Installing STM32CubeIDE 45
Installation on macOS 46
Installation on Linux 47
Configuring STM32CubeIDE 48
Introduction to STM32CubeMX 49
The workflow with STM32CubeMX and STM32CubeIDE 50
Capitolo 3 GPIO output 52
Project Objectives 52
Basic Theory: What is a GPIO? 52
Setting Up the Project 53
Selecting the NUCLEO-F401RE board 54
BlinkLed Project Setup 55
Project Generation 56
Importing the project into STM32CubeIDE 56
The structure of an STM32CubeIDE project 57
The main project folders 58
The .ioc configuration file 60
Compiling the project in STM32CubeIDE 60
Microcontroller programming 62
He's running by hand. 63
Pin Configuration 64
Flashing LED 69
Chapter 4 Serial Communication 71
Project Objectives 71
Basic Theory: Asynchronous Serial Communication (UART) 71
Reception Management Mode 72
Setting Up the Serial Polling Project 72
USART2 Pin Configuration 73
Configuring the Serial Interrupt Project 80
USART2 Pin Configuration 80
Capitolo 5 GPIO input - polling 81
Project Objectives 81
Basic Theory: Reading a GPIO Input 81
Setting Up the Project 81
Configuring the User Button pin 82
Reading the Button 84
In-depth study: Pull-up/down resistors 85
Capitolo 6 GPIO input - interrupt 87
Project Objectives 87
Basic Theory: Generating an Interrupt 87
Setting Up the Project 89
Configuring the User Button pin 89
Button Management 91

Deep Dive: Interrupt Priority - Basic Concepts 92
Chapter 7 Analog Input - ADC 94
Project Objectives 94
Basic Theory: Reading an Analog Signal 94
What is an ADC (Analog-to-Digital Converter)? 94
What is an ADC channel? 95
Setting Up the Project 96
A0 pin configuration 96
Chapter 8 Timers 101
Project Objectives 101
Basic Theory: What is a Timer? 101
Setting Up the Project 102
Chapter 9 PWM 108
Project Objectives 108
Basic Theory: What is PWM 108
Setting Up the Project 110
A5 pin configuration 110
Example 1 112
Example 2: 114

Perlatecnica is a nonprofit social promotion organization founded in 2014 by a group of engineering and computer science enthusiasts, with the goal of spreading digital culture through training, research, and outreach. Its volunteers—engineers, computer scientists, and enthusiasts—organize seminars, workshops, and online courses, creating freely accessible teaching materials and e-learning platforms.

With this book, we pursue the same mission: to make knowledge of STM32 microcontrollers and the basic technologies of electronics and physical computing accessible to everyone, from students to professionals. Through practical examples and step-by-step guides, the aim is to transform theoretical concepts into operational skills, stimulating the creativity and autonomy of those approaching these topics for the first time.

https://www.perlatecnica.it/

Introduction

This book is designed to guide you from theory to initial practical experience, with exercises and ready-to-use code snippets. The goal is that, by the end of the book, you will not only understand the operating principles of STM32 microcontrollers, but also be able to independently apply them to your own projects. A book can never be exhaustive on the subject matter covered, but a book like this can provide the reader with the right insights for conducting personal research and in-depth study.

The code for the examples is available at the following link:

https://github.com/Perlatecnica/TheHeartOfEmbeddedSystems/tree/master/GettingStarted/WS_CoreOfEmbeddedSystems

In the following chapters you will find:

- **Chapter 1: Microcontrollers**
  Introduction to microcontrollers, CPU bit count, addressing and memory map, alternate functions and their importance, sensors and actuators, digital and analog signals, A/D conversion and communication protocols (SPI, I2C, Serial).

- **Chapter 2: The Nucleo F401RE board**
  STM32 naming convention, Nucleo-F401RE hardware features, pin labels, installation and configuration ofSTM32CubeIDEand introduction to CubeMX.

- **Capitolo 3: GPIO output**
  GPIO theory, project setup, compilation and programming, and LED flashing.

- **Chapter 4: Serial Communication**
  Theory of how serial communication works, project configuration, compilation and programming, LED management via serial command.

- **Capitolo 5: GPIO input – polling**
  Reading a configured GPIO as input, handling the user button in polling, and learning more about pull-up/pull-down resistors.

- **Capitolo 6: GPIO input – interrupt**
  Generating and managing interrupts from digital pins, configuring and prioritizing interrupts.

- **Chapter 7: Analog Input – ADC**
  Analog to digital conversion theory, ADC configuration on STM32F401RE and main applications.

- **Chapter 8: Timers**
  Timer basics, configuration, and practical use for generating timed events.

- **Chapter 9: PWM**
  Pulse width modulation fundamentals, PWM configuration on STM32 and application examples.

Chapter 1 Microcontrollers

Introduction to Microcontrollers

And**microcontroller**It is an integrated electronic device that combines a processor (CPU), memory, and input/output (I/O) peripherals on a single chip. Microcontrollers are designed for embedded applications, i.e., electronic systems designed to perform specific tasks.

A microcontroller is made up of:

- **CPU (Central Processing Unit):** It executes the program's instructions. It is typically based on architectures such as ARM Cortex, AVR, or RISC.
- **Memory:**
  - **Flash:**To store the program.
  - **RAM:**For temporary data.
  - **EEPROM (optional):**For permanent and editable data
- **Peripherals:**
  - **GPIO:**General purpose input/output pins for communicating with external devices.
  - **ADC/DAC:**Signal converters, from analog to digital and vice versa.
  - **Timer:**For timing and counting functions.
  - **Communication buses - UART, I2C, SPI, CAN:**For internal communications or with external devices.
- **Oscillator:**
  - Generates the clock to synchronize the operations of the microcontroller.

The characteristics on which microcontrollers are typically compared are:

- **Resolution:**That is the nCPU bit number (eg. 8-bit, 16-bit, 32-bit).
- **Clock frequency:**Operating speed of the microcontroller, measured in MHz or GHz.
- **Energy consumption:**Essential for battery-powered or IoT applications.

The number of CPU bits

The **CPU bit count**A microcontroller's bit size (e.g., 8-bit, 16-bit, 32-bit) indicates the size of data it can process in a single instruction cycle, as well as the width of its internal registers and, typically, the architecture of the processing unit. It is a fundamental parameter that affects the microcontroller's performance, capabilities, and efficiency.

This number represents:

- **Size of operations:**
  - A CPU at**8-bit**It can process numbers or instructions of up to 8 bits (1 byte) at a time.
  - A CPU at**16-bit**the**32-bit**can process larger numbers or instructions (up to 16 or 32 bits), making it more efficient for complex calculations.
- **Width of registers:**
  - The CPU's internal registers are the same width as the CPU's number of bits.
  - For example, a CPU with32-bitIt will have 32-bit registers to store data and addresses.
- **Memory Address Management:**A CPU with a larger number of bits can handle a wider address range. For example, an 8-bit CPU can address

  $$2^8 = 256$$

  memory locations, while a 32-bit CPU can address

  $$2^{32} = 4.294.967.296$$

  that is, approximately 4 billion rentals.

The following table shows a comparison between the various architectures

Characteristic	8-bit	16-bit	32-bit
Data size	Up to 8 bit	Up to 16 bits	Up to 32 bit
Memory addresses	256 locations	65,536 locations	4 billion in rentals
Efficiency in calculations	Limited	Improve	Excellent
Examples of use	Simple systems	Basic Automation	Advanced applications

For example, an 8-bit microcontroller, such as the**ATmega328**(used in Arduino Uno), is ideal for simple tasks like turning on an LED or reading a sensor. However, a 32-bit

microcontroller, such as the**STM32F401RE**, is best suited for complex applications, such as motor control or real-time signal processing.

In short, the CPU bit count is a crucial parameter that determines the computing power, speed, and applications for which the microcontroller is best suited.

What does a microcontroller address?

When talking about**addressing**In a microcontroller, this refers to the CPU's ability to access specific memory locations. The number of bits used for addressing determines the amount of memory that can be addressed, or "reached," by the CPU. A microcontroller's memory can be:

- **Program Memory (Flash):**Contains the executable program code. The CPU needs to be able to access the instructions to execute the code, so it uses the addressing system to locate each instruction.
- **Data Memory (RAM):**Used to store temporary data, variables, and stacks. Addressing allows the CPU to read and write data to specific RAM locations.
- **Peripheral Memory (I/O):**It includes registers for peripherals such as UART, ADC, GPIO, etc. These registers are mapped to a portion of the memory address, and the CPU manages them as part of the addressing system.
- **EEPROM (optional):**For storing permanent data or configurations. This memory can also be addressed by the CPU.

A microcontroller like the**STM32F401RE**has a 32-bit CPU, which means that itsaddress busIt is 32 bits wide. This allows it to generate addresses between**0x00000000**and**0xFFFFFFFF**,that is 2^{32}memory locations, for a total of**4 GB**addressable. However, the available physical memory is much less than this theoretical limit.

- **Flash Memory (Program Memory):** 512 KB, da 0x08000000 a 0x0807FFFF.
- **RAM (Data Memory):** 96 KB, da 0x20000000 a 0x20017FFF.
- **Peripherals:**Mapped to a specific range (e.g., GPIO, UART).
- **System Memory:**A reserved area for the bootloader and other system functions.

How addressing works

The CPU accesses these memories using addresses. Each address identifies a specific location or register. Let's look at some step-by-step examples.

Suppose the microcontroller needs to read a variable stored in RAM. The variable is located at a specific address, for example0x20000004.

- **Address generation:**The CPU calculates or receives the address0x20000004and sends it to theaddress bus.
- **Memory access:**The control unit checks that the address is within the RAM range (from0x20000000 a 0x20017FFF). If the address is valid, it accesses the corresponding memory location.
- **Data recovery:**The CPU reads the data present in0x20000004and stores it in an internal log for further processing.

Now suppose the microcontroller needs to write an instruction into the Flash Memory, at the address 0x08000010:

- **Generated address:**The CPU sends the address0x08000010to the address bus.
- **Access to Flash:**The system checks that the address is in the Flash range (from0x08000000 a 0x0807FFFF). If yes, it allows writing (provided the Flash is in writable mode).
- **Data writing:**The instruction is written to the specified location.

Suppose instead that I want to control a peripheral, for example an LED controlled by a GPIO pin, and the GPIO register is at address0x40020014.

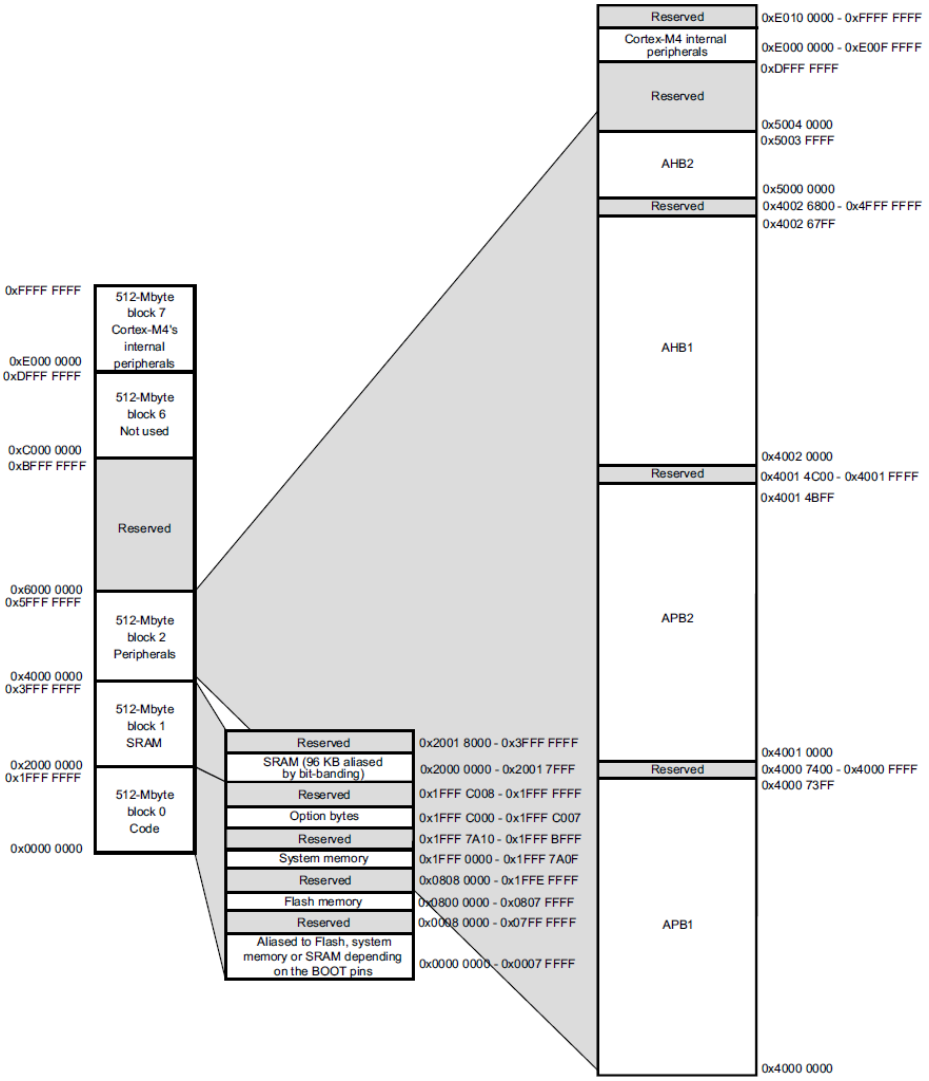
- **GPIO register address:**The CPU generates the address0x40020014, which corresponds to a GPIO register for setting the state of a pin.
- **Writing in the register:**The CPU writes a value (e.g.,1to turn on the LED) in the register.

The memory map

In the STM32F401RE, different memory areas are mapped to a specific range:

Address Range	Memory Type	Description
0x08000000 - 0x0807FFFF	Flash Memory	Program Memory (512 KB)
0x20000000 - 0x20017FFF	RAM	Data Memory (96 KB)
0x40000000 - 0x500607FF	Peripherals	Peripheral registers (GPIO, UART, etc.)
0x1FFF0000 - 0x1FFF77FF	System Memory	Bootloader and other system functions

The following figure, taken from the STM32F401RE microcontroller datasheet, explores the device's memory map in detail.



In practice, addressing allows the CPU to manage all components (RAM, Flash, peripherals) using a single address system. Each operation is driven by the microcontroller's logic and mapped to a portion of the address bus, making access to memory and devices simple and uniform.

Applications

The most common applications of microcontrollers are:

- **Home automation:**Control lights, thermostats, smart locks.
- **Automotive:**Engine control systems, airbags, ABS.
- **Industry:**Automation of processes and machinery.
- **IoT (Internet of Things):**Connected devices for data collection and analysis.
- **Consumer Electronics:**Home appliances, interactive toys, smart watches.

Microcontrollers are the foundation of modern embedded electronics, with applications ranging from home automation to industry. Understanding their structure and programming is essential for developing innovative technological solutions.

Alternate Functions of Microcontrollers

Modern microcontrollers are extremely versatile devices, featuring numerous pins that can be configured to perform different functions. These multiple configurations, often called*alternate functions*,allow the microcontroller to adapt to a wide range of applications, optimizing the use of available hardware resources.

What are theAlternate Functions

A microcontroller pin isn't limited to just one function. In addition to its basic function (typically a digital input or output), many pins can be used for other functions, such as:

- **Analog Inputs/Outputs** (ADC o DAC)
- **Serial communications** (UART, SPI, I2C, CAN, ecc.)
- **PWM (Pulse Width Modulation)**to control motors or LEDs
- **Timer inputs**
- **Special functions of the microcontroller**, come clock o debug

These alternative functions are configurable through internal control registers of the microcontroller.

Why they are important

Thealternatefunctions allow you to:

- **Optimize your use of pins:**A microcontroller with a limited number of pins can offer a lot of functionality thanks to dynamic configuration.
- **Reduce system size:**By implementing more functionality on fewer pins, PCB size and manufacturing costs are reduced.
- **Provide design flexibility:**The developer can choose how to configure the pins to best suit the application.

How They Work

Thealternatefunctions are managed via:

- **GPIO Configuration Registers:**Each pin has one or more configuration registers associated with it. These registers specify:
    - Pin mode (digital, analog, etc.)
    - Pin speed
    - Pull-up/pull-down resistors
    - Alternative function selected
- **Mapping of Alternate Functions:**Each pin has a mapping table that specifies which alternate functions are available. For example, the microcontroller datasheet provides a matrix listing the alternate functions for each pin.

- **Internal Multiplexer:**Inside the microcontroller, the pins are connected to a hardware multiplexer. The configuration of the registers determines which function is enabled for that pin.

ThealternateFunctions are a fundamental aspect of getting the most out of a microcontroller. Correct configuration is essential to ensure optimal application performance. Developers should always consult the datasheet and use software tools (such as HALs or specific drivers) to configure these features efficiently.

Alternate functions of the STM32F401RE microcontroller

For the convenience of the reader, the table of all thealternateFunctions offered by each pin of the STM32F401RE microcontroller. The tables are extracted from the datasheet, which we recommend consulting.

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port A	PA0	-	TIM2_CH1/ TIM2_ETR	TIM5_CH1	-	-	-	-	USART2_ CTS	-	-	-	-	-	-	-	EVENT OUT
	PA1	-	TIM2_CH2	TIM5_CH2	-	-	-	-	USART2_ RTS	-	-	-	-	-	-	-	EVENT OUT
	PA2	-	TIM2_CH3	TIM5_CH3	TIM9_CH1	-	-	-	USART2_ TX	-	-	-	-	-	-	-	EVENT OUT
	PA3	-	TIM2_CH4	TIM5_CH4	TIM9_CH2	-		-	USART2_ RX	-	-	-	-	-	-	-	EVENT OUT
	PA4	-	-	-	-	-	SPI1_NSS	SPI3_NSS/ I2S3_WS	USART2_ CK	-	-	-	-	-	-	-	EVENT OUT
	PA5	-	TIM2_CH1/ TIM2_ETR	-	-	-	SPI1_SCK	-	-	-	-	-	-	-	-	-	EVENT OUT
	PA6	-	TIM1_BKIN	TIM3_CH1	-	-	SPI1_ MISO	-	-	-	-	-	-	-	-	-	EVENT OUT
	PA7	-	TIM1_CH1N	TIM3_CH2	-	-	SPI1_ MOSI	-	-	-	-	-	-	-	-	-	EVENT OUT
	PA8	MCO_1	TIM1_CH1	-	-	I2C3_SCL	-	-	USART1_ CK	-	-	OTG_FS_ SOF	-	-	-	-	EVENT OUT
	PA9	-	TIM1_CH2	-	-	I2C3_ SMBA	-	-	USART1_ TX	-	-	OTG_FS_ VBUS	-	-	--	-	EVENT OUT
	PA10	-	TIM1_CH3	-	-	-	-	-	USART1_ RX	-	-	OTG_FS_I D	-	-	-	-	EVENT OUT
	PA11	-	TIM1_CH4	-	-	-	-	-	USART1_ CTS	USART6_ TX	-	OTG_FS_ DM	-	-	-	-	EVENT OUT
	PA12	-	TIM1_ETR	-	-	-	-	-	USART1_ RTS	USART6_ RX	-	OTG_FS_ DP	-	-	-	-	EVENT OUT
	PA13	JTMS_ SWDIO	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PA14	JTCK_ SWCLK	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PA15	JTDI	TIM2_CH1/ TIM2_ETR	-	-	-	SPI1_NSS	SPI3_NSS/ I2S3_WS	-	-	-	-	-	-	-	-	EVENT OUT

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port B	PB0	-	TIM1_CH2N	TIM3_CH3	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB1	-	TIM1_CH3N	TIM3_CH4	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB3	JTDO-SWO	TIM2_CH2	-	-	-	SPI1_SCK	SPI3_SCK/ I2S3_CK	-	-	I2C2_SDA	-	-	-	-	-	EVENT OUT
	PB4	JTRST	-	TIM3_CH1	-	-	SPI1_ MISO	SPI3_MISO	I2S3ext_S D	-	I2C3_SDA	-	-	-	-	-	EVENT OUT
	PB5	-	-	TIM3_CH2	-	I2C1_ SMBA	SPI1 _MOSI	SPI3_MOSI/ I2S3_SD	-	-	-	-	-	-	-	-	EVENT OUT
	PB6	-	-	TIM4_CH1	-	I2C1_SCL	-	-	USART1_ TX	-	-	-	-	-	-	-	EVENT OUT
	PB7	-	-	TIM4_CH2	-	I2C1_SDA	-	-	USART1_ RX	-	-	-	-	-	-	-	EVENT OUT
	PB8	-	-	TIM4_CH3	TIM10_CH1	I2C1_SCL	-	-	-	-	-	-	-	SDIO_ D4	-	-	EVENT OUT
	PB9	-	-	TIM4_CH4	TIM11_CH1	I2C1_SDA	SPI2_NSS/I 2S2_WS	-	-	-	-	-	-	SDIO_ D5	-	-	EVENT OUT
	PB10	-	TIM2_CH3	-	-	I2C2_SCL	SPI2_SCK/I 2S2_CK	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB12	-	TIM1_BKIN	-	-	I2C2_ SMBA	SPI2_NSS/I 2S2_WS	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB13	-	TIM1_CH1N	-	-	-	SPI2_SCK/I 2S2_CK	-	-	-	-	-	-	-	-	-	EVENT OUT
	PB14	-	TIM1_CH2N	-	-	-	SPI2_MISO	I2S2ext_SD	-	-	-	-	-	-	-	-	EVENT OUT
	PB15	RTC_ REFN	TIM1_CH3N	-	-	-	SPI2_MOSI /I2S2_SD	-	-	-	-	-	-	-	-	-	EVENT OUT

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port C	PC0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC2	-	-	-	-	-	SPI2_ MISO	I2S2ext_SD	-	-	-	-	-	-	-	-	EVENT OUT
	PC3	-	-	-	-	-	SPI2_MOSI /I2S2_SD	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC6	-	--	TIM3_CH1	-	-	I2S2_MCK	-	-	USART6_ TX	-	-	-	SDIO_ D6	-	-	EVENT OUT
	PC7	-		TIM3_CH2	-	-	-	I2S3_MCK	-	USART6_ RX	-	-	-	SDIO_ D7	-	-	EVENT OUT
	PC8	-	-	TIM3_CH3	-	-	-	-	-	USART6_ CK	-	-	-	SDIO_ D0	-	-	EVENT OUT
	PC9	MCO_2	-	TIM3_CH4	-	I2C3_SDA	I2S_CKIN	-	-	-	-	-	-	SDIO_ D1	-	-	EVENT OUT
	PC10	-	-	-	-	-	-	SPI3_SCK/ I2S3_CK	-	-	-	-	-	SDIO_ D2	-	-	EVENT OUT
	PC11	-	-	-	-	-	I2S3ext_ SD	SPI3_MISO	-	-	-	-	-	SDIO_ D3	-	-	EVENT OUT
	PC12	-	-	-	-	-	-	SPI3_MOSI/ I2S3_SD	-	-	-	-	-	SDIO_ CK	-	-	EVENT OUT
	PC13	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC14	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PC15	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port D	PD0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD2	-	-	TIM3_ETR	-	-	-	-	-	-	-	-	-	SDIO_ CMD	-	-	EVENT OUT
	PD3	-	-	-	-	-	SPI2_SCK/ I2S2_CK	-	USART2_ CTS	--	-	-	-	-	-	-	EVENT OUT
	PD4	-	-	-	-	-	-	-	USART2_ RTS		-	-	-	-	-	-	EVENT OUT
	PD5	-	-	-	-	-	-	-	USART2_ TX	-	-	-	-	-	-	-	EVENT OUT
	PD6	-	-	-	-	-	SPI3_MOSI /I2S3_SD	-	USART2_ RX	-	-	-	-	-	-	-	EVENT OUT
	PD7	-	-	-	-	-	-	-	USART2_ CK	-	-	-	-	-	-	-	EVENT OUT
	PD8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD10	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD11	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD12	-	-	TIM4_CH1	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD13	-	-	TIM4_CH2	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD14	-	-	TIM4_CH3	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PD15	-	-	TIM4_CH4	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port E	PE0	-	-	TIM4_ETR	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE1	-	TIM1_CH2N	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE2	TRACECL K	-	-	-	-	SPI4_SCK	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE3	TRACED0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE4	TRACED1	-	-	-	-	SPI4_NSS	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE5	TRACED2	-	-	TIM9_CH1	-	SPI4_MISO	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE6	TRACED3	-	-	TIM9_CH2	-	SPI4_MOSI	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE7	-	TIM1_ETR	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE8	-	TIM1_CH1N	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE9	-	TIM1_CH1	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE10	-	TIM1_CH2N	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE11	-	TIM1_CH2	-	-	-	SPI4_NSS	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE12	-	TIM1_CH3N	-	-	-	SPI4_SCK	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE13	-	TIM1_CH3	-	-	-	SPI4_MISO	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE14	-	TIM1_CH4	-	-	-	SPI4_MOSI	-	-	-	-	-	-	-	-	-	EVENT OUT
	PE15	-	TIM1_BKIN	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT

Port		AF00	AF01	AF02	AF03	AF04	AF05	AF06	AF07	AF08	AF09	AF10	AF11	AF12	AF13	AF14	AF15
		SYS_AF	TIM1/TIM2	TIM3/ TIM4/ TIM5	TIM9/ TIM10/ TIM11	I2C1/I2C2/ I2C3	SPI1/SPI2/ I2S2/SPI3/ I2S3/SPI4	SPI2/I2S2/ SPI3/ I2S3	SPI3/I2S3/ USART1/ USART2	USART6	I2C2/ I2C3	OTG1_FS		SDIO			
Port H	PH0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT
	PH1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT OUT

Physical Computing

The *Physical Computing*It is the intersection between the physical and digital worlds, where electronic devices, such as microcontrollers, are used to interact with the environment through sensors, actuators, and input/output devices. This approach is essential for developing interactive applications, innovative prototypes, and IoT solutions.

Microcontrollers are essential components of physical computing due to their ability to:

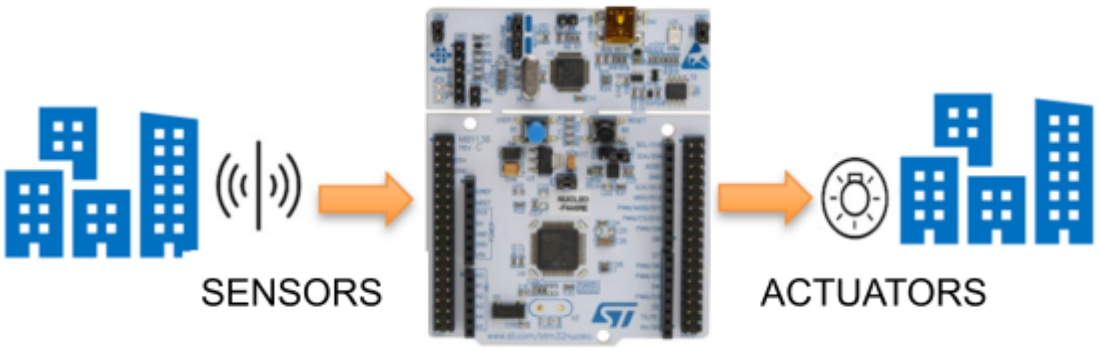
- **Detect the environment**through sensors (e.g. temperature, light, pressure).
- **Process data**coming from the sensors.
- **Interacting with the environment**via actuators (e.g. motors, LEDs, speakers).
- **Communication**with other devices via standard protocols (e.g. I2C, SPI, UART, Wi-Fi).

A physical computing system is based on three main components:

- **Input:**
    - Sensors to detect physical or environmental data (e.g. temperature, humidity, light).
    - Input devices such as buttons, potentiometers, joysticks, or touch sensors.
- **Processing:**
    - Microcontroller that processes data and decides what actions to take.
    - Algorithms programmed to analyze and respond to received inputs.
- **Output:**
    - Actuators to respond to inputs (e.g. motors, solenoid valves, relays).
    - Visual or audible feedback (e.g. LED, LCD display, buzzer).

Examples of Physical Computing Projects are:

- **Weather StationIoT:**
    - **Input:**Temperature, humidity and pressure sensors.
    - **Processing:**Calculating environmental data and sending it to the cloud.
    - **Output:**View on LCD display or online dashboard.
- **Robot Mobile:**
    - **Input:**Distance sensors (e.g. ultrasonic) and control buttons.
    - **Processing:**Obstacle avoidance algorithms.
    - **Output:**Controlling motors for movement.
- **Smart Home:**
    - **Input:**PIR sensors to detect motion and smart switches.
    - **Processing:**Automatic on/off logic.
    - **Output:**Control lights, shutters or connected devices.



So the sensors perceive the environment and provide this information to the microcontroller, which, based on the program being run, can perform actions via the actuators.

Physical computing with microcontrollers offers endless possibilities for creating innovative solutions that interact with the physical world. Understanding the basic principles and experimenting with practical projects is the best way to develop skills in this field.

Sensors

A sensor is a device that detects physical or chemical changes in the environment and converts them into electrical signals that can be read by an electronic system, such as a microcontroller. Sensors are used in a wide range of applications, from industrial automation to the Internet of Things (IoT), to gather information about the surrounding environment.

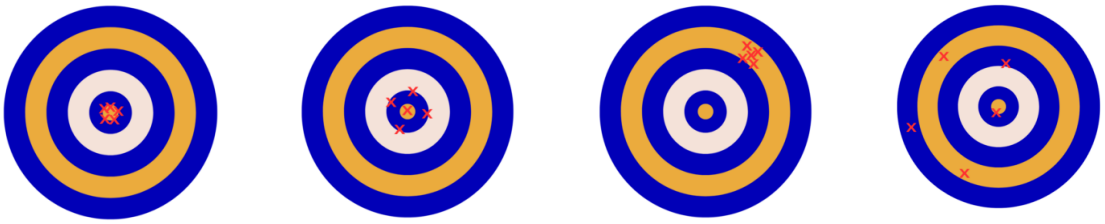


A sensor has the following main characteristics:

- **Measured Quantity**: It is the physical or chemical variable that the sensor detects, such as temperature, humidity, pressure, light, position, gas, or force.
- **Exit**The detected quantity is converted into an electrical signal, which can be:
  - **Analog**: A continuous signal (for example, a voltage value proportional to the measured quantity).
  - **Digital**: A discrete signal, often transmitted using protocols such as I2C, SPI, or UART.
- **Range**: The range of values that the sensor can measure.
- **Accuracy and Resolution**:
  - **Accuracy**: The sensor's ability to provide a value close to the real one.

    - **Precision**: the ability to repeatedly indicate the same measurement value, regardless of whether or not it corresponds to the true value.
    - **Resolution**: The level of detail with which the sensor detects changes.
- **Sensitivity**: The ability of the sensor to respond to small changes in the measured quantity.

The target example helps to easily understand the meanings of accuracy and precision of a sensor



High accuracy
High precision

High accuracy
Low precision

Low accuracy High
precision

Low accuracy Low
precision

Example: Temperature Sensor on the IKS4A1 Expansion Board

The **X-NUCLEO-IKS4A1**It is an expansion board designed for the STM32 Nucleo platforms, equipped with advanced sensors for motion detection and environmental monitoring. Among the integrated sensors are theSTTS22H,a high-precision digital sensor for measuring temperature.

The Technical Features of theSTTS22H:

- **Temperature Measurement Range**: from -40 °C to +125 °C.
- **Accuracy**: ±0.5°C within the standard operating range.
- **Resolution**: 16 bits (up to 0.0625°C per increment).
- **Communication Interface**:
    - **I²C**with configurable slave address.
    - Supports speeds up to 1 MHz (I²C Fast Mode Plus).
- **Energy Consumption**:
    - **1,75 µA**in active mode (at 1 Hz).
    - **0,5 µA**in standby mode.

The sensorSTTS22HIt measures temperature using a diode junction system. Temperature variations affect the electrical behavior of the diode, and the sensor processes these variations to provide highly accurate digital values.

The sensorSTTS22H easily integrates with any STM32 Nucleo board thanks to layout compatibilityArduino Uno R3present on theX-NUCLEO-IKS4A1Communication takes place via I²C bus.

So in summary, the sensor perceives the temperature through a change in the electrical behavior of the diode and makes the temperature data available through an I²C protocol.

The actuators

The **actuators**Actuators are devices that transform a control signal, usually electrical, into a physical action. They are used to perform movements, apply forces, or generate specific effects such as light, heat, or sound. Actuators represent the "active" part of an automation or control system, interacting directly with the surrounding environment.

Actuators can be classified according to:

- **Energy used:**
  - **Electrics:**Motors, electromagnets, solenoids.
  - **Plumbers:**Pistons and cylinders powered by pressurized fluids.
  - **Tires:**Actuators powered by compressed air.
- **Type of movement:**
  - **Linear:**They produce a rectilinear movement (e.g. pistons).
  - **Rotary:**They produce a rotary movement (e.g. DC motors, steppers).
- **Function performed:**
  - **Positioning:**To adjust the position of an object.
  - **By force:**To apply a specific force.

Electric Actuators

Electric actuators use electrical energy to generate movement or physical effects. Examples of these types of actuators include:

- **DC (Direct Current) Motors:**



- Continuous rotary motion.
- Simple to control via PWM signals.
- Used in robotics, vehicles, household appliances.

- **Stepper Motors:**



- Rotary movement with discrete steps.
- Ideal for applications requiring precise position control, such as 3D printers or CNC machines.

- **Servomotors:**
  - They provide precise control of position, speed and torque.
  - They combine a DC motor, a gearbox and a control system.
  - Used inarms roboticsand automation.



- **Solenoidi:**



- They generate linear motion when powered.
- Used in electronic locks, valves and locking mechanisms.

Hydraulic Actuators

- **Characteristics:**
  - High strength and robustness.
  - Ideal for heavy industrial applications (e.g. construction machinery).
- **Examples:**
  - Hydraulic cylinders for lifting and compression.
  - Hydraulic steering systems in vehicles.

Pneumatic Actuators



Pneumatic actuators use compressed air to generate movement.

- **Characteristics:**
  - Fast, cheap and safe.
  - Suitable for industrial environments and automated systems.
- **Examples:**
  - Pneumatic pistons for assembly machinery.

    - Pneumatically controlled valves.

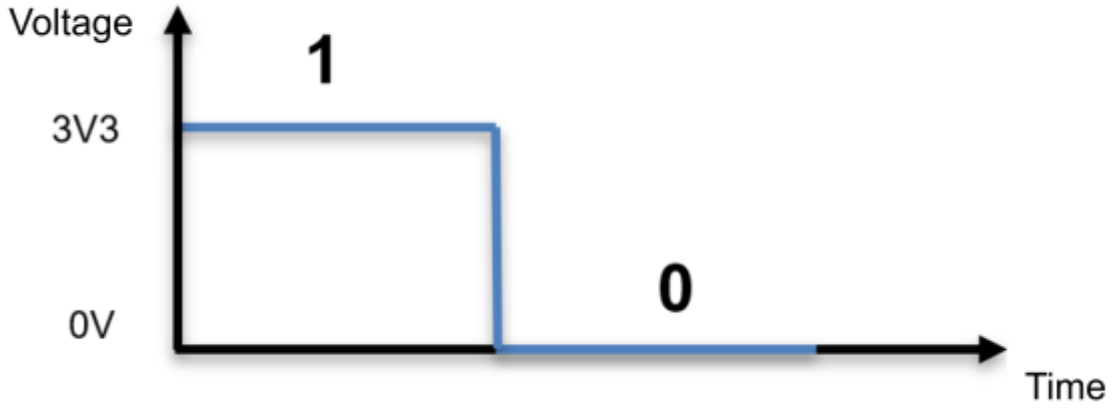
Thermal Actuators

They use temperature variations to generate movement or changes in state.

- **Examples:**
    - Wax actuators for thermostats.
    - Piezoelectric elements that deform in response to heat.

Digital signals

A digital signal is a type of signal that represents information using discrete values, typically binary (0 and 1). Unlike analog signals, which vary continuously, a digital signal only assumes a finite number of states, making it easier to process and transmit in electronic systems.



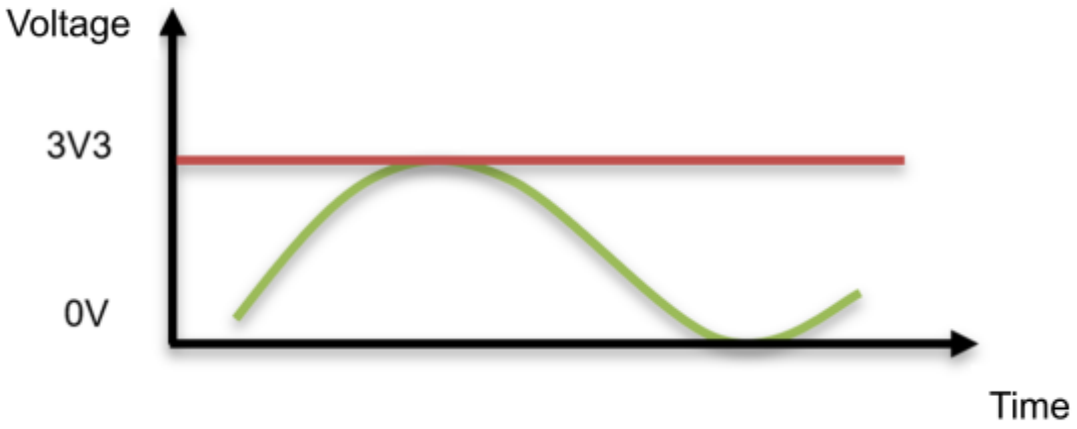
The main features are:

- **Discrete values:**Usually represented by voltage levels, for example:
  - 0: Low voltage (e.g. 0V).
  - 1: High voltage (e.g. 3.3V or 5V).
- **Noise immunity:**Digital signals are less sensitive to noise than analog signals, since the system detects only two distinct states.
- **Ease of processing:**They are used in digital systems, such as microcontrollers and processors, where logical and arithmetic operations can be performed precisely.

Digital signals are fundamental to communication and processing in modern electronic systems, including computers, IoT devices, and telecommunication networks.

Analog signals

An analog signal is a type of signal that varies continuously over time, representing information across an infinite range of possible values within a given interval. It is commonly used to represent physical phenomena such as sound, light, or temperature.



The main features are:

- **Continuity:**The signal value can take any value within the specified range.
- **Noise sensitivity:**Analog signals are more prone to noise and distortion than digital signals.
- **Natural representation:**They are ideal for representing continuously varying physical phenomena.

Examples of analog signals include voice captured by a microphone, light waves detected by an optical sensor, and voltage in a variable circuit.

Analog to Digital Conversion

At this point, it's clear that the microcontroller operates in the "digital world" but interacts with an "analog world." To enable this interaction, devices capable of converting signals are required.

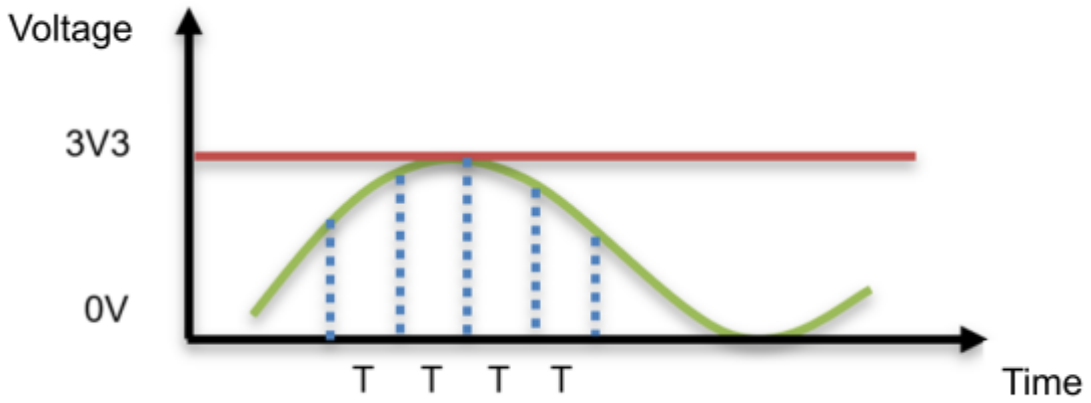
Analog-to-digital conversion (ADC) is the process of transforming a continuous analog signal into a discrete digital signal, which takes on a finite number of encoded values and can be processed by digital electronic systems such as microcontrollers or computers. This process is essential for acquiring and analyzing information from the physical world, such as temperature, light, or sound.

The ADC conversion process includes three main stages:

- **Sampling:**
  - The analog signal is measured at regular time intervals, defined by the sampling rate.
  - The sampling rate must be at least twice the maximum frequency of the signal, according to the Nyquist theorem, to avoid aliasing.
- **Quantizzazione (Quantization):** The continuous values of the sampled signal are mapped to a finite number of discrete levels, determined by the resolution of the ADC. For example, a 12-bit ADC divides the analog range into:

$$2^{12} = 4096 \text{levels}$$

- **Codifica (Encoding):** Each quantized value is represented as a binary number, readable by the digital system.



The main characteristics of an ADC are:

- **Resolution:**Indicates the number of bits used to represent digital values. For example, a 12-bit ADC has a resolution of 1/4096 of the total range.

$$Step = \frac{Vref}{2^{12}} = \frac{Vref}{4096}$$

- **Voltage Range:**The voltage range that the ADC can measure (e.g. 0-3.3V or ±5V).
- **Accuracy:**The difference between the actual value of the analog signal and the converted digital value.
- **Sampling Rate:**The speed at which the ADC can measure the signal, expressed in samples per second (SPS).
- **Quantization Error:**The difference introduced during the conversion between the continuous analog signal and the discrete digital signal.

Practical example: ADC on STM32F401RE

The microcontrollerThe STM32F401RE features a 12-bit ADC, which can convert an analog signal (for example a voltage between 0 and 3.3V) into a digital value between 0 and 4095.

The sampling rate is configurable and depends on the ADC clock frequency.

If the analog input voltage is 1.65V and the maximum reference voltage is 3.3V, the corresponding digital value is calculated as follows:

$$Digital\ value\ =\ \frac{Input\ Voltage}{Reference\ Voltage}\ *\ (2^{12}\ -\ 1)$$

$$Digital\ value\ =\ \frac{1.65}{3.3}\ *\ (4096\ -\ 1) \approx 2048$$

The digital value**2048** represents **1,65V**in the system.

The main applications of the ADC

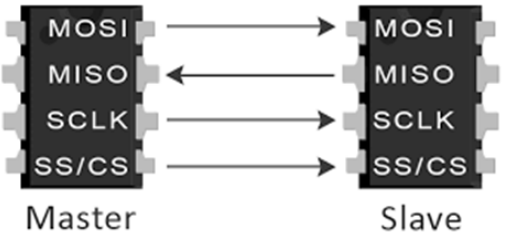
- **Audio:** Convert analog sound signals into digital data for recording or processing.
- **Sensors:**Measure physical quantities such as temperature, light, pressure, or other variables.
- **Imaging:** Convert signals from optical sensors into digital images.
- **Automation:**Control systems based on analog feedback, such as motors or HVAC systems (Heating, Ventilation & Air Conditioning).

The protocols ofcommunication

Communication with other devices connected to the microcontroller occurs via dedicated protocols that define the connection modes and communication rules. Below we examine the main protocols.

SPI - Serial Peripheral Interface

The SPI (Serial Peripheral Interface) protocol is a high-speed synchronous serial communication method used to exchange data between microcontrollers and peripheral devices such as sensors, displays, or memories. It operates in a master-slave configuration, with a master and one or more slaves.



The main features are:

- **Synchronous communication**: A clock (SCLK) synchronizes the data transfer between master and slave.
- **Full-duplex**Data is transmitted and received simultaneously on separate lines.
- **High speed**: It supports faster communication than other serial protocols such as I²C or UART.
- **Simple topology**: Relatively easy to implement, but requires more cabling than other protocols.

The main SPI lines are:

- **SCLK (Serial Clock)**: Generated by the master, synchronizes data transfer.
- **MOSI (Master Out Slave In)**: Line on which the master sends data to the slave.
- **MISO (Master In Slave Out)**: Line on which the slave sends data to the master.
- **SS/CS (Slave Select/Chip Select)**: Dedicated line to select which slave isactive.

It works according to the following steps:

- **Initial setup**The master sets the main parameters, such as clock speed, clock polarity (CPOL), clock phase (CPHA), and line roles. Each slave is connected to the master via dedicated lines (MOSI, MISO, SCLK), but each slave has its own SS/CS line.

- **Slave selection**The master drives the target slave's SS/CS line low (in the case of active-low logic). The other slaves ignore the communication by keeping their own SS/CS line high.
- **Data exchange**The master generates clock signals (SCLK) and transfers data via MOSI. The slave responds simultaneously by sending data via MISO. Data transfer occurs bit by bit, synchronized with the SCK signal.
- **End of communication**: The master brings the SS/CS line to a high level, signaling the end of communication with that slave.

Advantages of the SPI protocol:

- **High speed**: Transfer speeds that can reach tens of MHz.
- **Full-duplex**: Allows simultaneous two-way data transfer.
- **Simple hardware**: Easy to implement with dedicated lines.

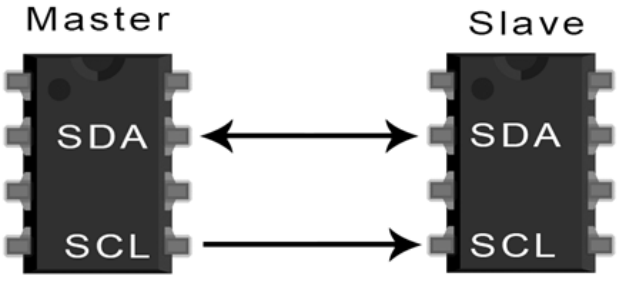
The disadvantages of the protocol are:

- **High pin usage**: Each slave requires a dedicated SS/CS line, which can become problematic with many devices.
- **No built-in addressing or error checking**: Requires additional management via software.

SPI is ideal for applications that require fast communication and a limited number of peripherals, such as sensors or high-speed memory devices.

I2C

The I²C (Inter-Integrated Circuit) protocol is a synchronous, multi-master, multi-slave serial communication protocol designed for low-speed communication between integrated circuits. It is widely used to connect microcontrollers to sensors, displays, and other peripheral devices.



The main features are:

- **Synchronous communication**: Data transfer is synchronized via a clock line (SCL).
- **Two-wire communication**: Requires only two lines, reducing pin usage compared to other protocols.
- **Address-based communication**: Each slave has a unique address, allowing communication with many devices using the same lines.

The main I²C lines are:

- **SCL (Serial Clock Line)**: The clock signal generated by the master to synchronize data transfer.
- **SDA (Serial Data Line)**: A two-way line used to send and receive data.
    - Both lines are open-drain and require pull-up resistors to maintain a default high state.

The operation is defined by the following steps:

- **Initial setup**The master configures the clock speed and defines communication parameters, such as addressing mode (7-bit or 10-bit).
- **Start condition**: The master generates a start condition by pulling the SDA line low while SCL remains high, signaling the start of a communication frame.
- **Addressing**:
    - The master sends the address of the target slave, followed by a read/write bit indicating the type of operation.
    - All devices on the bus listen, but only the addressed slave responds by lowering SDA (acknowledge or ACK).
- **Data transfer**:
    - Master and slave exchange data in 8-bit (byte) packets, each followed by an acknowledgment (ACK) or a no-acknowledge (NACK).
    - Clock pulses (SCL) control when data is read or written.
- **Stop condition**: The master generates a stop condition by releasing SDA high while SCL remains high, signaling the end of communication.

The I²C protocol offers the following advantages:

- **Low pin usage**: Only two lines are required for communication, regardless of the number of devices.
- **Multi-master capability**: Multiple masters can coexist on the same bus.
- **Addressing**Devices can share the same bus thanks to unique addresses, without additional lines.

The disadvantages are:

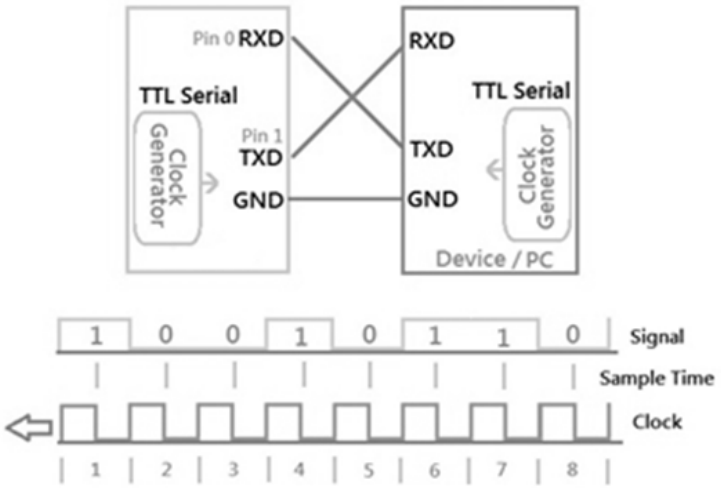
- **Slower than SPI**: Speeds limited to a few MHz, depending on the implementation.

- **Limited bus length**Due to capacity issues, the bus is only suitable for short distances.
- **Complexity**: Requires acknowledgment and addressing management, increasing software load.

I²C is ideal for applications requiring low to moderate transfer speeds and connecting multiple devices using a minimal number of wires, such as in embedded systems and IoT applications. Its simplicity and scalability make it a popular choice for communicating between components on a PCB.

Serial

The Universal Asynchronous Receiver-Transmitter (UART) protocol is a simple and widely used communication protocol that allows data to be transferred between devices, such as microcontrollers, sensors, or computers, using asynchronous serial communication. It operates without a clock signal and relies on a predefined configuration for synchronization.



The main features are:

- **Asynchronous communication**It does not require a clock signal from master to slave; synchronization occurs via start and stop bits. Both devices must know the communication speed.
- **Full-duplex**Data can be sent and received simultaneously on separate lines.
- **Point-to-point communication**: Typically used to directly connect two devices.
- **Configurable settings**Communication parameters, such as baud rate, parity, data bits and stop bits, must be agreed upon by both devices.

The main serial lines are:

- **TX (Transmit)**: Line on which the device sends data.
- **RX (Receive)**: Line on which the device receives data.
- **Optional control lines**: Like CTS (Clear to Send) and RTS (Request to Send) for hardware flow control.

The serial protocol operates in the following modes:

- **Configuration**: Both devices must agree on the communication parameters:
    - **Baud rate**: Communication speed in bits per second (e.g. 9600 bps).
    - **Bits in the date**: Number of bits in each data frame (e.g. 7 or 8 bits).
    - **Parity bit**: Optional bit for error checking (even, odd, or no parity).
    - **Bit on stop**: Number of bits that mark the end of a data frame (e.g. 1 or 2 bits).
- **Data Framing**Each byte of data is encapsulated in a frame constructed as follows:
    - **Bit di start**: A single bit (low level) that signals the start of the frame.
    - **Bits in the date**: Real data (e.g. 8 bit).
    - **Optional parity bit**: For error checking.
    - **Bit on stop**: One or more bits (high level) that mark the end of the frame.
- **Data transmission**:
    - The transmitter sends the encapsulated data frame by frame, bit by bit.
    - The receiver samples the data line at the agreed-upon speed (baud rate) to reconstruct the original data.
- **Error Handling**Errors such as framing or parity errors may occur, which must be handled by the receiving device.

The serial protocol offers the following messages:

- **Simplicity**: Easy to implement and requires minimal hardware.
- **Low pin usage**: Only two lines (TX and RX) are required for communication.
- **Asynchronous**: Does not require a clock line, simplifying wiring.

The disadvantages are:

- **Limited to two devices**: Direct connection between only two devices (unless using multiplexers).
- **No integrated addressing**: It cannot communicate natively with multiple devices.
- **Slower than synchronous protocols**: Due to the overhead of the optional start, stop, and parity bits.

The serial protocol is ideal for simple, low-speed communication between two devices. Its simplicity and widespread use make it a popular choice for debugging, firmware updates, and data logging in embedded systems.

Let's recap

We explored the world of**Physical Computing**, where microcontrollers are the beating heart of systems capable of connecting the physical and digital worlds. Through sensors, actuators, and communication protocols, microcontrollers collect information from the environment, process it, and respond with precise actions.

Key points to remember:

- **Connection with the environment:**
  - **Sensors**Devices that sense physical variables (e.g., temperature, light) and convert them into electrical signals. These signals can be:
    - **Analogue**: Continuous values that require ADC conversion to be understood by the microcontroller.
    - **Digital**: Discrete values, often transmitted via standard protocols such as I²C, SPI, or UART.
  - **Actuators**Devices that transform electrical signals into physical actions, such as movement, light, or sound, closing the loop of interaction with the environment.

- **Communication protocols:**
  - **SPI**: Ideal for high-speed, full-duplex connections to peripherals such as displays and memory.
  - **I²C**: Perfect for multi-slave systems with a small number of wires.
  - **UART**A simple and popular option for point-to-point communication. Each protocol has unique characteristics that make it suitable for specific scenarios, ensuring design flexibility.

- **Analog to Digital Conversion (ADC):**Converting analog signals to digital is essential to allow the microcontroller to interact with the real world, ensuring precision and reliability in measurements.

The Importance of Physical Computing. The ability to integrate input, processing and output enables the creation of interactive and intelligent applications, from home automation to mobile robots and weather stations.IoT.The secret to success lies in understanding the characteristics of the signals and choosing the most suitable communication protocol.

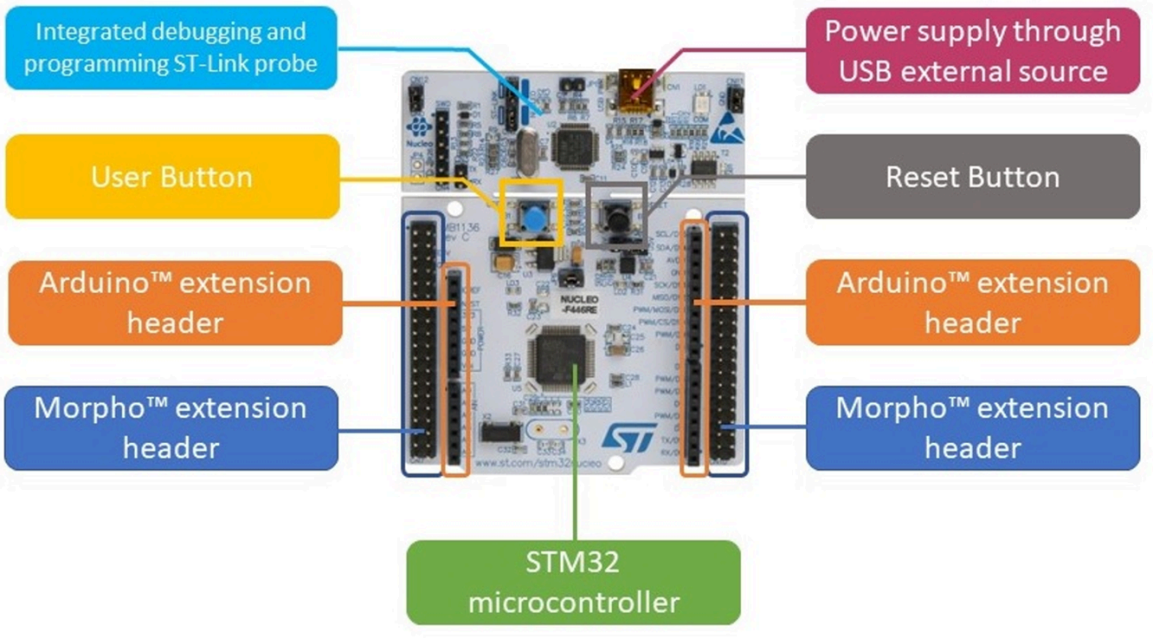
Chapter 2 The Nucleo F401RE board

The NucleoF401REIt is a development board designed by STMicroelectronics, based on the STM32F401RE microcontroller, part of the STM32 family. This board is ideal for prototyping and developing embedded applications thanks to its flexibility, standard interfaces (Arduino and Morpho), and ease of programming via ST-Link.

The STM32F401RE is a microcontroller in the STM32F4 series, featuring an ARM Cortex-M4 core with a floating point unit (FPU).

The main features are:

- **Core**: ARM Cortex-M4 a 84 MHz.
- **Flash Memory**: 512 KB.
- **RAM**: 96 KB.
- **Port I/O**: Up to 50 configurable and interruptible GPIOs.
- **Timer**: Multiple advanced and basic timers.
- **Interfaces**: Supporta UART, I2C, SPI e USB 2.0 OTG (On-The-Go).
- **ADC/DAC**: ADC a 12 bit.
- **Energy consumption**: Optimized for low power applications.



STM32 Microcontroller Naming Convention

The *naming convention*per imicrocontrollersSTM32 encodes their features. For the microcontroller**STM32F401RE**These are coded as follows:

- **STM32**: Indicates the family of microcontrollers produced by STMicroelectronics.
- **F**: Microcontroller series. The letter "F" represents high-performance microcontrollers.
- **4**: Specific series, oriented towards advanced performance.
- **01**: Specific model within the F4 series.
- **R**: Indicates the package type (LQFP64 for theNucleoF401RE).
- **AND**: Indicates the size of the Flash memory (512 KB).

Hardware features of theNucleoF401RE

The cardNucleoF401REoffers the following hardware features:

- **User LED and Programmable Button:**Status LED (LD2) connected to pin PA5, can be used to indicate states or signals in the program.
- **User button (USER)**: The blue button is connected to the pin**PC13**and is programmable to interact with the firmware.
- **Reset Button:**The black button allows you to reboot the microcontroller without having to unplug and reconnect the power supply.
- **Pin Standards: Arduino and Morpho:**
    - **Standard Arduino Uno R3**The board is compatible with Arduino expansion boards, thanks to its standard pin layout. This simplifies integration with existing sensors and modules.
    - **ST Morpho Connectors**: In addition to the Arduino layout, additional pins are available via the Morpho connectors, allowing access to all the microcontroller's features, such as additional GPIOs, ADCs and serial interfaces.
- **ST-Link Integrated:**The board comes with a programmer/debugger**ST-Link/V2-1**integrated, eliminating the need for external devices. This allows:
    - Firmware programming.
    - Debug tramite **SWD** (Serial Wire Debug).
    - A virtual USB interface for serial communication (**VCP**).

The NucleoF401REIt is a powerful and versatile platform for embedded development. The combination of the microcontroller**STM32F401RE**With standard interfaces like Arduino and Morpho, the built-in ST-Link and easily accessible peripherals make it suitable for both beginner and advanced projects.

Labels of the pin

In the context of electronics and microcontroller development, pin labels are identifiers used to describe the function or purpose of individual pins on a microcontroller, integrated circuit (IC), or other electronic component. These labels help users understand the role of the pin and connect it correctly in a circuit.

Common categories of pin labels are:

- **Power and Ground:**
  - **VCC/VDD/VIN:**Power input (positive voltage).
  - **GND:**Ground connection (reference to 0V).
  - **3V3 / 5V:**Outputs or inputs at specific voltages.
- **Digital Input/Output:**
  - **GPIO: General purpose input/output pins.**Often labeled as P0.0, P1.1, etc., or with the port number and pin.
  - **PWM:**Pin capable of generating pulse-width modulated signals.
  - **INT:** Interrupt pins used to detect specific events.
- **Analog Input/Output:**
  - **ADC / A IN:** Input pin for the analog-to-digital converter.
  - **DAC / A OUT:** Output pin for the digital-to-analog converter.
- **Communication:**
  - **UART:**
    - **TX:**Transmission line.
    - **RX:**Receiving line.
  - **I²C:**
    - **SCL:**Serial clock line.
    - **SDA:**Serial data line.
  - **SPI:**
    - **SCK:**Serial clock.
    - **ONE:** Master Out Slave In.
    - **MISO:** Master In Slave Out.
    - **CS / SS:** Chip Select / Slave Select.
  - **CAN:**
    - **SOUP:** CAN High.
    - **LIVE:** CAN Low.
- **Functions Speciali (Special Functions):**
  - **RESET / RST:**Pin used to reset the device.
  - **BOOT:**Pin used to configure the boot mode.
  - **CLK:**Input/output pin for the clock.
  - **IN:**Enable pin to turn a device on or off.
  - **LED / IND:**Pins for indicator LEDs.

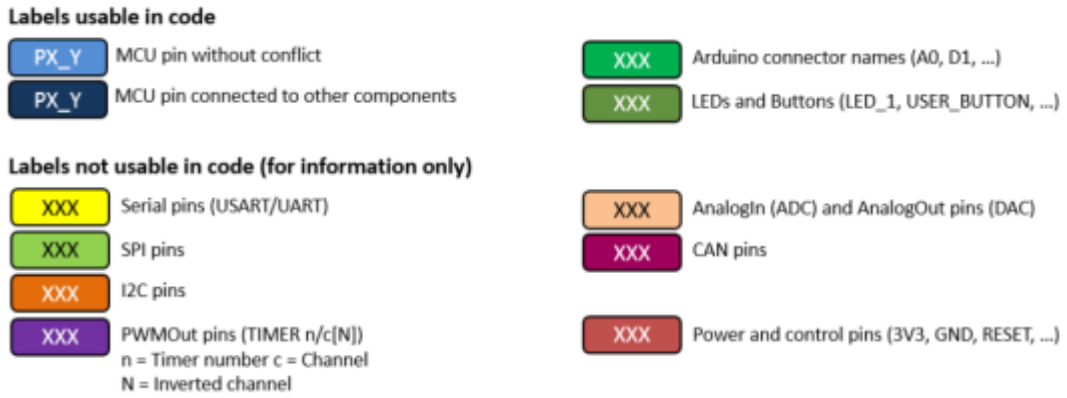
Microcontroller manufacturers generally label pins based on:

- **Functional grouping:**Pins can have multiple functions (for example, a GPIO pin that also acts as an SDA line for I²C).
- **Pin Mapping:**Defined in the microcontroller datasheet, often with diagrams or tables showing possible configurations.

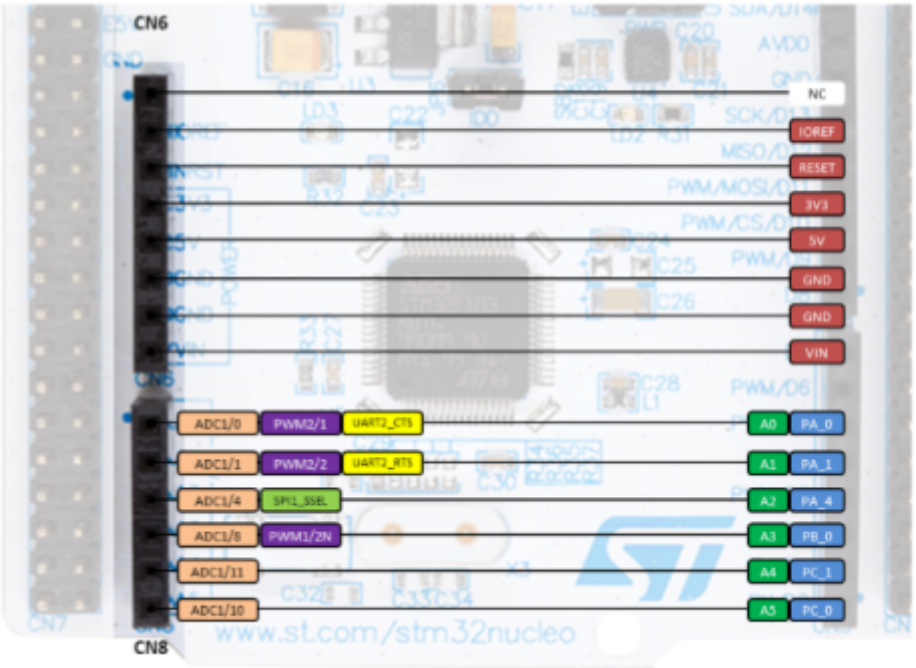
Example:
A pin could be labeled as**P1.0/TXD/PWM1**, indicating that it can function as:

- General Purpose Input/Output (GPIO)**P1.0**.
- UART Transmission (**TXD**).
- PWM output (**PWM1**).

When working with a microcontroller, always refer to the datasheet or pin diagram for an accurate description of pin labels and configurations.

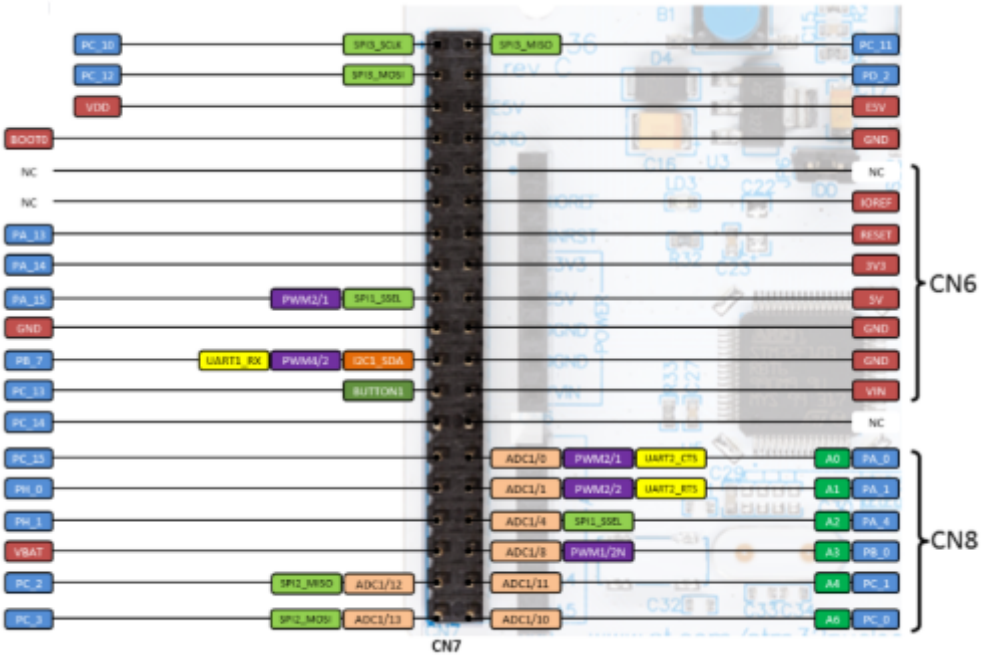


Arduino-compatibile headers





Morpho headers. Theseheaders damageaccessto all pins of the STM32.





Installation

To realize the projects described in this manual we will use two software tools provided free of charge by STMicroelectronics: STM32Cube MX and STM32Cube IDE.

Starting with STM32Cube IDE version 2.0, the two environments are distributed as separate applications and must be installed individually.

STM32Cube MX It is used to select the development board, configure the microcontroller peripherals and generate the project initialization code.

STM32Cube IDE Instead, it is used to write application code, compile firmware, program it on the microcontroller, and debug it.

Installing both tools is therefore necessary to follow all the practical exercises proposed in the manual.

In the following paragraphs we will see how to install and configure the two development environments.

System Requirements

Before proceeding with the installation ofSTM32CubeMXand STM32CubeIDE, you need to make sure your computer meets the required hardware and software requirements.

Supported operating systems

- **Windows:** Windows 10 Windows 11, a 64 bit.
- **Linux:**Ubuntu LTS 22.04 or 24.04 and Fedora 43.
- **macOS:** macOS 15 (Sequoia) o macOS 26 (Tahoe).

Hardware requirements

- **RAM memory:** at least 4 GB;to use STM32Cube MX simultaneously,STM32CubeIDEand other development tools 8 GB or more are recommended.
- **Disk space:**At least 6 GB available for STM32Cube IDE. Additional space is recommended for STM32Cube MX, STM32Cube firmware packages, and projects developed during the labs.
- **Architecture:**use versions of software that are compatible with your computer's operating system and architecture.

Software requirements

STM32CubeIDEincludes the tools needed to compile and debug STM32 projects.

STM32CubeMXmanages the microcontroller configuration and the generation of the initialization code.

To follow the exercises described in the manual, you need to install both tools.

Requirements may change with new software releases. Before installing, it is recommended to consult the official STMicroelectronics documentation.

Download STM32CubeMX and STM32CubeIDE

To develop the applications described in this manual you need to download and install two free tools made available by STMicroelectronics: STM32CubeMX and STM32CubeIDE.

Download STM32CubeMX

1. Visit the official STMicroelectronics website: https://www.st.com/en/development-tools/stm32cubemx.html
2. Locate the section dedicated to downloading the software.
3. Select STM32CubeMX and the version compatible with your operating system (Windows, macOS or Linux).
4. Start downloading the installation package.

For the exercises in this manual we will use STM32CubeMX, maintaining the traditional configuration and code generation environment for STM32 microcontrollers.

Download STM32CubeIDE

1. Visit the official STMicroelectronics website: https://www.st.com/en/development-tools/stm32cubeide.html
2. Locate the download section.
3. Select the Eclipse-based version of STM32CubeIDE compatible with your operating system.
4. Start downloading the installation package.

Registration and login

During the download, STMicroelectronics may require you to log in with a myST account or fill out a form with your name and email address and verify it.

You must also accept the license terms provided for each software.

Once both downloads are complete, you can proceed with installing the two tools.

Installation on Windows

After downloading the installation packages, we proceed to install STM32CubeMX and STM32CubeIDE.

Installing STM32CubeMX

Locate the STM32CubeMX package downloaded from the STMicroelectronics website.

Extract the contents of the ZIP archive to a temporary folder.

Run the file SetupSTM32CubeMX-<version>-Win.exe.

In the wizard, choose whether to install the program for the current user only or for all users on the computer. The first option is generally recommended.

Accept the license terms.

Select the proposed installation folder or specify a different one.

Complete the installation by following the on-screen instructions.

Once finished, STM32CubeMX will be available among the installed applications.

Installing STM32CubeIDE

Locate the STM32CubeIDE package downloaded from the STMicroelectronics website.

If the package is distributed as a ZIP archive, extract it to a temporary folder.

Launch the Windows installer.

Accept the license terms.

Select the destination folder or keep the default one.

During installation, check that the components and drivers needed to communicate with the ST-LINK programmer/debugger are present.

Complete the wizard.

Once installation is complete, both tools will be available and can be launched separately.

Note: For the exercises, we will be using a NUCLEO-F401RE board, which has a built-in ST-LINK programmer/debugger. It is therefore important that the relevant drivers are installed correctly so that STM32CubeIDE can communicate with the board via the USB port.

Installation on macOS

To use STM32CubeMX and STM32CubeIDE on an Apple computer you need to download the respective installation packages, selecting the versions compatible with your operating system and processor architecture (Intel or Apple Silicon).

Installing STM32CubeMX

1. Locate the STM32CubeMX package downloaded from the official STMicroelectronics website.
2. Extract the contents of the ZIP archive.
3. Launch the installation application`SetupSTM32CubeMX-<version>.app`.
4. If prompted, allow the application to run through your macOS security settings.
5. Follow the wizard, accept the license terms and select the installation folder.
6. Complete the installation.

Once finished, STM32CubeMX will be available among the installed applications and can be launched from the Launchpad.

Installing STM32CubeIDE

1. Locate the STM32CubeIDE package downloaded from the STMicroelectronics website.
2. Open the installation file in format`.dmg`.
3. Accept the license terms and start the wizard.
4. Follow the installer instructions, selecting the proposed destination or a different location, where allowed.
5. Complete the installation.

Once finished, STM32CubeIDE will be available among your computer's applications.

Verifying the installation

Start STM32CubeMX and STM32CubeIDE separately and verify that both open correctly.

For the hands-on labs, we will use STM32CubeMX to configure the microcontroller and generate the projects, while STM32CubeIDE will be used to develop the application code, compile, program, and debug the NUCLEO-F401RE board.

Installation on Linux

To install STM32CubeMX and STM32CubeIDE on Linux, you need to download the packages compatible with your distribution. The procedures may vary slightly depending on the operating system used.

Installing STM32CubeMX

1. Download the STM32CubeMX package for Linux from the official STMicroelectronics website.
2. Extract the contents of the ZIP archive to a folder.
3. Open Terminal and navigate to the folder containing the installer.
4. Make the installation file executable with the command:
   `chmod +x SetupSTM32CubeMX-<version>`
5. Start the program with the command:
   `./SetupSTM32CubeMX-<version>`
6. Follow the wizard, accept the license terms, and select the installation directory.

If you want to install the software in a shared system directory, you may need administrator privileges.

Installing STM32CubeIDE

STM32CubeIDE is distributed for Linux through several packages:

- Debian/Ubuntu: DEB package.
- Fedora and compatible distributions: RPM package.
- Other supported distributions: Generic Linux installer.

1. Download the appropriate package from the official STMicroelectronics website.
2. If the package is contained in an archive, extract it.
3. Open the terminal and navigate to the directory containing the installer.
4. Start the installation using the appropriate command.

For example, for a shell installer:

```
sudo sh ./st-stm32cubeide_<version>_<architettura>.<pacchetto>
```

Replace the placeholders with the actual name of the downloaded file.

5. Follow the instructions in the terminal until the installation is complete.

Configuring USB permissions

To program and debug the NUCLEO-F401RE board, the user must have permission to access the ST-LINK device connected via USB.

During the installation of STM32CubeIDE the rules are also normally installed`udev`dedicated to ST-LINK, when provided for by the selected package.

If the card is not recognized, it will be necessary to verify the presence of the rules`udev`, user permissions, and USB connection.

Verifying the installation

Once the installation is complete, launch STM32CubeMX and STM32CubeIDE and verify that both are working correctly.

The microcontroller configuration will be done through STM32CubeMX, while compilation, programming and debugging will be handled through STM32CubeIDE.

ConfiguringSTM32CubeIDE

Once the installation is complete, let's start STM32CubeIDE to configure the development environment we will use during the exercises.

Starting STM32CubeIDE

Launch STM32CubeIDE from your operating system's application menu.

On first launch, the environment may prompt you to select a working folder (*workspace*) in which the projects will be organized.

Configuring the workspace

The workspace is a directory used by STM32CubeIDE to store workspace settings and manage development projects.

For the exercises in this manual we can use, for example, the following folder:

```
C:\WS_CoreOfEmbeddedSystems
```

Once the workspace has been selected, we confirm to access the development environment.

Using with STM32CubeMX

Starting with version 2.0, STM32CubeIDE no longer integrates the STM32CubeMX graphical configurator.

For the projects described in this manual we will therefore use STM32CubeMX as a separate application, through which we will configure the microcontroller and generate the initial project.

The project will then be imported into STM32CubeIDE, where we can write the application code, compile the firmware, program it on the NUCLEO-F401RE board and debug it.

The complete procedure for creating, generating and importing a project will be illustrated in Chapter 3, through the creation of the first practical example: the flashing of the green LED on the Nucleo board.

Introduction toSTM32CubeMX

STM32CubeMX is a free software tool developed by STMicroelectronics to simplify the configuration of STM32 microcontrollers and the automatic generation of hardware peripheral initialization code.

Starting with STM32CubeIDE 2.0, STM32CubeMX is used as a separate application from the development environment. Through its graphical interface, you can select the microcontroller or development board, configure the pins,i clockand peripherals, define the project parameters and generate the source code necessary for its initialization.

By selecting STM32CubeIDE as the target development environment, STM32CubeMX generates a project that can then be imported, compiled and programmed using STM32CubeIDE.

The main features are:

- **Selecting the microcontroller or development board:**allows you to identify the STM32 device to use or directly select a development board, such as the NUCLEO-F401RE, through a graphical interface.
- **Microcontroller graphical configuration:**It allows you to assign pin functions and configure hardware peripherals, such as GPIO, UART, SPI, I²C, ADC and timers, through an intuitive interface.
- **Clock Configuration:**allows you to define the clock sources, prescalers and operating frequencies of the microcontroller through a graphical representation of the clock tree (*Clock Configuration*).
- **Automatic code generation:**produces the C language source code needed to initialize the microcontroller and configured peripherals, using the HAL (Hardware Abstraction Layer) or, where supported, LL (Low Layer) libraries.

- **Firmware Package Management:**allows you to select, download and manage STM32Cube packages relating to the different microcontroller families, containing drivers, libraries and application examples.
- **Support for middleware and software components:**allows you to configure, when available for the selected device, software components such as real-time operating systems, file systems and communications stacks.
- **Energy consumption analysis:**provides tools for estimating energy consumption under different operating conditions for supported devices.
- **Project Management and Generation:**allows you to define the name, destination directory and development environment(*Toolchain/IDE*)for which to generate the project. In our case, we'll select STM32CubeIDE, which we'll use to develop, compile, and program the firmware.

The workflow with STM32CubeMX and STM32CubeIDE

In the exercises proposed in this manual we will use STM32CubeMX and STM32CubeIDE as two complementary tools, each dedicated to a specific phase of firmware development.

The development process is divided into the following phases:

1. **Creating the project in STM32CubeMX:**we select the NUCLEO-F401RE tab and define the name and directory of the project.
2. **Hardware Configuration:**Through the STM32CubeMX graphical interface we configure the pins, peripherals and operating parameters of the microcontroller.
3. **Code Generation:**We select STM32CubeIDE as the target development environment and generate the project, including the initialization code and the necessary libraries.
4. **Firmware Development:**We import the project into STM32CubeIDE and implement the application code in C language, using the functions provided by the HAL libraries.
5. **Compilation and programming:**We compile the project, generate the executable firmware and transfer it to the microcontroller's Flash memory using the ST-LINK programmer integrated into the Nucleo board.
6. **Test e debug:**We verify the application's functioning directly on the board, using the debugging tools available in STM32CubeIDE.

Changing the hardware configuration

During development you may need to change the pin or peripheral configuration.

In this case we reopen the project configuration file, with extension `.ioc`, through STM32CubeMX, we make the desired changes and generate the code again.

We then return to STM32CubeIDE to continue development and compile the updated project.

To prevent the application code from being overwritten during a new build, it is important to place the changes in the specially designated sections, delimited by comments.`USER CODE BEGIN`and`USER CODE END`.

In the next chapter, we will apply this workflow to the creation of our first project: the blinking of the green LED on the NUCLEO-F401RE board.

Chapter 3GPIO output

The GPIO (General Purpose Input/Output) is one of the most important peripherals on a microcontroller. GPIO pins can be configured as digital inputs or outputs, making them extremely versatile for interfacing with the outside world.

In this first hands-on project, we will learn how to configure a GPIO pin as a digital output to flash the green LED built into the NUCLEO-F401RE board.

We will use STM32CubeMX to select the board, configure the microcontroller, and generate the initial project. Then, using STM32CubeIDE, we will analyze the project structure and write the application code in C.we will fill inthe firmware and thewe will transferon the board using the integrated ST-LINK programmer.

This exercise represents the starting point for all subsequent applications developed in the manual and will allow us to become familiar with the entire development flow of an STM32 embedded system.

Project Objectives

- Understand how a GPIO pin configured as a digital output works.
- Create a new STM32 project via STM32CubeMX, selecting the NUCLEO-F401RE board.
- Configure the PA5 pin, connected to the green LED on the board, as a GPIO output.
- Generate the microcontroller initialization code and import the project into STM32CubeIDE.
- Understand the structure of an STM32 project and identify the sections dedicated to the application code.
- Use HAL library functions to control the logical state of a GPIO pin.
- Introduce the concept of time delay to make an LED flash.
- Compile the firmware and program it on the NUCLEO-F401RE board via the built-in ST-LINK debugger/programmer.

Basic Theory: What is a GPIO?

GPIOs are configurable pins that can operate in:

- **Digital inputs:**To read the logical state (high or low) of an external signal.
- **Digital output:**To generate a logic state (high or low) and control external devices such as LEDs or relays.

When a GPIO pin is configured as a digital output:

- A state**alto (HIGH)**typically corresponds to a positive voltage (e.g. 3.3V).
- A state**basso (LOW)**corresponds to a voltage close to 0V.

Setting Up the Project

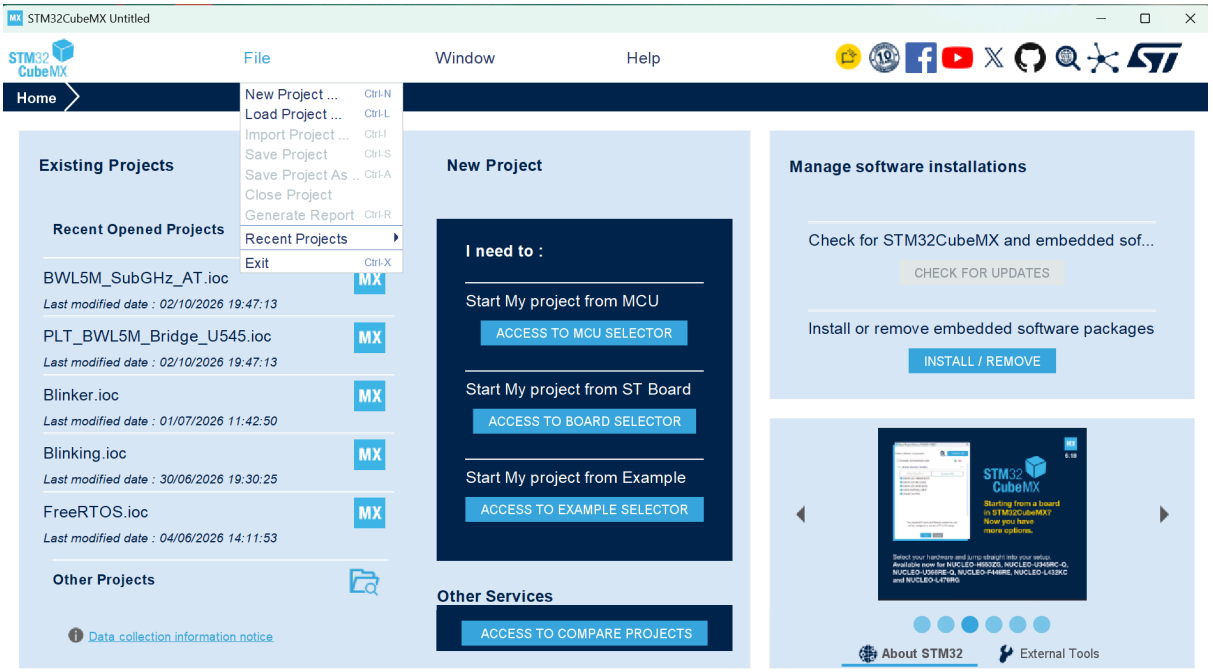
As illustrated in the previous chapter, our development flow involves using STM32CubeMX for microcontroller configuration and project generation, and STM32CubeIDE for firmware development, compilation, and debugging.

Let's start the previously installed STM32CubeMX to create our first project.

When you open the program, the home screen appears, from which you can create a new project or open an existing one.

We select**New Project**to access the interface for choosing the microcontroller or development board.

In our case we will use the card**NUCLEO-F401RE**by STMicroelectronics.



Selecting the NUCLEO-F401RE board

In the new project creation window, select the tab**Board Selector**, which allows you to directly choose a development board produced by STMicroelectronics.

In the search field**Commercial Part Number**we insertNUCLEO-F401RE.

STM32CubeMX will displayin the list of available cards (*Boards List*) the corresponding card.

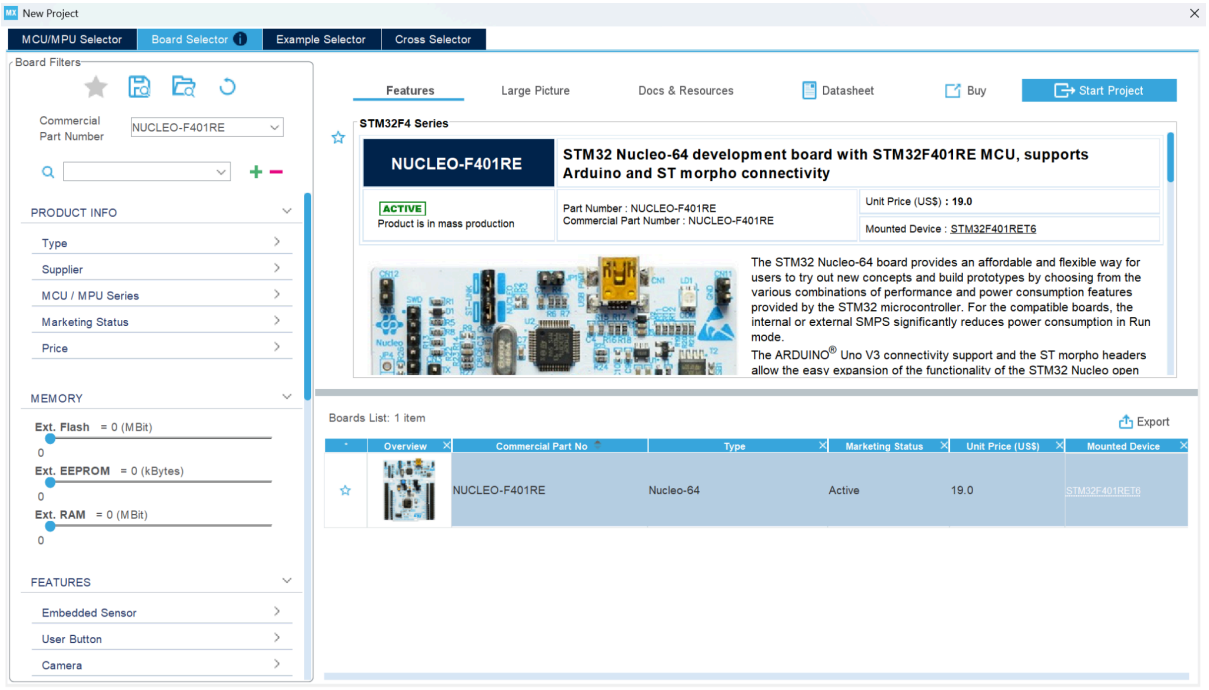
We select**NUCLEO-F401RE**and click the button**Start Project**.

In this way we have selected the development board that we will use in the project and the related STM32F401RE microcontroller.

STM32CubeMX may ask if we wantinitializethe card's peripherals with default configurations (*Initialize all peripherals with their default Mode?*).

For this exercise we select**Yes**, so as to use the initial configurations provided for the Nucleo card.

The microcontroller configuration interface will then open, where we can view the pin layout and available peripherals.



Project setupBlinkLed

After selecting the NUCLEO-F401RE board and accepting the peripheral initialization with the default values, STM32CubeMX displays the microcontroller configuration interface.

Before generating the code, we need to name the project and specify the development environment we will use.

We then select the tab**Project Manager**, available at the top of the interface.

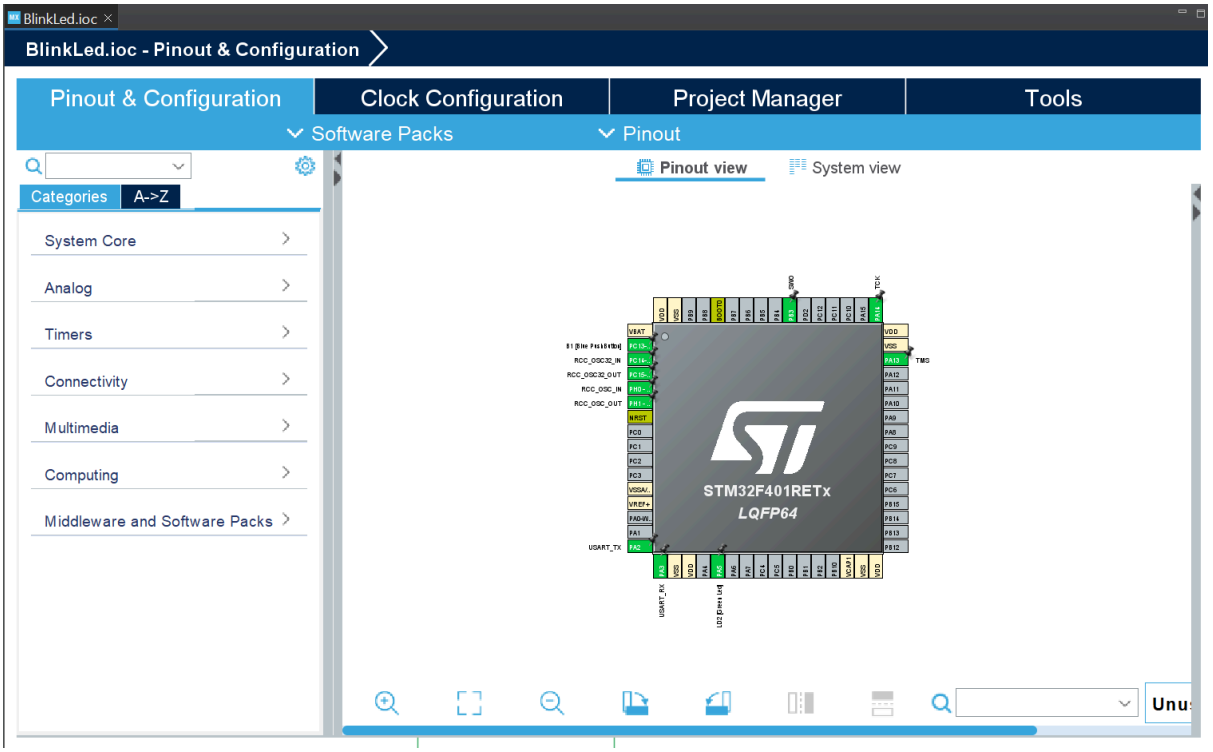
In the section**Project**we set the following parameters:

- **Project Name:** `BlinkLed`
- **Project Location:**we select a folder in which to store the course projects, for example`C:\WS_CoreOfEmbeddedSystems`.
- **Toolchain/IDE:**we select`STM32CubeIDE`.

This last setting is particularly important: it tells STM32CubeMX to generate a project compatible with STM32CubeIDE.

Let's keep the other parameters unchanged for now.

The project will thus be prepared for the automatic generation of the microcontroller initialization code and for the subsequent development of the firmware in STM32CubeIDE.



The configurations we make will always be editable as they are saved in the file*BlinkLed.ioc*.

Project Generation

After defining the project name, the target directory and STM32CubeIDE as the development environment, we can proceed with code generation.

Before doing so, let's check the tab**Pinout & Configuration**that the PA5 pin, to which the green LED of the NUCLEO-F401RE is connected, is configured as`GPIO_Output`.

The detailed procedure for configuring the GPIOs will be covered in more detail in the following paragraphs.

So let's go back to the tab**Project Manager**and click the button**GENERATE CODE**, available at the top of the interface.

STM32CubeMX automatically generates the microcontroller initialization code, source files and libraries needed for the project.

Once the generation is complete, in the selected directory we will find the folder`BlinkLed`, containing the project intended for STM32CubeIDE.

The project is now ready to be opened in the development environment, where we will write the application code,we will fill inthe firmware and we will program the Nucleo board.

Importing the project into STM32CubeIDE

Once the code generation is complete in STM32CubeMX, we can open the project in the STM32CubeIDE development environment.

We start STM32CubeIDE and select the previously configured workspace.

To import the newly generated project, proceed as follows:

1. From the main menu we select**File → STM32 Project Create/Import**.
2. In the window that appears, we locate the section**Import STM32 Project**.
3. We select**STM32CubeMX/STM32CubeIDE Project**and we click on**Next**.
4. We indicate the path of the directory containing the project`BlinkLed`, previously generated by STM32CubeMX.
5. We confirm the import by clicking on**Finish**.

At the end of the procedure, the project`BlinkLed`will appear in the panel**Project Explorer**, located on the left side of the development environment.

By expanding the project structure, we will be able to view the folders and files automatically generated by STM32CubeMX, which form the basis for the development of our application.

From now on we will use STM32CubeIDE to write the application code, compile the firmware and program it on the NUCLEO-F401RE board.

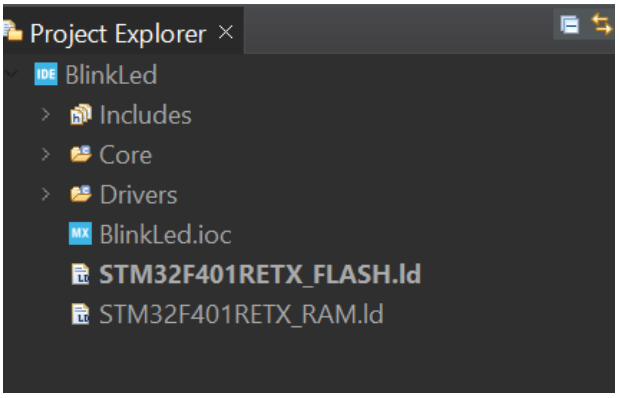
Any changes to the pin, clock and peripheral configuration will be made through STM32CubeMX, reopening the file `.ioc` of the project and regenerating the code.

The structure of an STM32CubeIDE project

After importing the project `BlinkLed` in STM32CubeIDE, we canto examine itthe structure before you start writing the application code.

The files and folders that make up the project were automatically generated by STM32CubeMX, based on the previously defined microcontroller configuration.

On the left side of the development environment we find the panel **Project Explorer**, which allows you to view and manage project files.



Expanding the project `BlinkLed`, we can locate the folders containing the source code, header files, libraries, and other elements needed to compile and run the firmware.

The basic structure is common to STM32 projects generated for STM32CubeIDE, although some files and folders may vary depending on the microcontroller and peripherals used.

In the following paragraphs, we will analyze the main folders to understand where the automatically generated code is located and where to insert our application code.

The main project folders

An STM32 project generated using STM32CubeMX and imported into STM32CubeIDE is organized into folders that allow you to separate the application code, libraries, and files required for the microcontroller to operate.

Let's examine the main ones.

Includes

The Includes entry, when present in the Project Explorer, collects references to header files (*header file*, with extension `.h`) used by the project.

This does not necessarily represent a physical folder, but a view of the include paths configured in the development environment.

These files contain function declarations, data structure definitions, constants, and macros used by the source code.

For example, the instruction:

```
#include "stm32f4xx_hal.h"
```

allows you to include the definitions needed to use the STM32F4 family HAL libraries.

Core

The folderCorecontains the main firmware files, including those automatically generated by STM32CubeMX and those into which we will insert our application code.

The main subfolders are:

- **Inc:**contains project-specific header files, such as`main.h`, which defines macros, constants, and declarations used by the program.
- **Src:**contains the source files in C language, including`main.c`, which represents the application's entry point. In this folder we can also find other files, such as those dedicated to interrupt management.
- **Startup:**In projects where it's present within Core, it contains the microcontroller's startup code, generally written in assembly language. The startup file initializes the necessary resources before the main program runs and contains the interrupt vector table. In some project structures, the startup file is located in a separate folder.

The Core folder therefore represents the main point of reference for the development of our application.

Drivers

The Drivers folder contains the libraries and files needed to interact with the microcontroller hardware.

Among the main subfolders we find:

- **CMSIS:**Contains standard definitions and interfaces for using Arm Cortex-M processors and STM32 devices. It provides access to core functionality and hardware registers, among other things.
- **STM32F4xx_HAL_Driver:** contiene i driver HAL (*Hardware Abstraction Layer*) and, when applicable, LL (*Low Layer*) for managing the peripherals of the STM32F401RE microcontroller.

HAL libraries allow you to use peripherals through predefined functions, avoiding having to directly manage all the hardware registers.

For example, the function:

`HAL_GPIO_WritePin()`

allows you to change the logical state of a GPIO pin without having to write directly to the peripheral's registers.

Project Organization

This organization allows you to distinguish the application code from the hardware libraries and initialization files.

During the exercises we will work mainly on the file`main.c`, contained in the folder`Core/Src`, using the functions provided by the HAL libraries.

It's important to remember that some of the code is automatically generated by STM32CubeMX. To preserve our changes across subsequent builds, we'll use the user code sections, identified by comments.`USER CODE BEGIN`and`USER CODE END`.

The .ioc configuration file

When creating the project, STM32CubeMX generates a file with the extension`.ioc`, in our case called`BlinkLed.ioc`.

This file contains information related to the microcontroller configuration, such as pin assignments, peripheral enablements, clock configuration, and parameters used to generate the initialization code.

The file`.ioc`allows you to maintain and modify the project's hardware configuration even after code generation.

If you need to change the configuration, simply open the file`BlinkLed.ioc`via STM32CubeMX, make the desired changes and proceed with code generation again.

Subsequently, returning to STM32CubeIDE, we can continue developing the application using the updated configuration.

It is important to insert the application code in the dedicated sections, delimited by comments`USER CODE BEGIN`and`USER CODE END`, so that it is preserved during subsequent generations of the code.

This way, hardware configuration and firmware development remain separate, but are part of the same project.

Compiling the project in STM32CubeIDE

The project`BlinkLed`, generated by STM32CubeMX and subsequently imported into STM32CubeIDE, already contains the code necessary to initialize the microcontroller and the configured peripherals.

Even though we haven't implemented the application logic for blinking the LED yet, the project is already compilable.

We can then verify that the code generation was successful.

In the panel**Project Explorer**, we select the project`BlinkLed`.

To start the compilation we can use one of the following methods:

- Click on the hammer icon (**Build**) present in the toolbar.
- Select**Project → Build Project**from the main menu.
- Right-click on the project`BlinkLed`and select**Build Project**.

STM32CubeIDE starts compiling using the GCC compiler for the Arm architecture.

During the process, in the window**Console**, messages relating to the various operations performed by the compiler and the linker are displayed.

If no errors are found, the compilation ends successfully and the firmware executable file is generated, usually in the format`.elf`.

The ELF file (*Executable and Linkable Format*) contains the machine code and information needed to upload and debug the program on the microcontroller.

The file is normally saved in the folder`Debug`of the project, when the Debug build configuration is selected.

Once the compilation is complete, it is important to check that the Console does not report any errors.

The possible ones*warning*They do not necessarily prevent the firmware from being built, but should still be examined for possible issues in the code.

The project is now ready to be programmed on the NUCLEO-F401RE board.

Microcontroller programming

After successfully completing the project`BlinkLed`, we can transfer the firmware to the Flash memory of the STM32F401RE microcontroller on the Nucleo board.

The NUCLEO-F401RE board integrates a programmer/debugger**ST-LINK**, which allows you to program and debug the microcontroller directly through a USB connection, without using external devices.

Connecting the card

We connect the NUCLEO-F401RE board to the computer via a USB cable, using the connector associated with the ST-LINK.

Let's check that the card is powered and that the operating system recognizes the device.

Firmware programming

In STM32CubeIDE we select the project`BlinkLed` nel pannello Project Explorer.

To transfer the firmware to the card and run it, we can proceed as follows:

1. We right-click on the project`BlinkLed`.
2. We select**Run As → STM32 C/C++ Application**.
3. If prompted, confirm the creation of the running configuration (*Run Configuration*), keeping the default parameters for connection via ST-LINK.
4. STM32CubeIDE establishes the connection to the microcontroller and transfers the firmware to Flash memory.
5. Once programming is complete, the microcontroller starts executing the program.

Alternatively, after configuring the first run, we can use the button**Run**, represented by the green triangle icon in the toolbar.

Programming Verification

During the procedure, STM32CubeIDE displays messages in the Console regarding the connection with ST-LINK and the firmware transfer.

If the programming completes successfully, the microcontroller executes the uploaded code.

In our case, since we haven't yet entered the instructions to control the green LED, the program will only perform the initialization operations provided by the automatically generated code.

Possible connection problems

If STM32CubeIDE cannot communicate with the board, let's check:

- That the NUCLEO-F401RE is correctly connected to the computer.
- That the USB cable supports data transfer and not just power.
- That the ST-LINK programmer is recognized by the operating system.
- That the necessary drivers are installed correctly.
- That there are no other active applications using the same ST-LINK device at the same time.

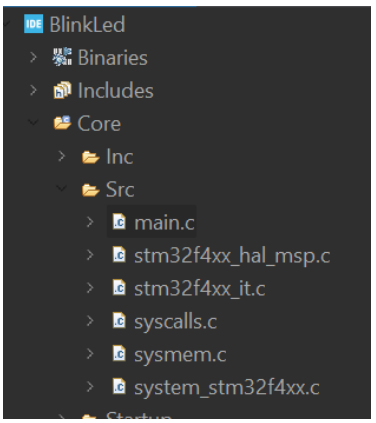
Once any connection issues have been resolved, we can repeat the programming procedure.

The card is now ready to run our first application program.

He's running by hand.

The file`main.c`, generated byCubeMXfor a cardNucleoF401RE,It is the main file of a HAL (Hardware Abstraction Layer)-based project for the STM32F401RE microcontroller. It contains the initial configuration and the main firmware loop.

This file is located in*Core/Src*



The file includes various headers necessary for the code to function:

- `stm32f4xx_hal.h`: Provides definitions and APIs for the HAL library.
- Specific headers for configured devices (e.g.`gpio.h`, `usart.h`, `tim.h`).

Inside it there are declarations of configuration functions such as:

- `void SystemClock_Config(void)` ; Thatconfigure the system clock.
- `void MX_GPIO_Init(void)` ;what's uponfigura i pin GPIO.
- Other specific functions to configure peripherals (e.g. UART, Timer).

The main function`main()`This is the entry point of the program. This invokes:

- l'HAL initialization,`HAL_Init();`
- lakeConfiguring the System Clock in the function`SystemClock_Config()`.
- the Initializationof Peripherals. Call initialization functions generated by CubeMX, such as MX_GPIO_Init(); andMX_USART2_UART_Init();
- iIt also includes the Main Loop (while):

```
while (1)
{
// Custom functions
}
```

CubeMXIncludes comments to define where to write your code:

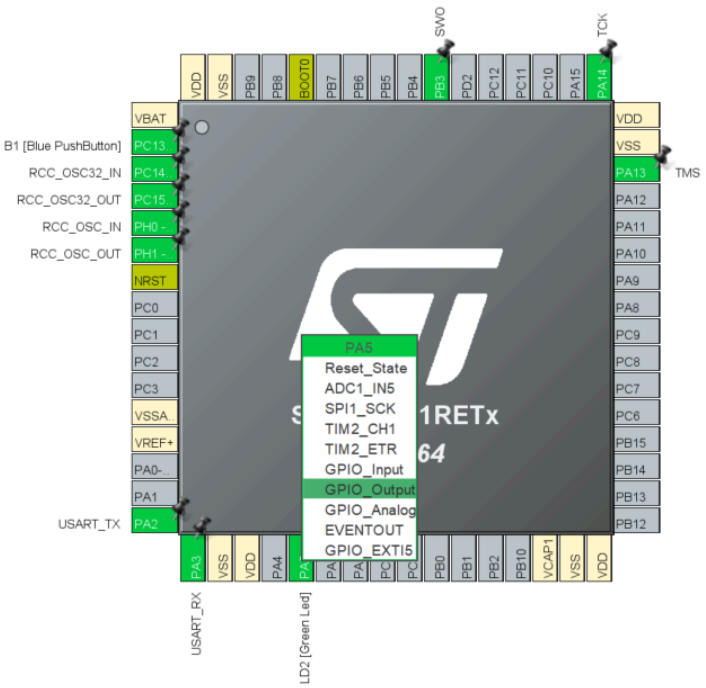
- `/* USER CODE BEGIN 0 */`and`/* USER CODE END 0 */`to define global variables.
- `/* USER CODE BEGIN WHILE */`and`/* USER CODE END WHILE */`to insert custom code into the main loop.

Basically, the file`main.c`It is an automatically generated starting point, structured to beextensible.Developers add their own code in the designated sections without altering the automatically generated parts, making it easy to regenerate inCubeMXwithout losing any changes. The file includes the necessary header files and incorporates all the initialization functions that are called in the main() function.

Pin Configuration

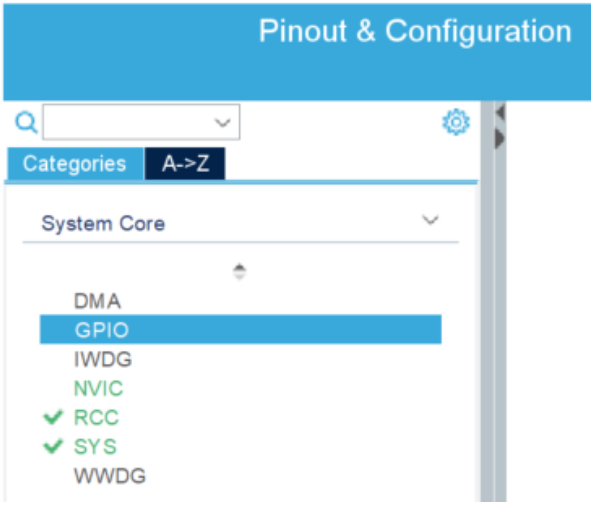
In the microcontroller view, identify the pin connected to the green LED. For the Nucleo-F401RE, the green LED is connected to thepin PA5.

CubeMXIt should have configured the pin as a GPIO output. If the pin is green, it's already configured. However, we can proceed as follows to analyze the configuration:

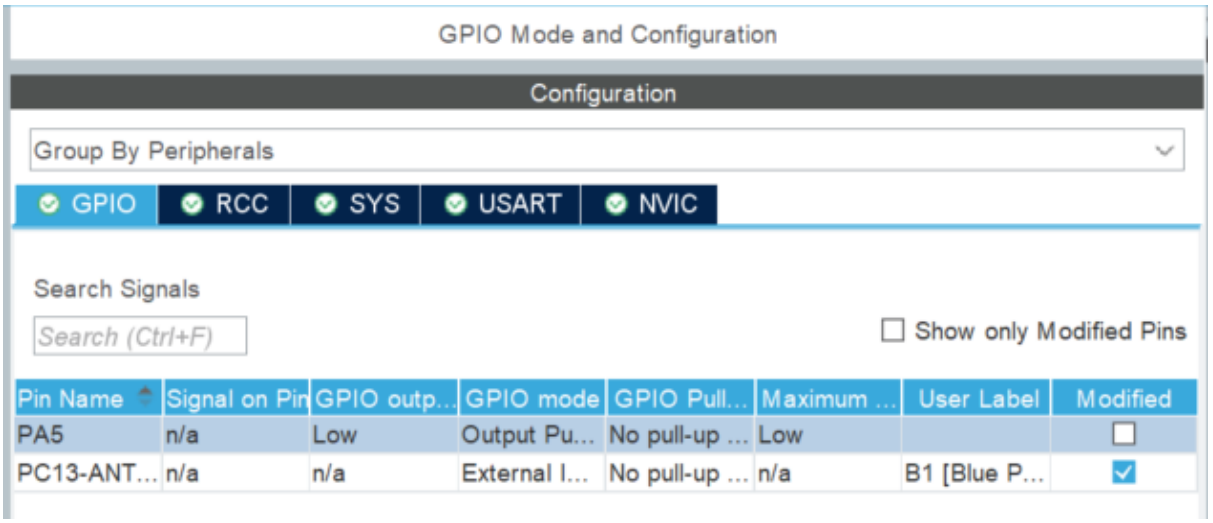


Click on PA5 and select**GPIO_Output**among all the possible ones*alternate functions*available for pinning.

In *System Core*select GPIO



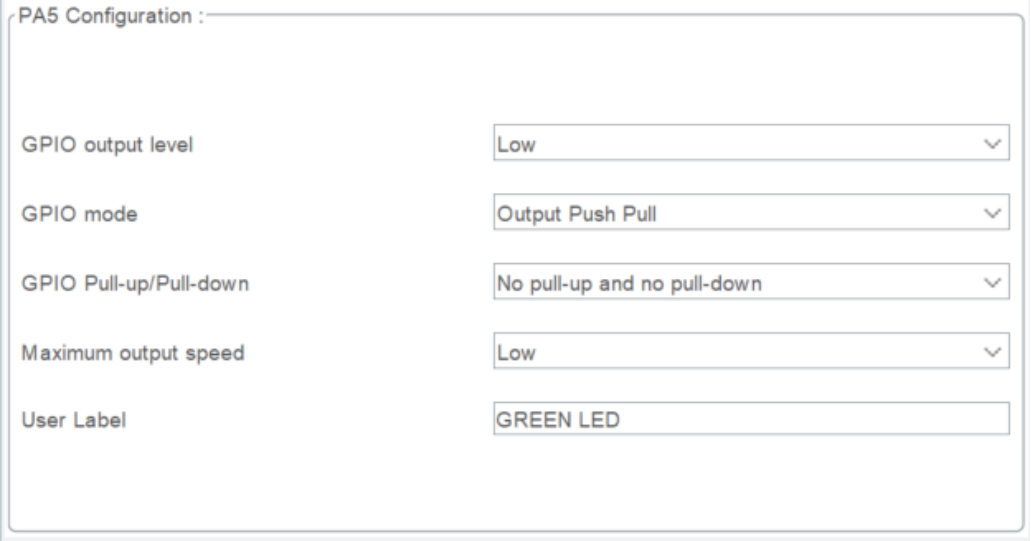
This will display all pins configured as GPIO.



He pines**PA5**(Green LED) in the list should be configured as follows:

- **Mode**: Output Mode
  i. In a GPIO configured in mode**push-pull**, the pin can actively drive both the high logic state (by connecting to the supply voltage) and the low logic state (by connecting to GND). In other words, the microcontroller internally supplies current both when the pin is high and when it is low. This configuration is typically the default for standard digital outputs, because it guaranteestransitionsfast and does not require external components.
  ii. In a GPIO configured in mode**open-drain**, instead, when the internal transistor is "open" the pin is disconnected (high impedance) and does not supply any voltage, while when the transistor is "closed" the pin is connected to ground (GND). In order to have a high level onan exitopen-drain, an external pull-up resistor (or internal, if available in the microcontroller) is required.
- **Pull-up/Pull-down**: No Pull (usually not necessary, but depends on the specific design)
- **Speed**: Low, Medium, High (choose according to your needs; for an LED it is generally sufficient**Low**)
- **Output Type**: Push-Pull

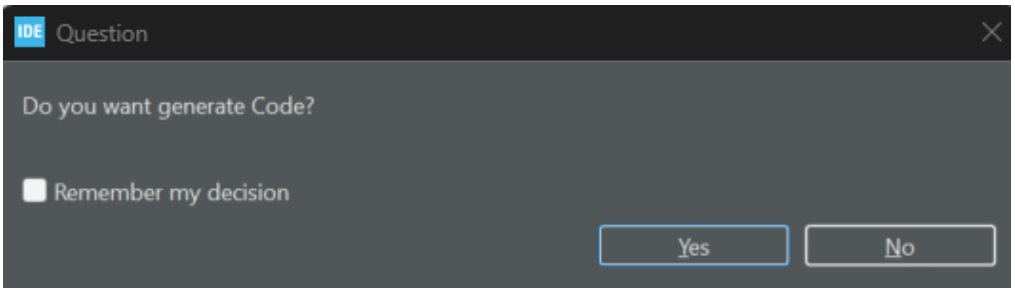
By clicking on the PA5 pin you can open the configuration window.



The default configuration values are fine for this project. We can define a label*User Label*to help identify the pin's functionality. This can be done for all pins, regardless of their functionality.

With these steps, the PA5 pin has been configured as a GPIO output pin and we can start using it.

By saving the project,CubeMXasks us if we want to generate the code.



We accept generation with one click on*Yes*. Popping up*Remember my decision*, code generation will be performed on every save.

The PA5 pin configuration, having been defined as a pinGPIO_Outputwill be present in the function `static void MX_GPIO_Init(void);`

Aspetti del Linguaggio C

- **`static`**: Limita la visibilità della funzione al file corrente, prevenendo conflitti di nome con altre funzioni simili in progetti complessi.
- **`void`**: Specifica che la funzione non accetta parametri e non restituisce un valore.

Let's examine the code of the initialization function generated by CubeMX.

```
GPIO_InitTypeDef GPIO_InitStruct = {0};
```

- Crea una variabile della struttura **GPIO_InitTypeDef**, utilizzata per configurare i parametri dei pin GPIO, come modalità, velocità, tipo di output, ecc.
- Inizializza la struttura a zero per garantire valori predefiniti.

```
__HAL_RCC_GPIOC_CLK_ENABLE();

__HAL_RCC_GPIOH_CLK_ENABLE();

__HAL_RCC_GPIOA_CLK_ENABLE();

__HAL_RCC_GPIOB_CLK_ENABLE();
```

- Queste macro abilitano il clock per i diversi gruppi di GPIO (C, H, A, B).
- Senza l'abilitazione del clock, i GPIO non funzioneranno.

```
HAL_GPIO_WritePin(GREEN_LED_GPIO_Port,                GREEN_LED_Pin,
GPIO_PIN_RESET);
```

- Imposta il livello iniziale del pin **GREEN_LED** su **RESET** (LOW).
- **GREEN_LED_GPIO_Port** e **GREEN_LED_Pin** sono macro definite automaticamente da STM32CubeMX per identificare la porta e il pin associati al LED.

```
GPIO_InitStruct.Pin = GREEN_LED_Pin;

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

GPIO_InitStruct.Pull = GPIO_NOPULL;

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GREEN_LED_GPIO_Port, &GPIO_InitStruct);
```

- **Pin**: Specifica il pin del LED verde.
- **Mode**: Configura il pin come **output push-pull** (OUTPUT_PP).
- **Pull**: Nessuna resistenza pull-up o pull-down attivata.
- **Speed**: Imposta la velocità del pin su **LOW** (sufficiente per un LED).
- **HAL_GPIO_Init**: Inizializza il pin con i parametri definiti.

Flashing LED

Having completed the configuration of the PA5 pin to which the green LED is connected, we can proceed with the implementation of the logic.

To flash an LED, we toggle the state of the GPIO pin between high (HIGH) and low (LOW).

A delay is needed between transitions to make the flashing visible.

With this logic we must follow the following steps:

- You write **1**on the register to turn on the LED.
- Wait for a certain amount of time (e.g. 500 ms).
- You write **0**on the register to turn off the LED.
- Repeat.

For the implementation we use the already available functions of the HAL (Hardware Abstraction Layer)

To turn the LED on or off, we use the write function of a pin. This function takes the input*brings*to which the pin belongs, the pin itself and the state you want to write.

```
HAL_GPIO_WritePin(GREEN_LED_GPIO_Port, GREEN_LED_Pin, GPIO_PIN_SET);
```

When we assign a name to the pin during configuration,CubeMXgenerates a macro for the port that includes the pin and a macro for the pin. In this case, the label assigned to pin PA5 isGREEN_LED,andCubeMXgenerated the macroGREEN_LED_GPIO_portwhich identifies the door andGREEN_LED_Pinwhich identifies the pin.

The two default macros, GPIO_PIN_SETandGPIO_PIN_RESETrespectively raise and lower the output of a digital pin.

Below is the code to insert into the loop in the USER CODE section.

```
/* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
   HAL_GPIO_WritePin(GREEN_LED_GPIO_Port, GREEN_LED_Pin, GPIO_PIN_RESET);
   HAL_Delay(500);
   HAL_GPIO_WritePin(GREEN_LED_GPIO_Port, GREEN_LED_Pin, GPIO_PIN_SET);
   HAL_Delay(500);
 /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
 }
 /* USER CODE END 3 */
```

To insert the delay between turning on and off and vice versa, we use another HAL function. The functionvoid HAL_Delay(uint32_ydelay) takes as input the waiting time expressed in milliseconds represented by a 32-bit integer.

The LED flashing can also be obtained using another HAL function.
The void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin) function changes the state of the digital output pin. If the current state is high, the next state will be low, and vice versa.

Below is the code to insert into the loop in the USER CODE section.

```
/* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
   HAL_GPIO_TogglePin(GREEN_LED_GPIO_Port,
GREEN_LED_Pin);
   HAL_Delay(1000);
   /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
 }
 /* USER CODE END 3 */
}
```

Fill out and download theprograma on the Core tab.

Chapter 4Serial communication

In this project, we'll learn how to configure the STM32F401RE microcontroller's USART2 peripheral for asynchronous serial communication to send and receive data from a PC terminal. We'll use a terminal (e.g., PuTTY or a generic serial monitor) to send text commands to the microcontroller and, in response, turn the green LED (LD2) on the Nucleo board off or on.

Project Objectives

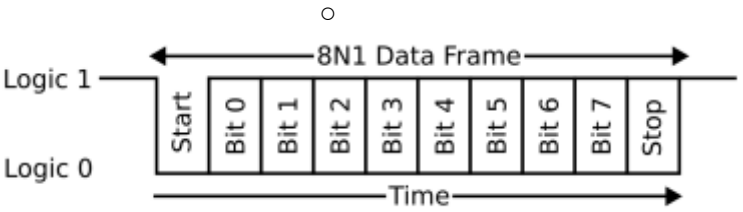
- Understand how a UART/USART serial port works in asynchronous mode.
- Setting up a basic project inSTM32CubeIDE with USART2 (PA2=Tx, PA3=Rx).
- Set communication parameters: baud rate,bitsof data, parity, stop bit.
- Send strings from the microcontroller to the PC (e.g. welcome message).
- Receive commands from the PC to control the LED (e.g. “ON”/“OFF”).

Basic Theory: Asynchronous Serial Communication (UART)

Universal Asynchronous Receiver Transmitter (UART) communication is one of the simplest and most popular methods for connecting microcontrollers and PCs.

Each byte of data is transmitted as a sequence of bits on the Tx channel and encapsulated in a frame composed of:

- **start bit**(level “0”),
- bo - b7**data bit**,
- possible**parity bit** (None, Even, Odd),
- 1 or 2**stop bit**(level “1”).



The transmission speed is measured in bits per second and is called*Baud rate.* (e.g. 9600, 115200 bps). Must be set to the same value on both the transmitter and receiver.
The Tx/Rx lines of the USART2 (hardware) port are assigned to the PA2 (USART2_TX) and PA3 (USART2_RX) pins respectively.

Reception Management Mode

There are two possible ways to receive bytes:

- **Polling:** The CPU repeatedly calls the function`HAL_UART_Receive()`(thecheck the flag`RXNE`)in a loop. Simple to implement, but keeps the CPU busy even when there's no data.

- **Interrupt:** An interrupt is triggered upon receipt of each byte (flag`RXNE`), executing a callback (`HAL_UART_RxCpltCallback`). The CPU can perform other tasks and is interrupted only when new data arrives.

Alternatively, for high throughput, i.e., for a large amount of data transmitted, DMA mode can be used for block transfers without continuous CPU intervention. This topic is beyond the scope of this book.

In the following paragraphs we will see step by step how to:

- Enable and configureUSART2 in STM32CubeIDE,
- Write the initialization code in`MX_USART2_UART_Init()`,
- Send a welcome message on boot,
- Manage the reception of commands via interrupt to turn the LED on/off.

Setting Up the Serial Polling Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*Serial_polling*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

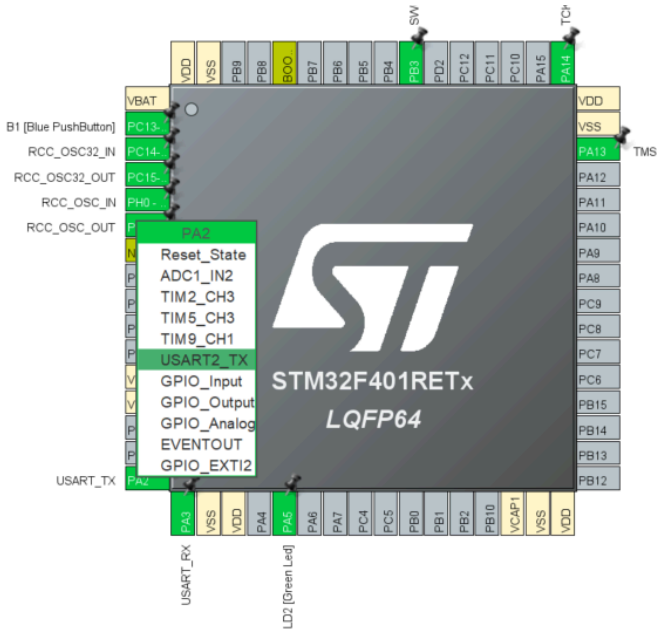
USART2 Pin Configuration

Steps to configure pinsUSART2 (PA2=Tx, PA3=Rx) in STM32CubeMXare as follows:

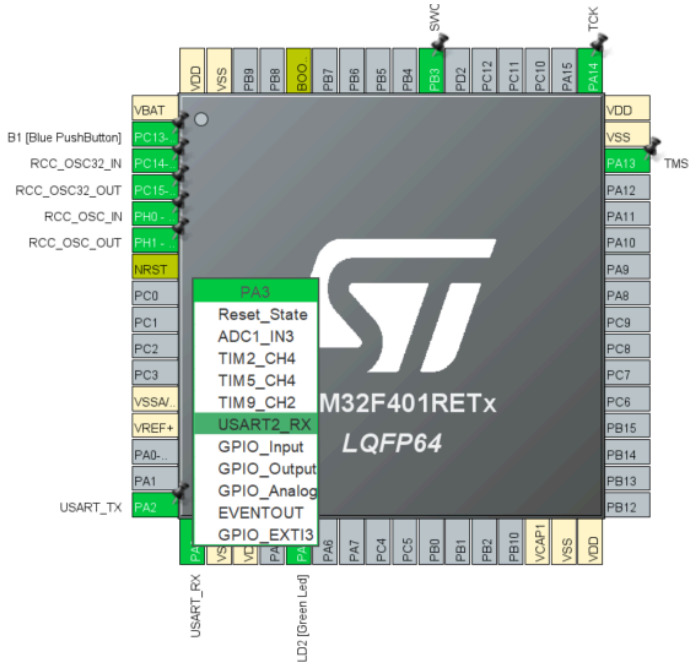
By double clicking on the file*Serial_polling.ioc*, open the configuration file and select*Pinout & Configuration Tool*.Locate the pins associated with the portUSART2,PA2 and PA3.

If you chose to perform a default configuration during project creation, the port may already be configured, resulting in green pins.

For pin PA2, the alternate function must be selected*USART2_TX*



Instead, for pin PA3, the alternate function must be selected*USART2_RX*.

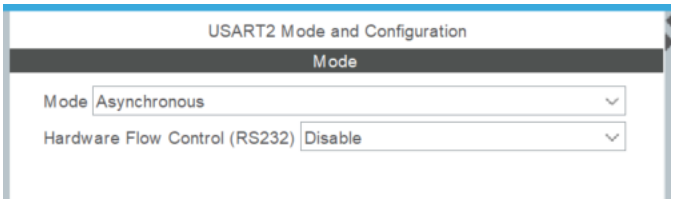


Having configured the alternatesfunctionrelated to the port pinsUSART2,we can move on to configuring the communication parameters.

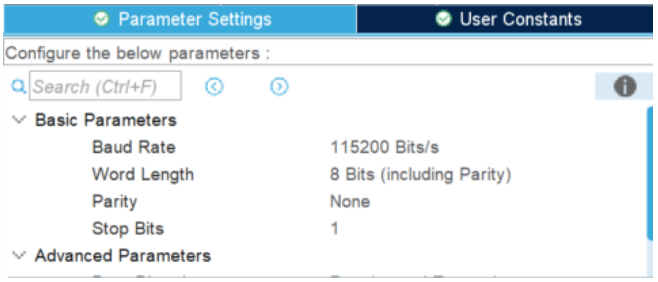
On the menu*Connectivity*, we select*USART2*



We choose asynchronous mode without enabling hardware flow control.



Finally, as parameters we select:



Finally, we configure the LED as seen in the previous chapter.
We save the project and accept the automatic code generation

At this point CubeMX will have generated the function`MX_USART2_UART_Init()`and instantiated the handle`huart2`ready to use.

```
UART_HandleTypeDef huart2;
```

We define the message to send to the client and the buffer for receiving the command. Insert the following code into the user-specific sections.

```
/* USER CODE BEGIN 1 */
    const char prompt[] = "Send command: ON or OFF\r\n";
    uint8_t rxBuf[4]; // spazio per "ON\0" o "OFF\0"
/* USER CODE END 1 */
```

We include the string handling library. We do this in the private includes section.

```
/* Private includes
----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
```

In the while loop we insert the following code

```
while (1)
{
    // 1) Send prompt
    HAL_UART_Transmit(&huart2, (uint8_t*)prompt, strlen(prompt), HAL_MAX_DELAY);

    // 2) Receive command in polling (3 characters + terminator)
    //    Here we always wait for 3 bytes; you can adjust to read until '\n'
    memset(rxBuf, 0, sizeof(rxBuf));
    do {
          HAL_UART_Receive(&huart2, &rxChar, 1, HAL_MAX_DELAY);
          if (rxChar != '\r' && rxChar != '\n' && idx < (sizeof(rxBuf) - 1)) {
              rxBuf[idx++] = rxChar;
          }
       } while (rxChar != '\n');
    rxBuf[3] = '\0'; // null-terminator

    // 3) Compare and act on the LED
    if (strcmp((char*)rxBuf, "ON") == 0)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);   // LED ON
    }
    else if (strcmp((char*)rxBuf, "OFF") == 0)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // LED OFF
    }
    // else: unrecognized command, repeat prompt
}
```

To send the prompt message we use the function`HAL_UART_Transmit()`

Its parameters are:

```
HAL_UART_Transmit(
      &huart2,      // 1) Pointer to the UART handle (here
                                USART2)
      (uint8_t*)prompt,  // 2) Pointer to the data buffer to
                                transmit (cast to uint8_t*)
      strlen(prompt),    // 3) Length of the buffer in bytes
(number
                                of characters in the prompt)
```

```
    HAL_MAX_DELAY      // 4) Maximum timeout in milliseconds
                              (blocks until completion)
);
```

- *&huart2*:Specifies which UART device to transmit to. The handle contains the settings (baud rate, word length, parity, etc.) configured in MX_USART2_UART_Init().

- *(uint8_t*)prompt*: is the pointer to the text to send, which is originally a C string (char *). The cast to uint8_t* is needed because HAL expects a byte buffer (uint8_t).

- *strlen(prompt)*: calculates how many bytes (characters) are in the prompt, not counting the '\0' terminator. This way, the function transmits exactly that many bytes.

- HAL_MAX_DELAY:“infinite” timeout value (effectively blocks the CPU until all bytes have been sent). If transmission is not possible within this time, the function will return an error.

The function`memset()`It is used to “clean” (reset) a memory area before using it, preventing “dirty” (residual) values from previous operations from remaining.

His signature is:

void *`memset`(void *s, int c, size_t n);

and the parameters are:

- **`rxBuf`**: pointer to the beginning of the array to be reset.
- **`0`**: byte value to write in each position (here zero).
- **`sizeof(rxBuf)`**: number of bytes to write to (the total size of the array).

So:

`memset`(rxBuf, 0, sizeof(rxBuf));

means “fill all the`sizeof(rxBuf)`byte of`rxBuf`with zeros”. In practice you zero the entire buffer`rxBuf`, setting each element to`0`, before receiving the new command via UART. This ensures that, even if the received command is shorter or not completed, the remaining positions remain zero (`'\0'`), avoiding readings of random values.

This code snippet is for**filtration**the characters received and to**prevent overflow**of the buffer`rxBuf`Let's analyze the three conditions:

```
if (rxChar != '\r' // 1) Do not store the carriage return
&& rxChar != '\n' // 2) Do not store the line feed
 && idx < (sizeof(rxBuf)- 1)) // 3) Make sure you leave space
```

```
                                    for the terminator '\0'
{
rxBuf[idx++] = rxChar; // 4) Store the valid character and
                              increase the index
}
```

- **`rxChar != '\r'`** Discard the “carriage return” character (`carriage return`, ASCII 0x0D) sent from many terminals when you press Enter.
- **`rxChar != '\n'`** Discard the “line feed” character (`line feed`, ASCII 0x0A), also generated at the end of the line.
- **`idx < (sizeof(rxBuf) - 1)`** Ensures that the index`idx`you always remain**minor**of the buffer length minus one, so as to reserve one byte for the null terminator (`'\0'`) at the end of the string. This avoids writing beyond the bounds of the array (`buffer overflow`).
- **`rxBuf[idx++] = rxChar;`** If all conditions are true, the received character is stored in`rxBuf`to the position`idx` and then `idx`is increased (`idx++`), preparing to write at the next available slot.

In short, this`if`It makes sure to save only the significant characters of the command (e.g. “O”, “N”, “F”, “F”) in the buffer, ignoring line terminators, and not to exceed the capacity of the array reserved for the string.

The function call

`HAL_UART_Receive`(&huart2, rxBuf, 3, HAL_MAX_DELAY);

usa l’API HAL in **blocking mode**to read data from the UART peripheral. In detail:

The parameters are

- **`&huart2`**: Pointer to the**handle**of USART2, which contains all the configurations (baud rate, frame formatting, etc.) and the status of the device.
- **`rxBuf`**: Pointer to the receive buffer, which is the array where the received bytes will be stored. It must be large enough to hold the expected number of bytes.
- **`3`**: Number of bytes to receive. The function will wait until it has received exactly 3 characters (or times out).
- **`HAL_MAX_DELAY`**: Maximum timeout in milliseconds for receiving: using`HAL_MAX_DELAY`The call never times out, blocking execution until the 3 bytes

have been queued by the UART driver.

What happens at runtime?

- The UART driver activates reception and waits for 3 incoming bytes.
- Each time a bit arrives, the hardware reconstructs it into the corresponding byte and stores it in the receive register.
- HAL_UART_Receive transfers each byte from the hardware register to the software buffer (`rxBuf`).
- Only after reading all 3 bytes does the function return control to the program, which continues executing the next line.

If for some reason the bytes don't arrive (e.g. a line disconnected), the code remains stuck on this call until the data arrives. If you wanted to handle a timeout instead, you could replace`HAL_MAX_DELAY`with an explicit millisecond value.

```
rxBuf[3] = '\0';
```

It is used to transform the byte array`rxBuf`in a real “C string”**finished**from the null character, so that all string manipulation functions (such as`strcmp`, `shine`, etc.) know where to stop.

This code block compares the string received via UART (`rxBuf`) with text commands`"ON"`and`"OFF"`and, based on the result, turns on or off the LED connected to PA5:

```
if (strcmp((char*)rxBuf, "ON") == 0)
{
     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    // LED
ON
}
else if (strcmp((char*)rxBuf, "OFF") == 0)
{
     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // LED
OFF
}
```

- **`strcmp((char*)rxBuf, "ON") == 0`**
  - `strcmp`compares two C strings character by character.
  - Returns**0**if the strings are exactly the same.
    Here you can check if`rxBuf` (cast a `char*`) is identical to the string`"ON"`.

- **`HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);`**
  - If the condition is true (received`"ON"`), calls the HAL to bring the PA5 pin high, turning on the LED (macro`GPIO_PIN_SET`).

- **`else if (strcmp((char*)rxBuf, "OFF") == 0)`**

    - If the first condition is false, check if `rxBuf` corresponds to `"OFF"`.

- **`HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);`**
    - If he received `"OFF"`, set PA5 to low level (`GPIO_PIN_RESET`), turning off the LED.

If the content of `rxBuf` it is neither `"ON"` nor `"OFF"`, the code enters neither block and simply jumps to the next loop to receive a new command.

Fill out and download the programa on the Core tab.

To interact with the Nucleus you need to open a serial client.

Configuring the Serial Interrupt Project

OpenSTM32CubeIDE, select the previously created workspace and create a project with the name *Serial_interrupt*. If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

USART2 Pin Configuration

The door USART2 It must be configured like the previous example. The LED configuration also doesn't differ from the cases seen previously.

In the case of interrupts, the control over the command is done within the callback function which is called every time a byte has been received.
The callback function that the programmer must implement is

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
   if (huart->Instance == USART2)
   {
       // 1) Filter out CR/LF and store valid chars
       if (rxChar != '\r' && rxChar != '\n' && idx < (sizeof(rxBuf)-1))
       {
           rxBuf[idx++] = rxChar;
       }
       // 2) If we saw LF, treat as end-of-command
       if (rxChar == '\n')
       {
           rxBuf[idx] = '\0';  // null-terminate
           // 3) Compare and act
           if (strcmp(rxBuf, "ON") == 0)
           {
               HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
           }
           else if (strcmp(rxBuf, "OFF") == 0)
           {
               HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
           }
           // 4) Reset buffer for next command
           idx = 0;
           memset(rxBuf, 0, sizeof(rxBuf));
       }
       // 5) Re-arm the UART receive interrupt for next byte
       HAL_UART_Receive_IT(&huart2, &rxChar, 1);
   }
}
```

Interrupt reception is instead initiated by invoking the HAL function

```
HAL_UART_Receive_IT(&huart2, &rxChar, 1);
```

Chapter 5 GPIO input - polling

In this project, we'll learn how to configure a GPIO pin as a digital input to read the state of the Nucleo board's user button. We'll use the button to turn the green LED on and off..

Project Objectives

- Understand how a GPIO pin works as a digital input.

- Setting up a basic project inSTM32CubeIDE.
- Read the status of the blue buttonNucleoF401REusing C code.

Basic Theory: Reading a GPIO Input

When you intend to use a GPIO pin as a digital input, you must take into account the two reading modes:

- **Polling:**The microcontroller continuously checks the state of the GPIO in a repetitive loop. This is a synchronous method; the CPU is constantly busy reading the pin, even when the state doesn't change.
- **Interrupt:** The microcontroller executes the main code and "stops" only when the GPIO changes state, generating an interrupt signal. It's an asynchronous method; the CPU takes care of other tasks until notified by the GPIO.

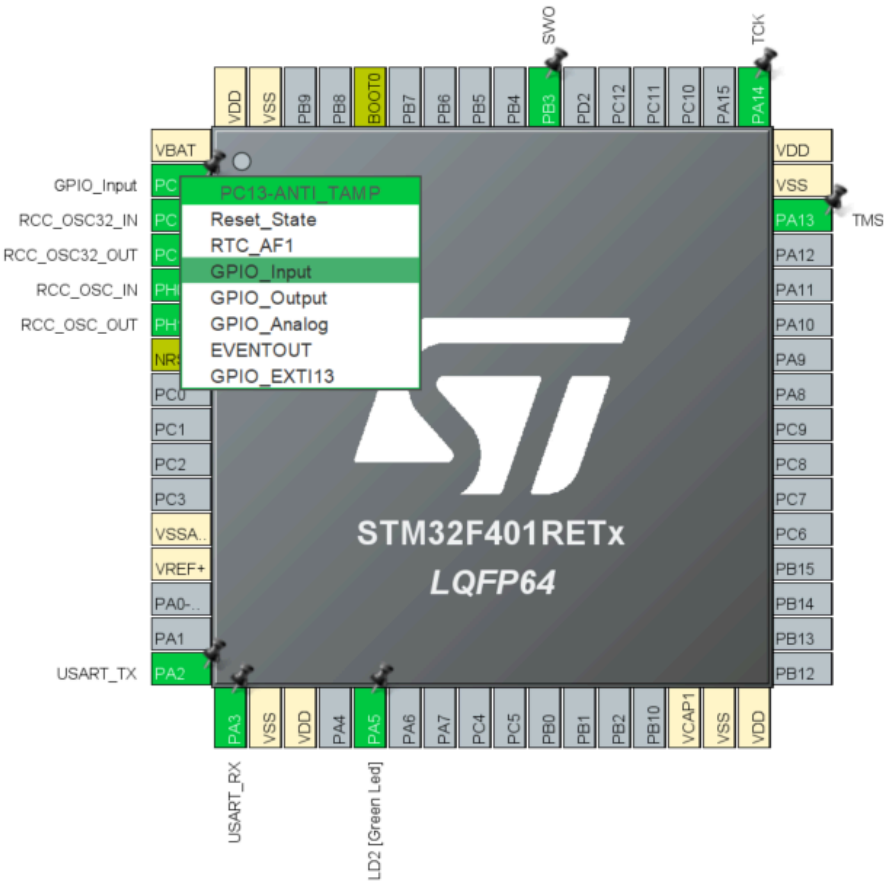
Setting Up the Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*UserButton_polling*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

Configuring the User Button pin

Open the Pinout & Configuration Tool and locate the pin associated with the user button on the boardNucleoF401RE,The User Button is connected to the PC13 pin.

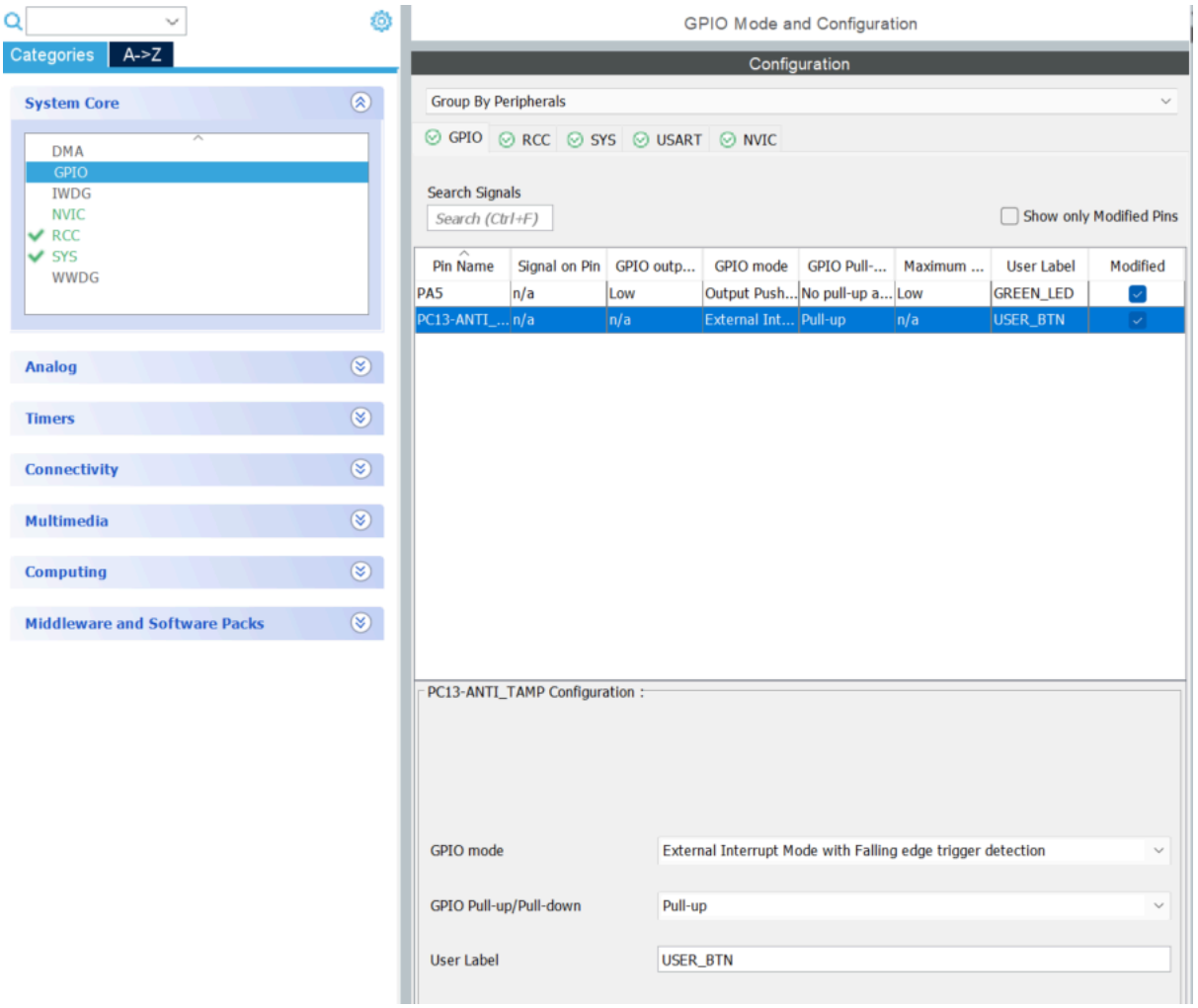
Click on the PC13 pin on the microcontroller map and set it asGPIO_Input.The pin may already have been configured by CubeMX.



If you want to turn on the LED, you also need to configure the corresponding pin as seen previously, assigning the label to the LED pin*GREEN_LED*.

In the card**GPIO Configuration**, select the pin**PC13**and configure the following parameters:

- **Mode:** `Input Mode`
- **Pull-up/Pull-down:** `Pull-up`(optional, but often recommended to improve button stability, as the button connects the pin to GND when pressed).
- **Label:**You can optionally assign a label like`USER_BTN`.



Once the pin configuration is complete, we can generate the code. To do this, simply save the project, or in general click on**Project -> Generate Code**.

STM32CubeIDEwill generate the code needed to configure the pin in input mode.

Reading the Button

In the file `main.c`(the`main.cpp`for a C++ project), in the main loop (`while(1)`), we read the state of the button using the function`HAL_GPIO_ReadPin`.

When the button is pressed the pin is brought to the low state encoded by the macroGPIO_PIN_RESET.So we will need to introduce a check to verify that the pin has assumed this value.

To turn on the LED, we use the toggle function of the pins associated with the green LED.

Below is the code to insert into the loop in the USER CODE section.

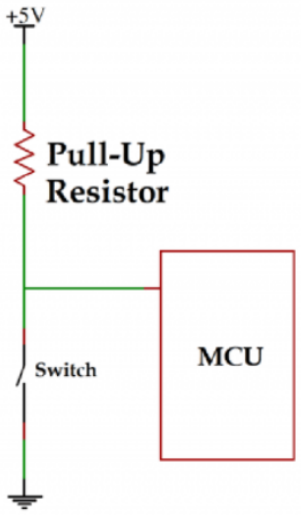
```
/* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
  if (HAL_GPIO_ReadPin(USER_BTN_GPIO_Port, USER_BTN_Pin) ==
GPIO_PIN_RESET){
        HAL_GPIO_TogglePin(GREEN_LED_GPIO_Port, GREEN_LED_Pin);
  }
  HAL_Delay(100);
      /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
 }
 /* USER CODE END 3 */
}
```

This code is for educational purposes only, in practice it should be improved by adding the*debouncing*.
Fill out and download theprograma on the Core tab.

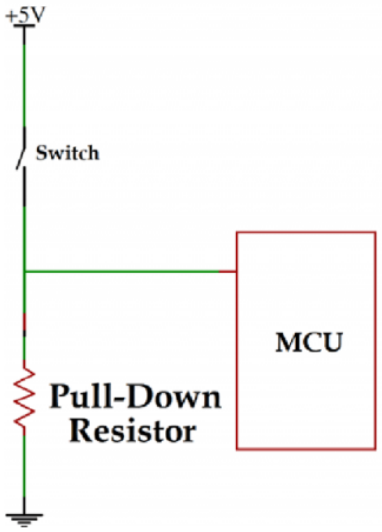
In-depth study: Pull-up/down resistors

Pull-up (or pull-down) resistors are used to ensure that a pin's signal remains stable in a well-defined state (high or low) when it is not actively connected to an external source, such as when a button is not pressed.

- **Behavior without pull-up resistor**When the button is not pressed, the pin is disconnected (in an "open" circuit) and its logical state is undetermined. This phenomenon is known as**floating state**(floating). A floating pin can detect noise signals or random voltage variations, leading to unstable or unpredictable readings.
- **Function of the pull-up resistor**The pull-up resistor is connected between the input pin and the positive supply voltage (+Vcc). When the button is released (not pressed), the resistor ensures that the pin is "pulled up" to the high logic state (HIGH). When the button is pressed, it directly connects the pin to GND,forcing him tolow logic state (LOW).



- **Button stability**With a pull-up resistor, the circuit ensures a clear and stable transition between the HIGH (released) and LOW (pressed) states. This eliminates noise and indeterminate states. The same logic applies to the pull-down resistor, which instead "pulls" the pin to GND, ensuring the LOW state when the button is not pressed.



Many modern microcontrollers such as the STM32, offer internal software-configurable pull-up/pull-down resistors.

Chapter 6GPIO input - interrupt

In this project, we'll learn how to configure a GPIO pin to generate a digital interrupt in response to pressing the Nucleo board's user button. We'll use this interrupt to turn the green LED on and off responsively and efficiently.

Project Objectives

- Understand how interrupts associated with a GPIO pin work.
- Setting up a basic project inCubeIDEwith interrupt handling.
- Use an interrupt to detect when the blue button is pressedNucleoF401REand handle events using C code.

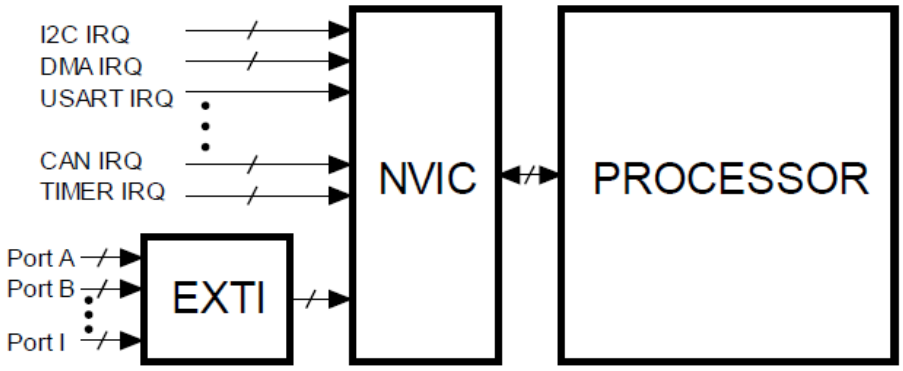
Basic Theory: Generating an Interrupt

An interrupt is a signal that allows the microcontroller to interrupt the execution of the main code to respond to a specific event, such as a GPIO pin changing state. This approach is asynchronous; the CPU can continue performing other tasks until it is notified of the event via the interrupt.

In the STM32F401RE microcontroller, interrupt management is entrusted to the**NVIC (Nested Vectored Interrupt Controller)**, That prioritizeand dispatches interrupts. When a GPIO pin configured to generate an interrupt detects a change in state (for example, a rising or falling edge), the microcontroller:

- **Detect the event**: The GPIO system registers the state change configured as a trigger (e.g. rising or falling edge).
- **Submit the request to the NVIC**: The GPIO notifies the NVIC that an interrupt event has occurred.
- **Executes the Interrupt Service Routine (ISR)**: The CPU temporarily suspends the main code to execute the associated functionall'interrupt(ISR), which manages the event.
- **Return to main code**: After completingl'ISR,The CPU picks up where it left off.

This system ensures a fast response to events without compromising overall efficiency, since the CPU does not have to continuously check the state of the GPIO (polling method).



The diagram in the figure represents the flow and management of interrupts in STM32 microcontrollers, using the main blocks involved:I AM OUT,NVIC, and the PROCESSOR.

- EXTI (External Interrupt/Event Controller):This block is responsible for handling interrupts generated by thepin GPIOInterrupt signals can come from various*port* (is.Port A, Port B, up to Port I), or from the pins configured to generate interrupt events. The taskof EXTIis to identify the pin that generated the interrupt and notify theNVIC.
- NVIC (Nested Vectored Interrupt Controller): This is the main component that manages andprioritizeall system interrupts. It receives requests bothfrom EXTI(external interrupts) or from other microcontroller peripherals, such as:
    - **I2C IRQ**: Interrupt generated by the I2C module.
    - **DMA IRQ**: Interrupt related to direct memory transfers (DMA).
    - **USART IRQ**: USART Serial Communication Module Interrupt.
    - **CAN IRQ**: CAN communication module interrupt.
    - **TIMER IRQ**: Interrupt generated by timers.
- The NVIC assigns aprioritytо interrupts, to ensure that the most critical ones are handled first. It can also handle nested interrupts, that is, suspending an ISR (Interrupt Service Routine) to execute another interrupt with a higher priority.
- PROCESSOR: This is the core of the microcontroller, responsible for executing the main code. Whenthe NVICnotifies an interrupt, the PROCESSOR temporarily suspends the normal flow of the program and switches to executing l'InterruptService Routine (ISR), or the interrupt handling function.Once finishedl'ISR, The PROCESSORresumes the main code where it left off.

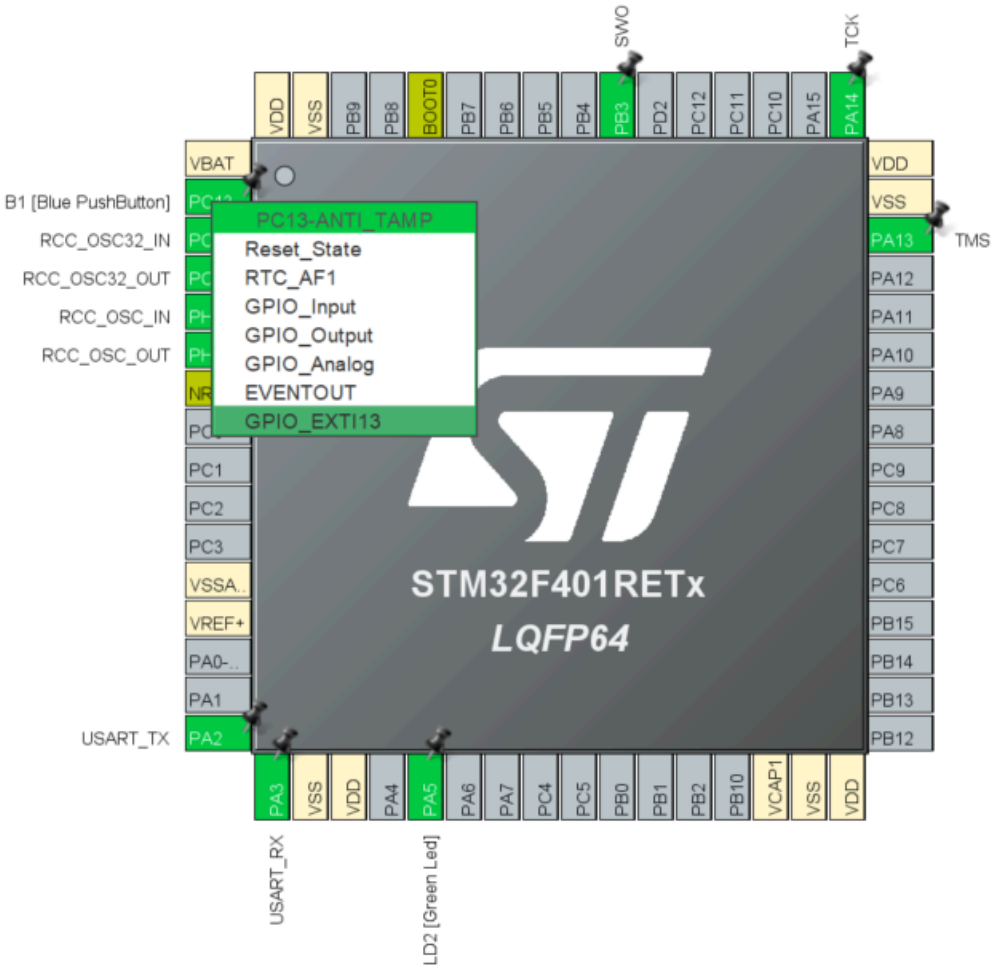
Setting Up the Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*UserButton_interrupt*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

Configuring the User Button pin

Open the Pinout & Configuration Tool and locate the pin associated with the user button on the boardNucleoF401RE,The User Button is connected to the PC13 pin.

Click on the PC13 pin on the microcontroller map and set it asGPIO_EXTI13.The pin may already have been configured by Cube.

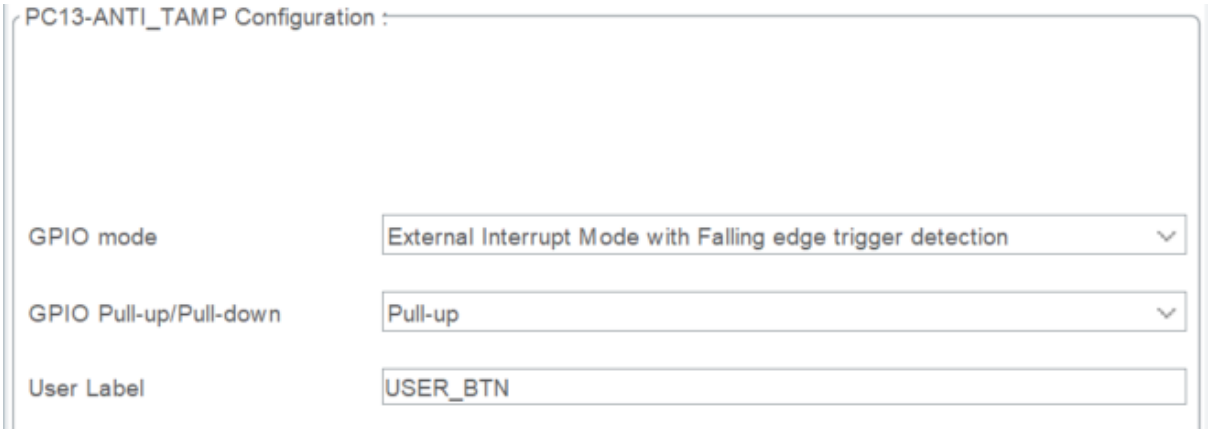


If you want to turn on the LED, you also need to configure the corresponding pin as seen previously, assigning the label to the LED pin*GREEN_LED*.

In the card**GPIO Configuration**, select the pin**PC13**and configure the following parameters:

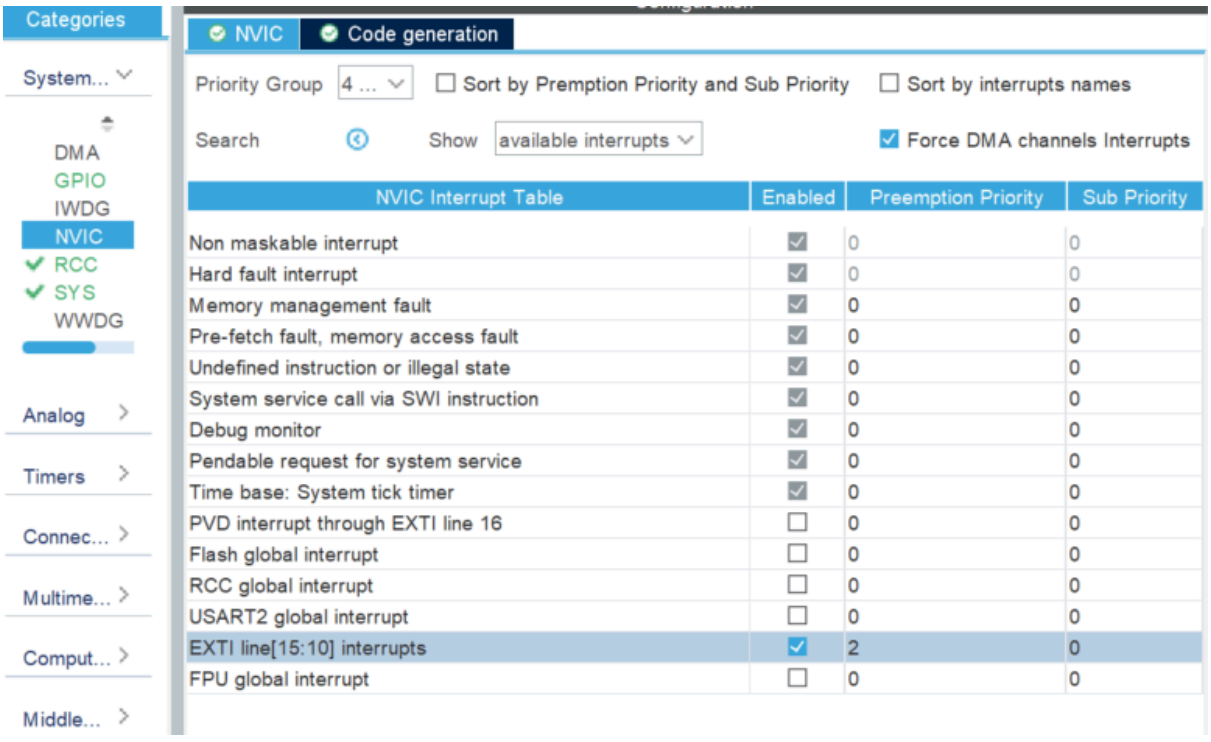
- **Mode:** `External Interrupt Mode with Falling Edge trigger detection`

- **Pull-up/Pull-down:** `Pull-up`(optional, but often recommended to improve button stability, as the button connects the pin to GND when pressed).
- **Label:**You can optionally assign a label like`USER_BTN`.



At this point all that remains is to enable the interrupt.

- In the card**NVIC Settings**(in the GPIO configuration or in the main configuration tab).
- Enable interrupt for**EXTI Line[15:10]**and set the desired priority.



The **priority**and the**sub-priority**Interrupts are two levels of configuration offered by the NVIC (Nested Vectored Interrupt Controller) in STM32 microcontrollers. These allow you to define which interrupts should be handled first in the event of conflicts or simultaneous events. For this project, we can consider priorities*2*for the interrupt generated by the button.

Let's save the project and generate the code.

Button Management

The initialization function contains the pin definition*USER_BTN*and the qualificationdell'interrupt.

```
GPIO_InitStruct.Pin = USER_BTN_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(USER_BTN_GPIO_Port, &GPIO_InitStruct);

/*Configure GPIO pin : LD2_Pin */
GPIO_InitStruct.Pin = LD2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);

/* EXTI interrupt init*/
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 2, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

The function remains to be implementedcallback, which is automatically invoked when an interrupt occurs.

The function to be defined in the file`main.c` And `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`This function is designed to handle the event associated with the generated interrupt. In our project, the goal is to change the state of the green LED every time the button is pressed. To do this, we will use the functiontoggleon the GPIO pin associated with the green LED,invertendone lostate at each callback call.

```
/* Private user code
---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
   if (GPIO_Pin == GPIO_PIN_13) {
       // It changes the GREEN led status
       HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
   }
}
/* USER CODE END 0 */
```

Fill out and download theprograma on the Core tab

Deep Dive: Interrupt Priority - Basic Concepts

When an interrupt occurs, the microcontroller must decidewhich ISR (Interrupt Service Routine) to execute. This is determined based on the priority and sub-priority configured in the NVIC.

The primary priority determines which interrupt takes precedence over others. For example:

- An interrupt with priority 1 takes precedence over one with priority 2.
- If two interrupts with different priorities occur simultaneously, the microcontroller executes the one with the higher priority (lower number) first.

Sub-priority is used to manage the execution order of interrupts that have thesame main priority.In case of simultaneous interrupts with the same main priority, the microcontroller uses the sub-priority to decide which ISR to execute first.

In STM32, the NVIC system supports up to 16 main priority levels (depending on the device) and multiple sub-priority levels.

The number of main and sub-priority levels is determined by thePriority Groupconfigured. This configures the number of bits assigned for priority and sub-priority.

Priority Grouping divides the bits available for priority into two fields:

- **Bits per Main Priority**: Determine the main priority level.
- **Bits per Sub-Priority**: Determine the relative priority between interrupts of the same main group.

The assignment is configured with the function:

```
HAL_NVIC_SetPriorityGrouping(uint32_t PriorityGroup);
```

The available groups are:

Priority Group	**Bits per Main Priority**	**Bits per Sub-Priority**
NVIC_PRIORITYGROUP_0	0	All bits per sub-priority
NVIC_PRIORITYGROUP_1	1	3
NVIC_PRIORITYGROUP_2	2	2
NVIC_PRIORITYGROUP_3	3	1
NVIC_PRIORITYGROUP_4	4	0 (main priority only)

If the priority group is not defined, the typical default value isGROUP_2which assigns two bits for priority and two for sub-priority. This obviously means that 2 bits will be possible^2 = 4 *priority*and 2^2 = 4 *sub-priority*.

Chapter 7 Analog input - ADC

In this project, we'll learn how to configure a GPIO pin to read an analog signal. We'll use a potentiometer to turn the green LED on and off depending on whether the read value exceeds a predefined threshold or not.

Project Objectives

- Understand how the Analog-to-Digital Converter (ADC) works on a microcontroller.
- Setting up a basic project inCubeIDEfor reading analog signals.
- Use a potentiometer to control the value of an analog signal and implement decision logic to turn an LED on or off based on a predefined threshold.
- Configuring the UART port for sending messages

Basic Theory: Reading an Analog Signal

An analog signal is a continuous signal that can take on an infinite number of values within a range. To process it, the microcontroller uses an analog-to-digital converter (ADC), which transforms the analog signal into a digital value that can be interpreted by the code. By appropriately configuring the ADC module, it is possible to read the value generated by a potentiometer connected to an analog pin and use this value to control specific behaviors, such as turning an LED on or off.

What is an ADC (Analog-to-Digital Converter)?

An ADC (Analog-to-Digital Converter) is an electronic component that transforms analog signals (continuous values) into digital signals (discrete values) that can be interpreted by a microcontroller. In the case of the STM32F401RE, the microcontroller is equipped with a 12-bit ADC, which allows the analog value to be represented in a range from 0 to 4095.

The main features of the ADC in the STM32F401RE are:

- **Resolution**: 12 bit (can also be configured to 6, 8 or 10 bit to reduce conversion times).

- **Channels**: Up to 16 channels, allowing you to acquire signals from multiple analog inputs.
- **Conversion modes**:
    - Single Conversion Mode.
    - Continuous Conversion Mode.
    - Regular and/or injected mode conversion.
- **Timer trigger**: Ability to synchronize conversions with an internal timer.
- **DMA support**Data transfer can be automated via DMA (Direct Memory Access).

What is an ADC channel?

An ADC channel is a physical or logical input associated with a microcontroller pin. Each channel corresponds to a specific analog source that the ADC can convert. The channels available on the STM32F401RE include:

- **Analog inputs**: Associated with GPIO pins configured in analog mode (for example, PA0 is associated with the ADC1_IN0 channel).
- **Internal channels**: Such as internal temperature sensor, VREFINT (internal voltage reference) and VBAT (battery supply voltage).

Below is the channel map of the ADC1 of the F401RE microcontroller

ADC Channel	Associated GPIO pin
ADC1_IN0	PA0
ADC1_IN1	PA1
ADC1_IN2	PA2
ADC1_IN3	PA3
ADC1_IN4	PA4
ADC1_IN5	PA5
ADC1_IN6	PA6
ADC1_IN7	PA7
ADC1_IN8	PB0
ADC1_IN9	PB1
ADC1_IN10	PC0
ADC1_IN11	PC1

ADC1_IN12	PC2
ADC1_IN13	PC3
ADC1_IN14	PC4
ADC1_IN15	PC5

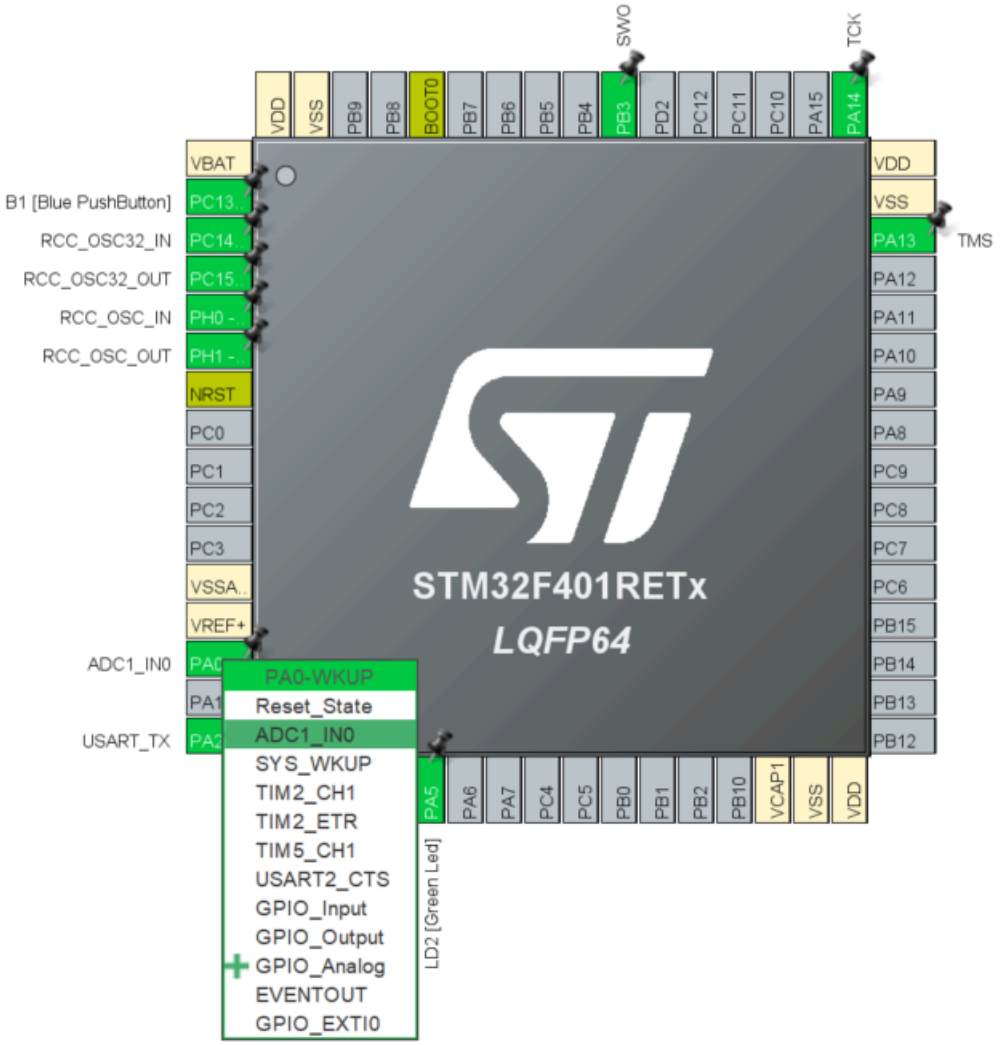
Setting Up the Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*Analog_input*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

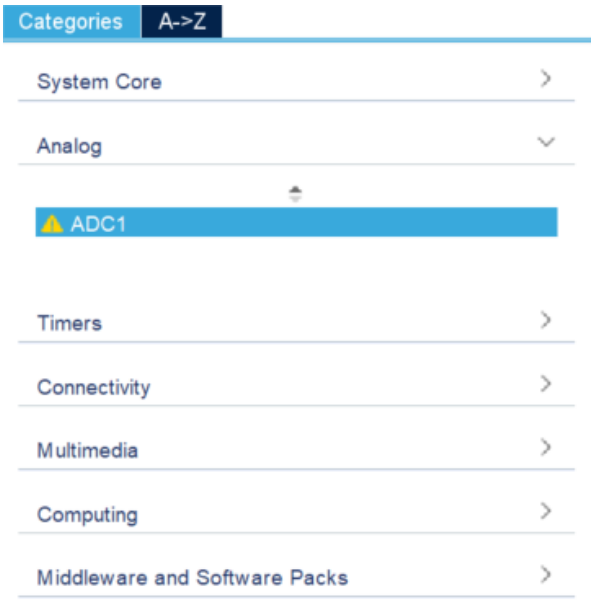
A0 pin configuration

Open the Pinout & Configuration Tool and locate the PA0 pin.

Click on the PA0 pin on the microcontroller map and set it as ADC1_IN0.



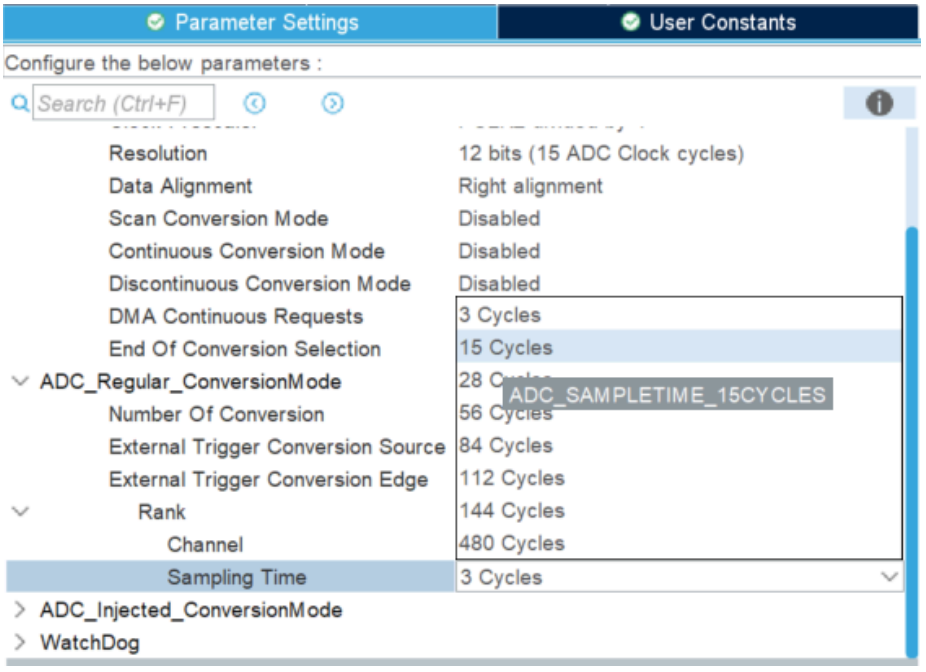
Select ADC1, the only one available for the pin*PA0*.



Ed in *Mode*make sure that*IN0*be selected. We need to select*IN0*and not the other channels, because the channel*0*it is the one associated with the pin*PA0*



In *Parameters Settings*we leave everything unchanged except the parameter*Sampling Time*since if you want to use all 12 bits of the converter, 15 cycles will be necessary



The *UART2*and the*LED*green ones are already initialized, having requested initialization to the values of*default*during project creation. If necessary, refer to the chapters dedicated to their initialization.
To verify that the LED and the UART2 pin are initialized, simply check that the respective pins are colored green.

We're ready to generate the code. We do this by saving the project, in which caseCubeIDEit will ask us if we want to generate it, orexplicitly askingon the menu*Project/Generate Code*.

If we want to print the value read by the ADC on the screen, we will use functions contained in the libraries*stdio*and*string*We must therefore proceed to include the respective header files in the include section provided for the*USER CODE*.

```
/* Private includes
----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */
```

To use the value read from the ADC, we need two private variables. One of type*uint16_t*to store the read value, encoded in 12bits,and an array of characters for the message to be sent. We insert them into the section allocated to*Private*

```
/* USER CODE BEGIN PV */
uint16_t measure = 0;
char msg[20];
/* USER CODE END PV */
```

To read the ADC we need to add the following code in the main loop section.

```
while (1)
 {
   /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
       HAL_ADC_Start(&hadc1);
       HAL_ADC_PollForConversion(&hadc1, 20);
       measure = HAL_ADC_GetValue(&hadc1);
       sprintf(msg, "Value read: %hu\r\n", measure);
       HAL_UART_Transmit(&huart2 , (uint8_t *)msg, strlen(msg), HAL_MAX_DELAY);
       HAL_Delay(50);
 }
```

Start the analog-to-digital converter (ADC) withHAL_ADC_Start, so as to start converting the analog signal into a digital value.

Wait for the conversion to complete usingHAL_ADC_PollForConversion, with a 20 millisecond timeout to avoid infinite locks.

Retrieve the converted value viaHAL_ADC_GetValueand stores it in the "measure" variable.

Formats the read value into a string usingsprintf,preparing it for broadcast.

Transmits the formatted message to the UART port(HAL_UART_Transmit)with a maximum timeout, ensuring that the message is sent successfully.

Introduces a short pause of 50 milliseconds(HAL_Delay)before performing any further operations.

Fill out and download theprograma on the Core tab.

To view messages sent by the Nucleo card, you will need to use a client, open the connection on the port assigned to the Nucleo by the operating system, selecting 115200 bps as the connection speed.

Chapter 8Timers

In this chapter, we will delve into the use of timers on the STM32 microcontroller, illustrating the fundamental concepts and guiding you through the practical configuration of the project.

Project Objectives

The main goal of this project is to help you experiment with and understand how timers work in the STM32 environment. Specifically, you'll learn to:

- **Generate periodic events:**Set up the timer to perform actions at regular intervals.
- **Handling interrupts:**Configure timer-derived interrupts to optimize task handling without overloading the processor.
- **Experiment in a practical way:**Put theoretical knowledge into practice with real-world exercises that include timer configuration using tools such asSTM32CubeIDEand the use of theMATTER.

Basic Theory: What is a Timer?

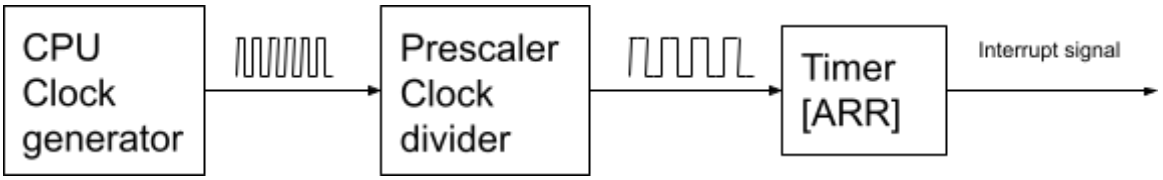
Timers are essential hardware devices for time control within embedded systems, as they allow you to count clock pulses and measure time intervals precisely.

With a timer, we can both measure time intervals and generate timed events by counting the pulses of a clock signal with a known frequency. By knowing the clock frequency and counting the number of pulses, we can precisely calculate the elapsed time interval.

Using a timer correctly involves balancing the clock frequency, prescaler, and autoreload rate to achieve the desired timing resolution and ensure efficient event handling, which is essential for the proper functioning of embedded applications.

To trigger an event after a certain period of time, you set a comparison value, which corresponds to the number of pulses to be counted, and you can adjust the counting interval using a prescaler.

The prescaler reduces the input frequency by dividing the clock by a predefined factor, thus allowing for longer time intervals. Furthermore, the timer has a register called ARR (Auto-Reload Register) in which the maximum value of thecount:Once this value is reached, the counter resets and, if properly configured, generates an event (for example, triggering an interrupt). In this way, the combination of prescaler and ARR allows you to achieve the desired timing resolution and efficiently manage time-based operations.



Additionally, timers operate in multiple modes, including PWM signal generation, input capture to measure the duration of external events, and output compare to handle time-based events, providing flexibility that allows the microcontroller to perform timing operations independently and reactively. This topic will be covered in the next chapter.

Setting Up the Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*Timer*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

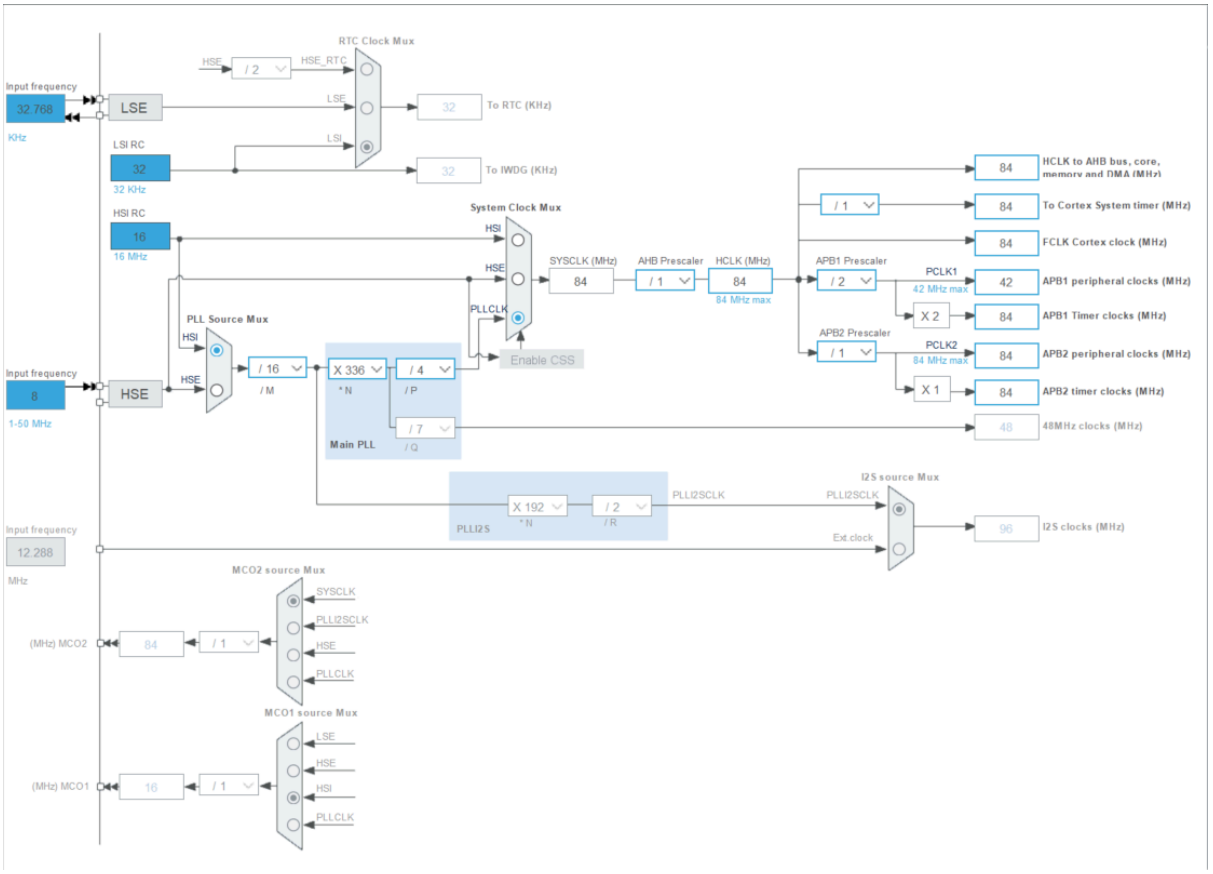
By double clicking on the file*Timer.ioc*let's access the microcontroller configuration.

From the page*Pinout & Configuration*let's move on to the page*Clock Configuration*On this page you can make some configuration changes to the peripheral clock.

We can change some ofblocksof the*tree*, that is, those relating to the prescalers, AHB and APB1 and APB2.

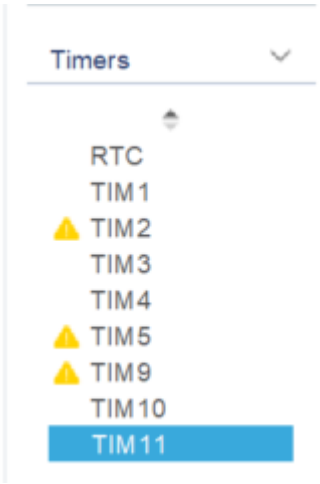
The AHB prescaler is a frequency divider applied to the main clock of the microcontroller to obtain the operating frequency.dell'AdvancedHigh-performance Bus (AHB). This bus connects the processor core to high-speed peripherals, and the prescaler reduces the system clock to match the operating specifications of different parts of the device, improving energy efficiency and ensuring system stability.

Advanced Peripheral Buses (APBs) are internal buses that connect the microcontroller core to peripherals, playing a fundamental role in the architecture of ARM-based embedded systems, such as the STM32. Specifically, these buses are designed to handle communication with devices that do not require high data transfer rates, such as UARTs, I2C, SPI, timers, and ADCs. In STM32 microcontrollers, APBs are generally divided into two groups: APB1, intended for lower-frequency peripherals, and APB2, which supports devices that operate at higher frequencies. The frequency of each bus is obtained by applying a prescaler to the main clock (derived from the APB).from the AHB),in order to adapt the operating speed to the specific needs of the connected peripherals, thus ensuring efficient communication and optimizing energy consumption.

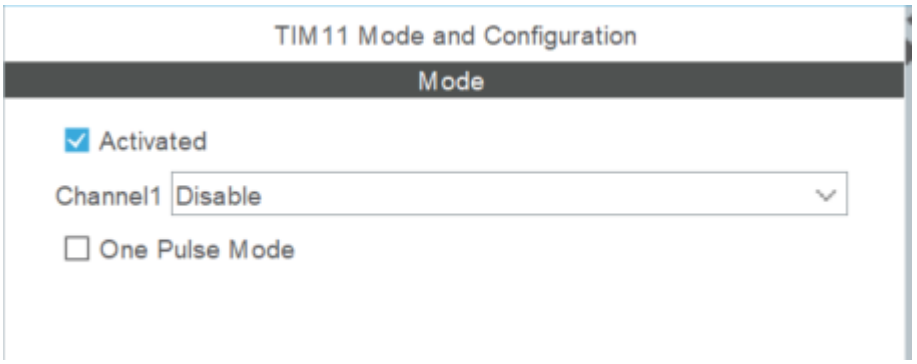


For the moment, no.we modifynothing, and we go back to the page of*Pinout & Configuration*.

We select*Timers*



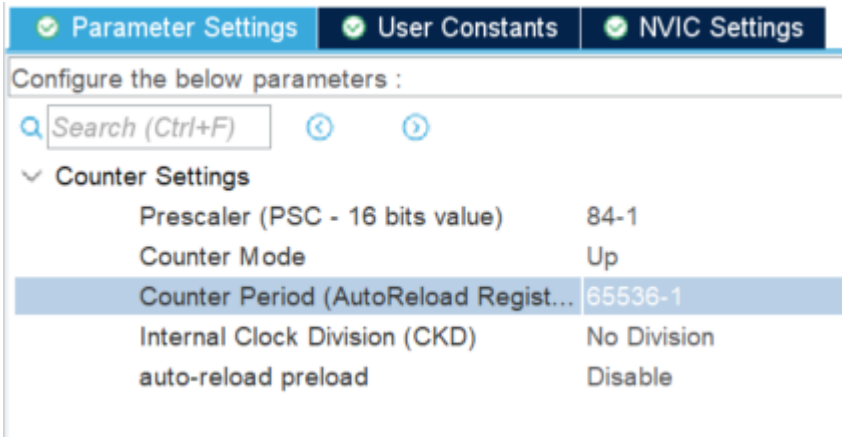
and then *TIM11*and we enter the timer configuration mode



Let's clear the field*Activated*to enable the timer and leave the other fields at their default values.

In the Parameter Settings, we define the prescaler value. Since the value starts at 0 and corresponds to a prescaler of 1, it can be convenient to insert an expression in which 1 is subtracted from the desired value. So, if we want a prescaler of 84, and need to enter 83, we enter the expression 84-1.

Let's do the same thing for valuedell'ARR.If we want to count the maximum value, that is 65536, but we have to insert 65535, we will write the expression 65536-1.



Since the clock frequency is 84 MHz, and we have selected a prescaler value of 83 (84-1), the timer will see a signal with a frequency equal to:

$freq_{desired} = 84Mz \div 84 = 1MHz$

i.e. 1 tick every$1 \div 1MHz = 1us$

Having entered ARR = 65536-1 we will be able to count at most 65536 us or 65.536 ms

In general, the following relationships hold:

$$PSC = \frac{TIM\ CLK}{Desired\ frequency} - 1$$

$$ARR = \frac{Desired\ frequency}{Timer} - 1$$

As for the*Counter Mode*, the value*Up*set the timer to count up. finally we disable the*auto-reload preload*.

We save the project and accept the code generation.

CubeIDEwill create an initialization function with the parameters entered in the file*Timer.ioc*

```
/**
 * @brief TIM11 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM11_Init(void)
{
 /* USER CODE BEGIN TIM11_Init 0 */
 /* USER CODE END TIM11_Init 0 */
 /* USER CODE BEGIN TIM11_Init 1 */
 /* USER CODE END TIM11_Init 1 */
 htim11.Instance = TIM11;
 htim11.Init.Prescaler = 84-1;
 htim11.Init.CounterMode = TIM_COUNTERMODE_UP;
 htim11.Init.Period = 65536-1;
 htim11.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
 htim11.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
 if (HAL_TIM_Base_Init(&htim11) != HAL_OK)
 {
   Error_Handler();
 }
 /* USER CODE BEGIN TIM11_Init 2 */
 /* USER CODE END TIM11_Init 2 */
```

If you want to send the elapsed time via serial port, we include the standard IO (stdio.h) library

We do it in the section*user* dedicated to includes.

```
/* Includes ------------------*/
#include "main.h"
/* Private includes ---------*/
/* USER CODE BEGIN Includes  */
#include <stdio.h>
/* USER CODE END Includes    */
```

Then you need to define some variables, the message buffer thatwe will sendon the serial port, one to store the length of the messages and one the value read by the timer

```
int main(void)
{
 /* USER CODE BEGIN 1 */
     char uart_buf[50];
     int uart_buff_len;
     uint16_t timer_val;
 /* USER CODE END 1 */
```

We then send a splash message to the serial port and start the timer*TIM11*

```
/* USER CODE BEGIN 2 */
 // SPLASH MESSAGE
uart_buff_len = sprintf(uart_buf, "Timer\r\n");
HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf, uart_buff_len, HAL_MAX_DELAY);
 // Starting the timer
HAL_TIM_Base_Start(&htim11);
 /* USER CODE END 2 */
```

At this point, we're ready to read the timer value. We take two consecutive readings separated by a known delay of 50 ms, and after measuring the elapsed time by difference, we send it to the serial port.

Finally, we wait another 1 second so that we can have spaced measurements.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
     timer_val = __HAL_TIM_GET_COUNTER(&htim11);
     HAL_Delay(50); //ms
     timer_val = __HAL_TIM_GET_COUNTER(&htim11) - timer_val;

     uart_buff_len = sprintf(uart_buf, "%u us\r\n", timer_val);
     HAL_UART_Transmit(&huart2, uart_buf, uart_buff_len, HAL_MAX_DELAY);

     HAL_Delay(1000); //ms

    /* USER CODE END WHILE */
```

Fill out and download theprograma on the Core tab.

To view messages sent by the Nucleo card, you will need to use a client, open the connection on the port assigned to the Nucleo by the operating system, selecting 115200 bps as the connection speed.

The displayed messages will report the 50ms wait time expressed in microseconds.

Chapter 9PWM

In this chapter, we will delve into the use of PWM (Pulse-Width Modulation) signals on the STM32 microcontroller, illustrating the fundamental concepts and guiding you through the practical configuration of the project.

Project Objectives

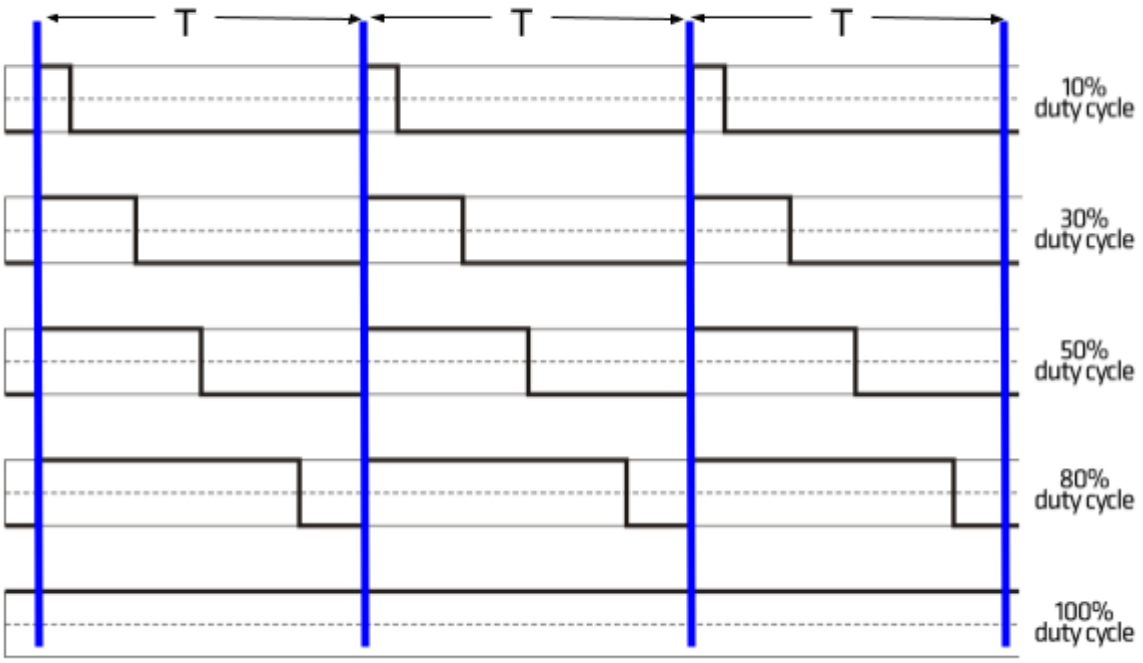
The main goal of this project is to help you experiment with and understand how PWM works in an STM32 environment. Specifically, you'll learn to:

- **Generate PWM signals:**Configure timer channels to produce PWM outputs with variable frequency and duty cycle.
- **Adjust the duty cycle:**Change the pulse width to control the light intensity of an LED.
- **Manage advanced modes:**Using PWM in center-aligned mode, generation of complementary outputs with dead-time and fault management.

Basic Theory: What is PWM

Pulse-width modulation (PWM) is a technique for generating a periodic digital signal in which the duration of the “high” level (duty cycle) is variable, while keeping the overall frequency constant.

- **PWM Frequency:**It determines the period T=1/f of the signal and is defined by how the prescaler and the Auto-Reload Register (ARR) value of the timer are set.
- **Duty-Cycle:**This indicates the percentage of time the signal remains “high” within each period, and is controlled by the comparison value (CCR). A 50% duty cycle means half the period is “high” and half is “low”.



The main registers involved in generating a PWM signal are:

- **Prescaler (PSC):**divides the base clock frequency to slow down the timer count, allowing for lower PWM frequencies or finer resolutions.
- **Auto-Reload Register (ARR):**Defines the maximum value of the counter; when it is reached, the timer resets and starts again from zero, thus establishing the PWM period.
- **Capture/Compare Register (CCRx):**contains the threshold value for the duty cycle; when the counter exceeds this value, the output changes state (from “high” to “low” or vice versa).

and therefore to generate a PWM signal we must act appropriately on these registers.

The first step is to set up the registers**PSC**and**ARR**in order to obtain the desired frequency:

$$f_{PWM} = \frac{f_{CLK}}{(PSC + 1)(ARR+1)}$$

Once the frequency is set, we need to define the duty cycle and we do this by setting the**CCR**to define when the signal should return to zero.

$$Duty\ Cycle = \frac{CCR}{ARR + 1} \times 100\%$$

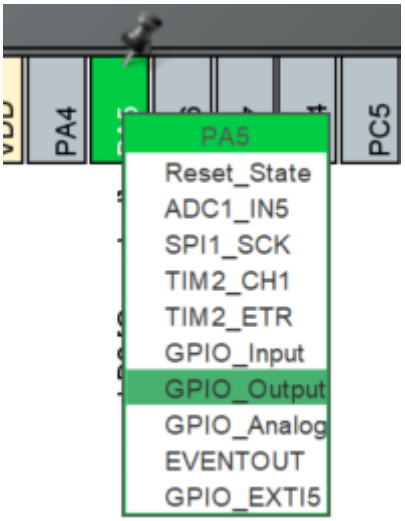
Setting Up the Project

OpenSTM32CubeIDE,select the previously created workspace and create a project with the name*PWM*.If you don't remember how to do it, you can review the previous chapter; the procedure for creating and configuring a project remains the same.

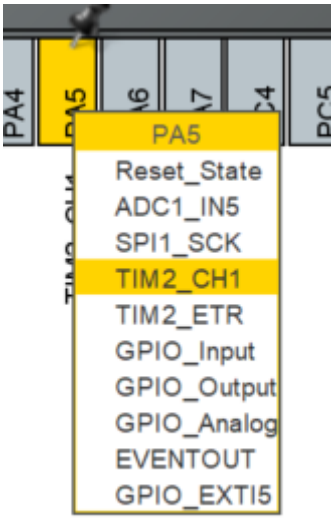
By double clicking on the file*pwm.ioc*let's access the microcontroller configuration.

A5 pin configuration

The LD2 LED is connected to pin A5, so we need to configure the timer this pin refers to. We select pin A5 in the configurator, which, if we chose to proceed with the default configuration during project creation, will be initialized asGPIO_Output.

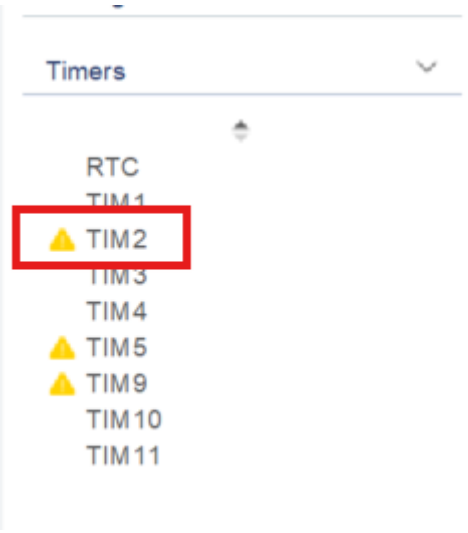


We select channel 1 of timer 2 for this pin

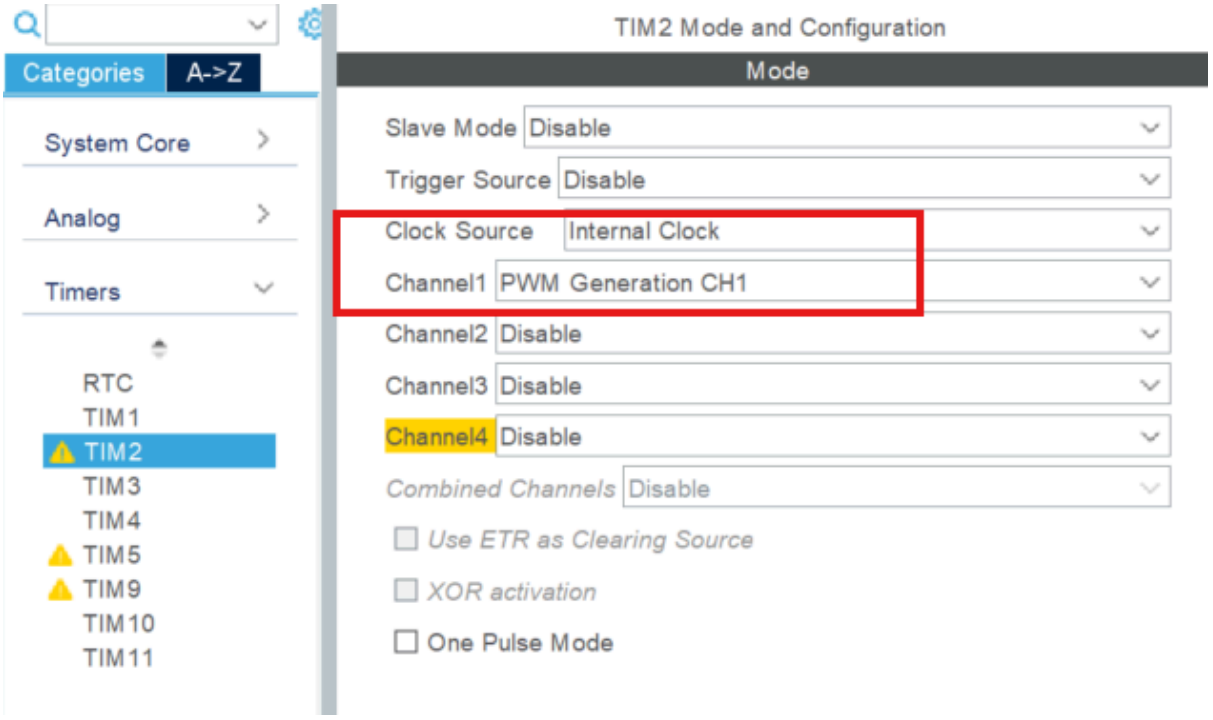


The selection turns yellow because it is necessary to proceed with further configuration steps, at the end of which, if everything has been configured correctly, it will turn green.

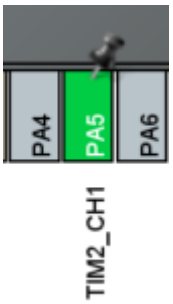
On the menu*Timers*we select the TIM2



In the configuration panel*TIM2 Mode and Configuration*we select for channel 1 the generation of a PWM signal and as*Clock Source* l'Internal Clock.



This configuration is enough to make the PA5 pin green, but the signal definition*mold*desired, it is not yet complete.

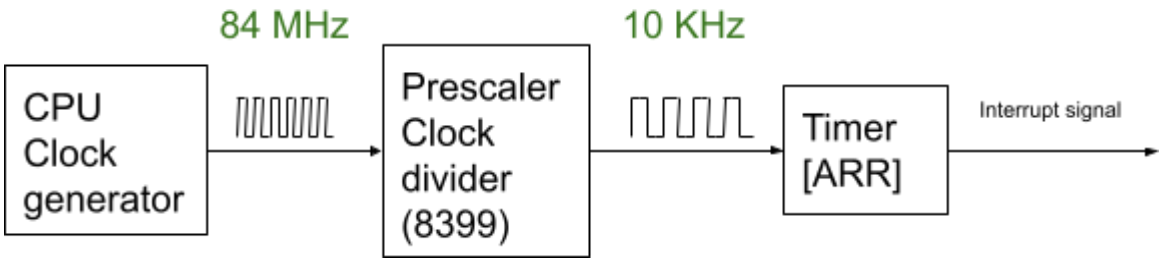


Example 1

As a first case, we want to make the Nucleo board LED turn on for 1 second and turn it off for 1 second.

To achieve this, we need a signal that has a period of 2 seconds with a duty cycle of 50%.

For the*Prescaler*We enter the value 8399 to bring the clock signal (84 MHz) to 10 KHz.



PSC+1 = 8400; lowers 84 MHz to 10 kHz counting. Remember that the prescaler is a register whose value starts from*zero*Entering the value 8399 is equivalent to entering a scaling factor 8399+1=8400.
So at the output of the prescaler we will have a clock signal of frequency:

$$f_{prescaler} = \frac{84 \times 10^6}{8400} = 10^4 = 10\ KHz$$

The signal obtained from the prescaler is therefore a signal that has a period of:

$$T_{prescaler} = \frac{1}{10^4} = 0,0001\ s$$

To calculate the value to be entered into the register*ARR*, that is, the duration of the period of 2 seconds, we need to understand how many pulses of the signal there are*10 KHz*we have to count.

We want the signal to be high for 1 second and low for the remaining second of the period.

We then need to count a number of pulses of the 10KHz signal to make 2 seconds.

$$(ARR + 1) \times T_{prescaler} = 2\,s$$

and therefore

$$ARR = \frac{2}{T_{prescaler}} - 1 = 20000 - 1 = 19999$$

All that remains is to define the*Ton*that is, the duration of the pulse within the period. We do this by inserting the value*9999*in the field*Pulse*.

Indeed

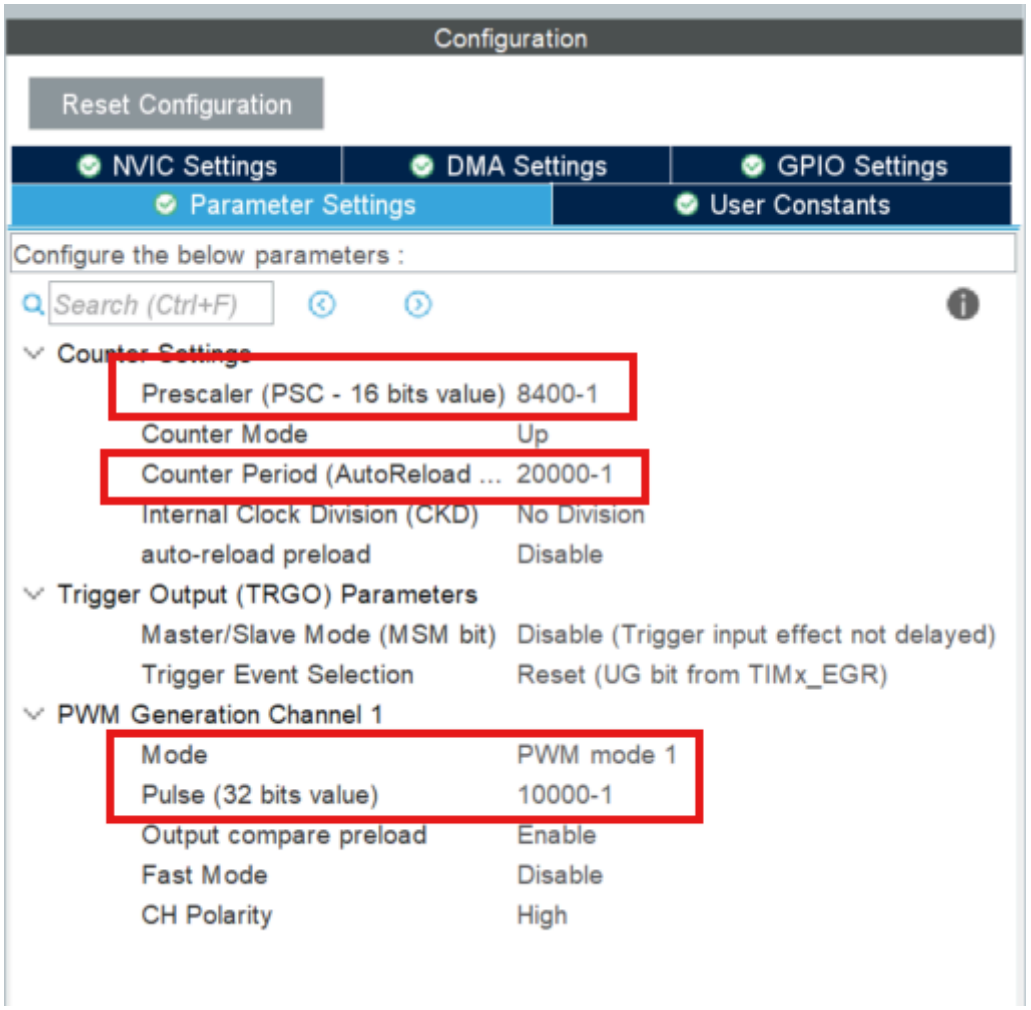
$$Duty\ Cycle = \frac{CCR + 1}{ARR + 1} = \times 100\%$$

from which:

$$CCR = \frac{Duty\ Cycke \times (ARR+1)}{100\%} - 1 = \frac{50\% \times (ARR+1)}{100\%} - 1 = \frac{ARR+1}{2} - 1 = \frac{20000}{2} - 1 = 10000 - 1$$

In CubeIDE, in the Tools panel*Parameter settings*, the value assigned to the CCR register is indicated by*Pulse*.

The following figure shows the panel*Parameter Settings*with the values to be assigned, leaving the others at their default value.



To improve readability, it is good practice to enter the desired value in the registers subtracting one to account for the values starting from zero.

At this point we can generate the code by saving the project and accepting code generation.

All that remains is to add the code to start generating the signal.

```
/* USER CODE BEGIN 2 */
 HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
 /* USER CODE END 2 */
```

Compile and download to the Nucleo as before. You'll see the green LED flash.

Example 2:

Now we want to get an effect*fade-in*on the LED, that is, we want it to start from off and then gradually light up and then start the cycle again.

To get the effect*fade-in*, we need to get the file back*pwm.ioc*and make changes.

We want to bring the PWM frequency to 1 KHz starting from a frequency downstream of the prescaler of 1 MHz

In the *prescaler* we enter the value*84-1*. Indeed:

$$f_{prescaler} = \frac{f_{clock}}{PSC + 1}$$

Therefore

$$PSC = \frac{f_{clock}}{f_{prescaler}} - 1 = \frac{84 \times 10^6}{10^6} - 1 = 84 - 1 = 83$$

As for the value to be entered in the*ARR*, let's start from the frequency of 1 KHz that we want the PWM signal to have.

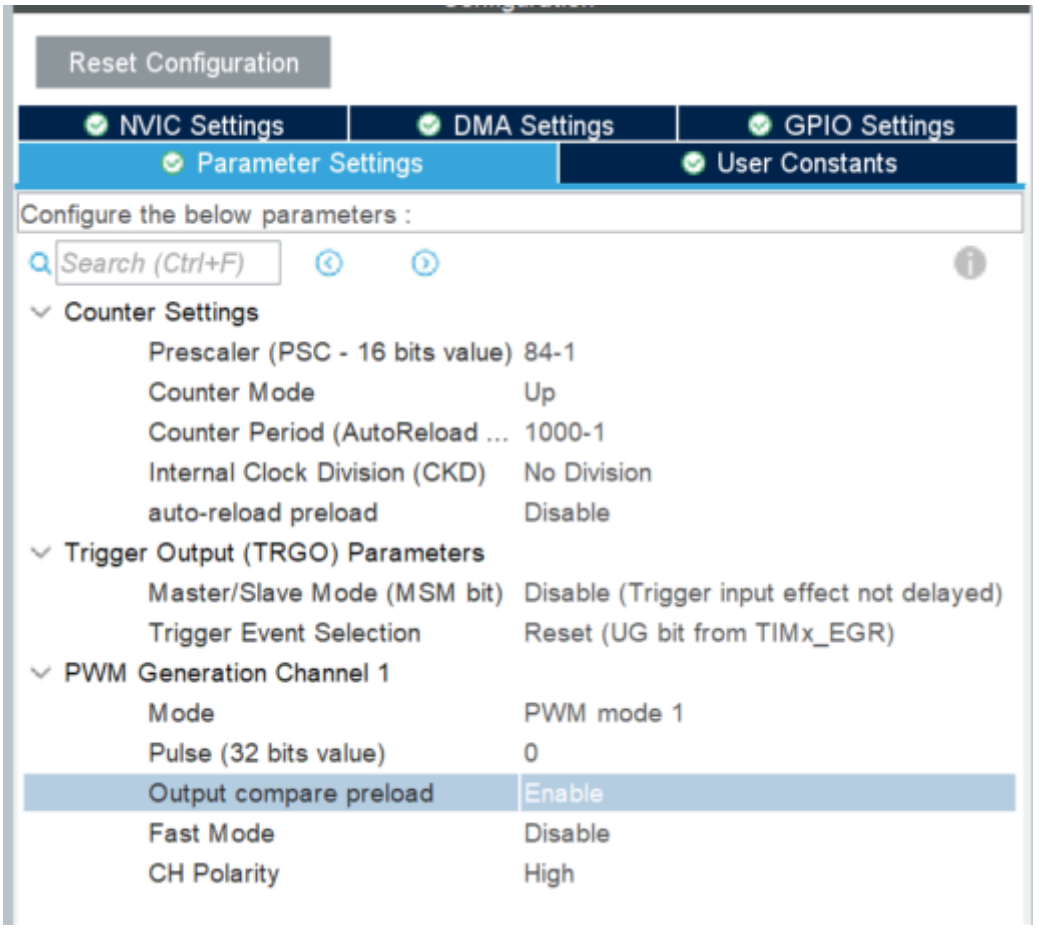
$$T_{mold} = \frac{1}{f_{mold}} = \frac{1}{10^3} s = 10^{-3} s = 0,001\, s = 1ms$$

$$(ARR + 1) \times T_{prescaler} = 1ms$$

$$(ARR + 1) \times 10^{-6} = 0,001$$

$$ARR = \frac{0.001}{10^{-6}} - 1 = 10^3 - 1 = 999$$

So the value to be inserted in the*ARR*it's 999.
As for the*CRR (Pulse)*we leave it to*0*and we will modify it appropriately from code.




```
/* USER CODE BEGIN 2 */
 HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);  // avvia subito il PWM
 uint32_t maxPulse = htim2.Init.Period;  // = 999
 /* USER CODE END 2 */
  /* USER CODE BEGIN WHILE */
 while (1)
 {
        for (uint32_t pulse = 0; pulse <= maxPulse; pulse++)
          {
            __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
            HAL_Delay(1);  // 1 ms per step → fade di ~1 s
          }
        /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
 }
 /* USER CODE END 3 */
}
```

The code starts generating the signal*mold*, and reads the period value from the register*ARR*to assign it to the variable*max Pulse*. The cycle*for*assign to the*ton*that is to*pulse*a value that starts from*0*and reaches the duration of the period, that is, 100% of the*dutycycle*.

At this point, we can generate the code, compile it, and download it to the Nucleo as we did previously. We'll see the green LED's brightness increase from zero to its maximum value.